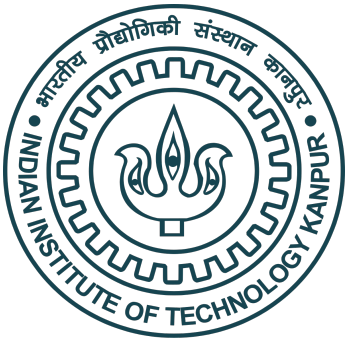
Monocular Metric Depth Estimation Model Fine-Tuned for Dashcam Videos

A Thesis Submitted
In Partial Fulfillment of the Requirements
for the Degree of ***Master of Technology***

Arka Dipta Sarkar

(Roll No.: 241030021)



to the

Department of Civil Engineering (GeoInformatics),
Indian Institute of Technology Kanpur.

November, 2026

Certificate

It is certified that the work contained in the thesis titled **"Monocular Metric Depth Estimation Model Fine-Tuned for Dashcam Videos"**, by **"Arka Dipta Sarkar"** (**Roll No. 241030021**) has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

November, 2026

Prof. Bharat Lohani
Department of Civil Engineering,
Indian Institute of Technology Kanpur.

Declaration

This is to certify that the thesis titled **"Monocular Metric Depth Estimation Model Fine-Tuned for Dashcam Videos"** has been authored by me. It presents the research conducted by me under the supervision of **Prof. Bharat Lohani**.

To the best of my knowledge, it is an original work, both in terms of research content and narrative, and has not been submitted elsewhere, in part or in full, for a degree. Further, due credit has been attributed to the relevant state-of-the-art and collaborations with appropriate citations and acknowledgments, in line with established norms and practices.

Arka Dipta Sarkar
Roll No. 241030021
M.Tech, GeoInformatics
Department of Civil Engineering,
Indian Institute of Technology Kanpur,
Kanpur, 208016, India.

Synopsis

Name of the Student: Arka Dipta Sarkar **Roll No**: 241030021
Degree for which submitted: M.Tech **Department**: Civil Engineering
Thesis Title: Monocular Metric Depth Estimation Model Fine-Tuned for Dashcam Videos
Thesis Supervisor: Prof. Bharat Lohani
Month and Year of Thesis Submission: November 2026

Finding accurate depth maps from a single monocular RGB camera is fundamental to autonomous driving, advanced driver-assistance systems (ADAS), scenario generation and video analytics. LiDAR and RADAR sensors provide accurate metric-quality depth maps but are expensive and the depth maps provided are quite sparse. Dashcams, on the other hand, are already widely installed in millions of vehicles and capture vast unlabeled driving footage every day. [19, 28, 32]

This thesis presents a full pipeline for generating uniform depth maps in absolute meters from a single dashcam frame without the help of any depth estimation sensors at inference time. It is important to note that this thesis focuses exclusively on the KITTI Eigen Split dataset [10, 14, 15], owing to its widespread availability and the limited computational resources available for this study. Nevertheless, the proposed pipeline incorporates the necessary tools and components to facilitate its adaptation to other datasets, including nuScenes, IDD, and TiAND, enabling the generation of similar outputs on these datasets as well [6, 23, 39].

This research follows 9 sequential stages:

First, using Segment Anything Model 3 (SAM3) [7] per-frame instance segmentation masks were generated for vehicles, pedestrians and all the stationary objects and infrastructure for the given datasets. These masks enable per-category depth evaluation for accessing the accuracy of the models.

Second, 5 State-of-the-Art Monocular Metric Depth Estimation pretrained models: UnidepthV2 [31], DepthPro[5], Depth Anything V2 large outdoor[43], Depth Anything V3 Nested Giant V1.1[25] and Depth Anything V3 Metric Large [25] are evaluated on the complete KITTI Eigen-Split test dataset on per-frame per-instance per-category level under raw and various other scale-alignment strategies. Out of these, Depth Anything V3 Metric Large (DA3-M) gave the best results, and is used as the base model for further stages.

Third, in-depth error analysis of DA3-M is done to check for any pattern or bias in the model's output.

Fourth, an optional step is kept for removing internal reflections from the dash-cam videos using DC Gaussian model. [40]

Fifth, a mathematical scale/shift correction is explored using predicted dimensions of vehicles and predicted camera intrinsics as anchors with RANSAC [12] and Weighted Least Squares (WLS) against global average dimensions of the vehicles to improve the depth estimation of the model. The final results show a $5-6\%$ improvement over unscaled predictions, but well under any LiDAR anchored scale-alignment strategies, thus needing to perform fine-tuning of the model for further improvements.

Sixth, an optional step for generating camera-axis depth maps. for nuScenes, TIAND and IDD datasets. KITTI Eigen-Split dataset already had pre-generated depth maps and train-test split benchmark. [10, 14, 15].

Seventh, Fully Supervised Fine-Tuning is performed on the model using LiDAR depth maps as ground truth. For this a comprehensive ablation study is done on the DA3-M covering 13 loss functions, 3 architecture variants, 4 augmentation strategies, and 5 optimizer/scheduler combinations [18, 22, 26, 27, 36, 38]. Further 8 loss function combinations were derived and also tested. The best configuration out of these is finally showcased, which resulted in great improvement in metric depth estimation.[1, 3, 8, 10, 17, 24, 34, 44, 48]

Eighth, Self Supervised Fine Tuning is performed on the model from the consecutive frame data in-order to improve its temporal depth consistency. For this also a comprehensive ablation study is done covering 14 pure SSL, 3 OXTS assisted, 5 stationary object tracking based loss functions and few loss combinations, all of them using a mixture of photometric inverse warping with predicted and oracle pose data with temporal flow consistency and stationary object tracking without LiDAR depth maps. The best configurations out of these is finally used to improve the temporal prediction of DA3-M model, which resulted in major improvement in metric depth estimation but lower than previous strategy.[4, 11, 13, 16, 17, 21, 37, 41, 45, 47]

Ninth, Sequentially combining the Self Supervised with the Supervised Fine Tuning Strategy which resulted in further improvement in the depth estimation.

The thesis concludes with discussion of key findings of this research, limitations that could have been avoided and future directions like implementing the same for other datasets and trying to achieve real time inference.

Acknowledgements

I take this opportunity to acknowledge and express my gratitude to all those who supported and guided me during our project work. I am grateful to my parents for their abundant grace and blessings bestowed upon me, which enabled me to successfully complete this project.

Arka Dipta Sarkar

Contents

Acknowledgements vi

List of Figures xi

List of Tables xii

Abbreviations xv

Symbols xvii

1 Introduction 1
- 1.1 Overview . . . 1
- 1.2 Motivation and Practical Context . . . 3
  - 1.2.1 The Cost Problem . . . 3
  - 1.2.2 Use Cases . . . 3
  - 1.2.3 The Indian Road Environment . . . 6
- 1.3 Research Gaps . . . 6
- 1.4 Study Objectives . . . 7
- 1.5 Thesis Organization . . . 9

2 Literature Review 11
- 2.1 Overview . . . 11
- 2.2 Classical Methods and Their Limits . . . 11
- 2.3 The Beginning of Deep Learning Era . . . 12
- 2.4 Self-Supervised Learning: Removing the LiDAR Dependency . . . 12
- 2.5 Dense Prediction Transformers . . . 14
- 2.6 Foundation Models for MMDE . . . 14
  - 2.6.1 DepthPro . . . 15
  - 2.6.2 UniDepth V2 (UD2) . . . 15
  - 2.6.3 Depth Anything V2 (DA2) . . . 16
  - 2.6.4 Video Depth Anything (VDA) . . . 17
  - 2.6.5 Depth Anything V3 (DA3) . . . 18

2.7 Instance Segmentation: SAM3 . . . 19
2.8 Loss Functions for Supervised Fine-Tuning . . . 19
2.9 Loss Functions for Self-Supervised Fine-Tuning . . . 24
2.10 Metric Depth, Relative Depth, and Evaluation . . . 33
2.11 Geometric Priors and Mathematical Correction . . . 34
2.12 Instance-Level Evaluation and Video Consistency . . . 35
2.13 Reflection Suppression in Dash-Cam Imagery . . . 37
2.14 Research Gaps . . . 37

3 Methodology **39**
3.1 Pipeline Overview . . . 39
3.2 Stage 1: SAM3 Instance Segmentation . . . 40
3.2.1 Model Loading and Offline Mask Generation . . . 40
3.2.2 Prompt Engineering . . . 41
3.2.3 Semantic Categories Used for Segmentation . . . 43
3.2.4 NMS and Mask Quality Filtering . . . 44
3.3 Stage 2: Multi-Model Inference and Evaluation . . . 45
3.3.1 Models and Inference APIs . . . 45
3.3.2 Evaluation Metrics . . . 46
3.3.3 Scale-Alignment Strategies . . . 48
3.4 Stage 3: DA3-M Error Pattern Analysis . . . 50
3.5 Stage 4: Internal-Reflection Removal (Optional) . . . 51
3.6 Stage 5: Vehicle-Anchored Scale/Shift Correction . . . 52
3.6.1 Motivation . . . 52
3.6.2 Camera Intrinsic Estimation via DA3-G . . . 52
3.6.3 Vehicle Prior Specification . . . 53
3.6.4 Orientation-Aware Anchor Depth Estimation . . . 54
3.6.5 RANSAC + WLS Global Scale/Shift Fit . . . 56
3.6.6 Two-Pass Evaluation . . . 57
3.7 Stage 6: LiDAR-to-Depth-Map Generation Templates . . . 58
3.8 Stage 7: Supervised Fine-Tuning . . . 60
3.8.1 Why Decoder-Only . . . 60
3.8.2 Loss Functions . . . 62
3.8.3 Ablation Design . . . 67
3.8.4 Combined Recipe Evaluation . . . 68
3.8.5 Final Fine-Tuning . . . 69
3.9 Stage 8: Self-Supervised Fine-Tuning . . . 71
3.9.1 Motivation and Formulation . . . 71
3.9.2 Pose Estimation . . . 72
3.9.2.1 PoseNet . . . 73
3.9.2.2 Relative Pose from KITTI OXTS . . . 74
3.9.3 Differentiable Inverse Warping . . . 74

3.9.4 Photometric Loss and Auto-Masking . . . 76
3.9.4.1 Edge-Aware Smoothness . . . 77
3.9.4.2 Temporal and Geometric Consistency . . . 77
3.9.5 SSL Loss Registry . . . 78
3.9.6 Data Augmentation and Experimental Setup . . . 80
3.9.7 Combined Recipe Evaluation . . . 81
3.9.8 Final Fine-Tuning . . . 83
3.10 Stage 9: Sequential Combined Fine-Tuning . . . 85

4 Datasets and Evaluation Protocol **87**
4.1 Scope of the Dataset Used . . . 87
4.2 KITTI Eigen-Split . . . 88
4.2.1 Depth representation and valid pixels . . . 88
4.2.2 Training, validation, and test data . . . 89
4.3 Other Dataset Pipelines: Implemented Templates, Not Evaluated Data 90
4.4 Evaluation Protocol . . . 91

5 Analysis and Results **93**
5.1 Overview . . . 93
5.2 Stage 1: SAM3 Segmentation Quality . . . 93
5.3 Stage 2: Baseline Multi-Model Comparison . . . 95
5.3.1 Unscaled Zero-Shot Results . . . 95
5.3.2 Scale-Aligned Comparison . . . 96
5.3.3 Per-Category Analysis (DA3-M) . . . 98
5.4 Stage 3: DA3-M Error Pattern Analysis . . . 101
5.5 Stage 5: Vehicle-Anchored Correction Results . . . 104
5.5.1 Anchor Fit Quality . . . 104
5.5.2 Correction Results . . . 105
5.6 Stage 7: Supervised Fine-Tuning Results . . . 107
5.6.1 Individual Loss Function Ablation . . . 107
5.6.2 Architecture Ablation . . . 109
5.6.3 Augmentation Ablation . . . 111
5.6.4 Optimiser and Scheduler Ablation . . . 111
5.6.5 Combined Loss Ablation . . . 112
5.6.6 Final Fine-Tuning . . . 114
5.7 Stage 8: Self-Supervised Fine-Tuning . . . 118
5.7.1 SSL Loss Function Ablation . . . 118
5.7.2 Stationary-Object Loss Ablation . . . 120
5.7.3 Final Combined Recipe Comparison . . . 120
5.7.4 Detailed Unscaled Metric Comparison . . . 123
5.7.5 Scale-Alignment Comparison . . . 123
5.8 Stage 9: Combined Fine-Tuning . . . 125

5.8.1 Overall Evaluation Results 128
5.8.2 Category-Wise Evaluation 128
5.8.3 Depth-Range Analysis 129
5.9 Summary of Key Results 131

6 Conclusions and Future Work **133**
6.1 Summary of Work 133
6.2 Key Findings 137
6.3 Research Contributions 139
6.4 Limitations 141
6.5 Future Work 142

Appendix A Fully Supervised Loss Function Equations **145**

Appendix B Fully Supervised Ablation Hyperparameter Summary **155**

Appendix C Self-Supervised Loss Function Equations **163**

Appendix D Self-Supervised Ablation Hyperparameter Summary **175**

References **185**

List of Figures

5.1 Overview of the SAM3 instance segmentation mask of one of the KITTI dataset imagery to visualize the scene segmentation 94
5.2 Scale-Aligned comparison and Depth bin analysis for DA3-M 98
5.3 Multi scale-aligned Object-category RMSE across scale-alignment strategies 100
5.4 Image Overview for DA3-M signed-difference distribution at frame level (652 frames, clip80) 102
5.5 Unscaled RMSE by category and clip variant for DA3-M 103
5.6 Only-Scale vehicle anchored Depth Bin Analysis for DA3-M 106
5.7 Scale-Shift vehicle anchored Depth Bin Analysis for DA3-M 106
5.8 Radar chart for all individual loss function ablation study for supervised fine-tuning, normalized, lower area is better 108
5.9 Radar Chart for Architecture ablation study for Supervised fine-tuning, normalized, lower area is better 110
5.10 Depth Bin analysis of fine-tuned DA3-M on loss combination of A4 (50 epochs) 115
5.11 A4 Validation RMSE and Test RMSE per epoch (max epochs 50) . . 116
5.12 A4 Validation RMSE and Test RMSE per epoch (max epochs 100) . . 117
5.13 Radar Chart for all SSL loss functions from ablation study, normalized, lower area is better 119
5.14 Various training curves of A3 loss function for self supervised fine-tuning 122
5.15 Training Curve for A4 Supervised loss combination with the starting checkpoint taken from the best model of A3 Self-Supervised loss combination for Sequentially Combined Fine-Tuning 127
5.16 Depth-range analysis of the combined fine-tuned DA3-M model. The left plot shows pixel-weighted RMSE across depth intervals, while the right plot shows signed bias calculated as predicted depth minus LiDAR depth. 130

List of Tables

3.1 Segmentation categories and their use cases 43
3.2 Error pattern analysis table with range and interpretations 51
3.3 Vehicle prior sub-category specification 53
3.4 Orientation-Aware anchor usability 54
3.5 Ablation Design for Supervised Fine-tuning 68
3.6 Combined Loss Recipes for Supervised Fine-tuning 69
3.7 Combined Loss Recipes for Self-Supervised Fine-Tuning 82

5.1 Unscaled zero-shot performance on the KITTI Eigen-Split test set (652 frames, Eigen crop, $D \in [0.001, 80]$ m). All metrics are computed using pixel-weighted whole-frame evaluation. 95
5.2 Performance of the five depth estimation models under different scale-alignment strategies (652 frames, pixel-weighted whole-frame evaluation). Median, mean and affine alignment use LiDAR ground truth as oracle references. 97
5.3 Unscaled per-category RMSE of DA3-M from the final 652-frame mask-level evaluation. Each category is evaluated over the valid LiDAR pixels within its SAM3 masks. 99
5.4 Results of different Clip variants for DA3-M on the mask-level performance 100
5.5 DA3-M signed-difference distribution at frame level (652 frames, clip80). Positive differences indicate depth underestimation, while negative differences indicate overestimation. 102
5.6 Comparison of vehicle-anchored correction with oracle-based alignment on the 652-frame KITTI Eigen-Split test set. RMSE represents average RMSE, not a pixel-weighted aggregate. Oracle methods use LiDAR ground truth for alignment. 105
5.7 Comparison of individual loss functions using DA3-M decoder-only fine-tuning, base augmentation and AdamW with cosine warmup. Three learning-rate candidates were tested for each configuration, with a maximum of five epochs and early stopping patience of three epochs. RMSE represents pixel-weighted RMSE on the 46-frame held-out test set. 107

5.8 Architecture ablation using MAE-Canonical loss, base augmentation and AdamW with cosine warmup on the 46-frame test set. 109
5.9 Augmentation ablation using MAE-Canonical loss, decoder-only training and AdamW with cosine warmup on the 46-frame test set. 111
5.10 Optimiser and scheduler ablation using MAE-Canonical loss, decoder-only training and base augmentation on the 46-frame test set. 112
5.11 Full-frame KITTI Eigen-Split evaluation of the combined supervised loss recipes (652 frames). RMSE is pixel-weighted, while AbsRel and δ_1 are averaged across frames. 113
5.12 Comparison of the two standalone full-set supervised A4 runs. The 50-epoch result is aggregated from the saved full-frame prediction CSV; the 100-epoch result is from the post-training evaluation of the validation-selected checkpoint. 116
5.13 Selected self-supervised loss-registry ablation results. RMSE is pixel-weighted true RMSE; AbsRel and δ_1 are reported as provided in the evaluation matrix. 118
5.14 Comparison of stationary-object-based self-supervised loss configurations. RMSE represents pixel-weighted true RMSE. 120
5.15 Loss compositions used in the final self-supervised recipe comparison. 121
5.16 Final self-supervised fine-tuning results on the 652-frame KITTI Eigen-Split test set. Checkpoints are selected using validation RMSE. Test RMSE is pixel-weighted. 121
5.17 Detailed unscaled evaluation results for the final self-supervised configurations on the KITTI Eigen-Split test set. 123
5.18 Comparison of PW-RMSE across different scale-alignment methods for the final self-supervised configurations. 124
5.19 Training and evaluation performance of the sequential self-supervised and supervised fine-tuning experiment using Recipe A4 on KITTI Eigen-Split. 127
5.20 Full-frame evaluation of the combined fine-tuned DA3-M model on KITTI Eigen-Split. 128
5.21 Category-wise evaluation of the combined fine-tuned DA3-M model on KITTI Eigen-Split. 129

B.1 Verified settings for the small-subset supervised HPT ablation. 155
B.2 Supervised ablation dimensions and configurations. 157
B.3 Supervised combined-loss recipes defined in the training pipeline. . . 158
B.4 Configuration of the standalone 100-epoch full-set A4 supervised run. 158
B.5 Checkpoint initialisation and principal training settings for the full-set supervised runs discussed in Chapter 5. 161

C.1 Current loss-registry entries and their categories. 174

D.1 Configuration in the supplied stationary-fix SSL HPT notebook. . . . 175

D.2 Saved stationary-object SSL ablation metrics across unscaled and LiDAR-aligned evaluations. Each cell lists RMSE (m), AbsRel, δ_1, and mean error (LiDAR depth minus predicted depth, m), in that order; positive mean error indicates depth underestimation. 179
D.3 Configuration of the final OXTS-based A3 self-supervised training run. 179
D.4 Final SSL recipe definitions in the supplied notebook. 182

Abbreviations

ADAS	Advanced Driver-Assistance Systems
AMP	Automatic Mixed Precision
AV	Autonomous Vehicle
BerHu	Reverse Huber Loss
CNN	Convolutional Neural Network
CV	Coefficient of Variation
DA2	Depth Anything V2 (Metric-Outdoor-Large)
DA3-G	Depth Anything V3 Nested-Giant-Large 1.1
DA3-M	Depth Anything V3 Metric-Large
DoF	Degrees of Freedom
DepthPro	Depth Pro (Apple Research)
DPT	Dense Prediction Transformer
EG-SSI	Edge-Guided Scale-Shift Invariant Loss
FOV	Field of View
GT	Ground Truth
HPT	Hyperparameter Tuning
IDD	India Driving Dataset
IoU	Intersection over Union
KITTI	Karlsruhe Institute of Technology & Toyota Technological Institute
LiDAR	Light Detection and Ranging
LR	Learning Rate
MAD	Mean Absolute Deviation
MAE	Mean Absolute Error
MiDaS	Mixing Datasets for Zero-Shot Cross-Dataset Transfer
MMDE	Monocular Metric Depth Estimation
MRF	Markov Random Field
NMS	Non-Maximum Suppression

OLS	Ordinary Least Squares
OXTS	Oxford Technical Solutions (GPS/IMU unit in KITTI)
RADAR	Radio Detection And Ranging
RANSAC	Random Sample Consensus
RGB	Red-Green-Blue
RMSE	Root Mean Square Error
SAM3	Segment Anything Model 3
SGD	Stochastic Gradient Descent
SILog	Scale-Invariant Logarithmic Loss
SOTA	State of the Art
SSIM	Structural Similarity Index Measure
SSL	Self-Supervised Learning
TGM	Temporal Gradient Matching
TIAND	Traffic and Indian Autonomous Navigation Dataset
UD2	UniDepth V2 ViTL14
ViT	Vision Transformer
VNL	Virtual Normal Loss
WLS	Weighted Least Squares

Symbols

D	Predicted depth map	m
D_{gt}	Ground-truth depth map from LiDAR	m
D_{raw}	Raw network output (before focal-length scaling)	m
$\hat{D}$	Depth map after scale/shift alignment	m
$F_{\min}$	MAE-Canonical depth floor (inverse-depth clamp)	0.5 m
e_i	Depth residual: $\hat{D}_i - D_{\text{gt}}^{(i)}$	m
d_i	Log-depth residual: $\log \hat{D}_i - \log D_{\text{gt}}^{(i)}$	—
n_i	Valid LiDAR pixel count in frame i	px
N	Total valid LiDAR pixel count	px
δ_k	Threshold accuracy at 1.25^k, $k \in \{1, 2, 3\}$	—
s, t	Global scale factor and shift offset	— / m
d_h, d_w	Anchor depth from height/width prior	m
d_{anchor}	Final precision-weighted anchor depth	m
w_h, w_w	Per-dimension anchor precision weights	—
$\mathbf{A}$	WLS design matrix: $[D_{\text{raw}} \mid \mathbf{1}]$	—
$\mathbf{W}$	Diagonal weight matrix in WLS fit	—
f_x, f_y	Focal lengths in x and y axes	px
H_r, W_r	Real-world vehicle height/width prior	m
h_p, w_p	Pixel bounding-box height/width	px
$\mathbf{K}$	Camera intrinsic matrix (3×3)	—
$\mathbf{T}$	Relative camera pose transformation	SE(3)
$\mathbf{R}$	Rotation matrix component of $\mathbf{T}$	SO(3)
$\mathbf{t}$	Translation vector component of $\mathbf{T}$	m
$\boldsymbol{\omega}$	Axis-angle rotation vector (PoseNet output)	rad
$\mathbf{p}$	Homogeneous pixel coordinate $[u, v, 1]^\top$	px
$\mathbf{P}$	Back-projected 3D point	m

$\hat{I}$	Synthesised (differentially warped) frame	—
V	Valid warp mask	—
ε	Numerical stability constant	10^{-6}
λ	Variance weight in SILog loss	0.85
α	SSIM weight in photometric residual	0.85
τ	Static-pixel threshold for TGM/temporal losses	0.05 m
$\mathrm{pe}(\cdot,\cdot)$	Photometric residual: $\alpha\frac{1-\mathrm{SSIM}}{2} + (1-\alpha)\ell_1$	—
$\mathcal{L}$	Loss function (generic)	—
∇	Spatial gradient operator	—

Dedicated to my beloved parents and friends.

Chapter 1

Introduction

1.1 Overview

Depth estimation from a standard RGB image is a basic requirement for autonomous driving and traffic analysis. For obtaining highly accurate metric depth maps, sensors like LiDAR, RADAR and Stereo cameras are used but their high cost and maintenance limits large-scale usage. For example a simple mechanical spinning LiDAR may cost thousands of dollars per unit, strict extrinsic calibration and regular maintenance due to typical outdoor environment conditions. Alternatively consumer dashcams are more practical as they cost a fraction of a LiDAR setup, require no specialized calibrations or maintenance and are already mounted on millions of vehicles across the globe, continuously capturing massive volumes of uncalibrated RGB driving footage.

The high cost and deployment gap for the depth sensors and a massive untapped potential of the dashcams is the starting point of this thesis. If a software model can reliably extract metric depth maps from a single dashcam RGB image input without any additional sensors then every dashcam, in theory, can become a low-cost depth

sensor with minimal calibrations. This process of generating a metric depth map from a single RGB image without any calibration information at the inference time is called *Monocular Metric Depth Estimation (MMDE).*

Recent advances in large-scale vision transformers and dense prediction transformers have made MMDE a reality. Unidepth V1 ,UniDepth V2 [30, 31], DepthPro [5], Depth Anything V2 [43] and Depth Anything V3 [25] are some of the best SOTA models available at the time of this research and they all achieve strong results on standard benchmarks. But the main concern this thesis addresses is *which model is the most accurate and how can they be improved further?* These model's performance on dashcam footage with variable focal lengths, windshield internal reflections, mixed traffic and edge-case conditions are not studied well enough. Getting the model to work well in any general scenario to getting it to work well in various traffic conditions is very important for our use case.

This thesis covers the full pipeline: zero-shot benchmarking five SOTA models, a training free mathematical correction attempt, Fully Supervised and Self-Supervised fine-tuning with a comprehensive ablation study. This thesis work is structured in nine stages, implemented in Jupyter notebooks and evaluated only on the KITTI Eigen-Split benchmarks. All quantitative results are also only reported based on KITTI Eigen-Split dataset, though other configurable LiDAR projection templates are present for additional datasets like nuScenes[6], IDD[39] and TiAND[23] to run on this pipeline too.

1.2 Motivation and Practical Context

1.2.1 The Cost Problem

Purchasing a complete depth estimation sensor setup (like LiDAR, RADAR, Stereo Camera) for an autonomous vehicle company or a research lab in India is not feasible at large scale due to its high initial cost. Dashcams, on the other hand, are cheaply available, already mounted and recording terabytes of footage across the globe. If a fine-tuned MMDE model can extract usable metric depth maps out of these footage, then the same data that is being used for personal security and insurance purposes, can become a source of 3D scene information.

LiDAR depth maps also suffer with sparsity issue. A Velodyne HDL-64E rotating 3D laser scanner, which was used in KITTI dataset, after projection yields around 22,000 valid depth pixels per 1242×375 frame, which is about 5% of the image. The remaining 95% of the image has no depth data. Thus we will not get a uniform scene wide depth maps for better scene understanding. But using this to verify the depth maps generated from the model is still viable. Also we can improve the uniform depth maps generated from MMDE models by either anchoring or supervise training the model with these sparse data, which is what this literature answers later.

1.2.2 Use Cases

The use cases of metric depth out of dashcam imagery are broad. The notable ones are:

- **Video-to-3D simulation generation for autonomous vehicle testing:**
  Dashcam footage from everyday traffic scenarios like cars cutting lanes, cyclists

emerging from blind spots, cattle crossing highways and near-miss events that no synthetic simulator could generate, can be used to reconstruct 3D scenes if depth for each frame is available. A dashcam clip with depth annotation can be lifted into a 3D point cloud sequence, with which 3D scenarios can be synthesized. Then an AV software setup can be tested against these reconstructed scenarios and edge cases without any physical tests on the road. Using real world dashcam footage as a source for generating synthetic test scenarios is sometimes called world model-based simulation which is an important framework for AV safety evaluation and when using only dashcam as input, metric depth for each frame is the missing piece.

- **Forward-collision warning and ADAS on existing vehicles.**

  With a per-frame depth map in absolute meters, a simple system can compute the time to contact from the nearest obstacle. This would require no additional hardware except for the dashcam and a small onboard compute unit, turning any vehicle ADAS enabled.

- **Near-miss and incident detection for fleet safety.**

  Fleet operators like truck companies and cab aggregators use dashcams to monitor their vehicles driving conditions. So if there were any accident or near-miss then instead of manually reviewing the complete footage for each vehicle, they could use a MMDE powered incident detection software for automatically flagging such clips.

- **Accident reconstruction and insurance.**

  Estimation of vehicles speed and inter-vehicle distances are possible with metric depth from the dashcam footage. This information is useful during insurance claim verification and fault determination at the time of any incident.

- **Road condition and pothole mapping.**

  Without using any dedicated survey vehicles, we can use metric depth maps from dashcams on vehicles traversing on roads to detect surface deformations like: potholes, road edge degradation, deformed speed breakers, which is particularly relevant for municipal road maintenance and highway authorities.

- **Traffic monitoring and vehicle counting.**

  Without the use of any fixed infrastructure sensors, per category metric depth enables range based vehicle detection, which can be used counting vehicles within 0 to 30 m.

- **Speed estimation from monocular video.**

  Without using a radar gun, vehicle speeds can be estimated from a fixed dashcam using metric depth and optical flow.

- **Uniform LiDAR anchored depth maps.**

  By anchoring the existing LiDAR data with the models metric depth map can help create highly accurate uniform metic depth maps with proper edge awareness, leveraging LiDAR accuracy and MMDE model's uniformity.

All of these applications have one thing in common: the depth must be expressed in absolute metres and very accurate. A general monocular depth estimation model usually tells relative depth: which tells you that one object is twice as far as another object. But that is not enough to compute the time-to-contact, reconstruction of a 3D scene at real scale, or detect an object is a distance threshold. MMDE is the specific solution that help solve all of the above in a budget friendly manner.

1.2.3 The Indian Road Environment

In this thesis work, the current MMDE models are primarily evaluated and trained on KITTI dataset which comes from Karlsruhe, Germany. This dataset was chosen because of its wide adaption in the research field for benchmarking and evaluation use cases. But this dataset is quite in contrast when compared against Indian road conditions with low traffic density, low traffic diversity, lesser animals and right handed traffic. So for a model fine-tuned on KITTI-like data learns priors about that kind of scene, thus the same model can't be used on an Indian urban road.

In India, the traffic is dense and mixed with two-wheelers, four-wheelers, e-rickshaws, auto-rickshaws, buses, trucks and other types of vehicles, sharing the same left-handed traffic lane. Lighting conditions range from bright tropical sunlight to unlit night-time rural roads.The windshields also pickup a lot of dusts and carry internal reflections. The physical dimensions and appearance of vehicles are also very different compared to European vehicles. So if a model that works well for KITTI, will work in Germany quite well, but the same may fail when deployed on a dashcam in Indian cities.

The best solution for this is *domain-specific fine-tuning*, where the model is fine-tuned on Indian datasets like IDD and TIAND, then re-evaluated for Indian use cases. Although this has not been done in this thesis due limited computational resources at the time of this study but necessary configurable tools and components are provided in the repository, to facilitate future benchmarking and evaluations.

1.3 Research Gaps

This thesis work has these specific research gaps present in the existing literature:

- **Foundation models exist, but their performance on unconstrained dash-cam footage with variable focal lengths is insufficiently studied.**
  Models are being evaluated on clean and calibrated datasets. But when the focal length is unknown, the lens is not clean and the road environment doesn't resemble the training dataset, then how the models behaves is an open question.

- **Per-object-category and per-distance-range depth error characterization of foundation models has not been done.**
  Using aggregate Eigen-crop RMSE doesn't tell us if the model is bad at close-range objects or at far-range background objects. That's why this thesis implemented a SAM3-based pipeline to answer this question.

- **Training-free scale/shift correction using vehicle-size priors has not been rigorously evaluated against oracle intrinsics.**
  The idea for this correction was to use the known dimensions of a vehicle and the predicted camera intrinsic from the model to correct its depth estimation. But in the hind-sight this method was not evaluated for the oracle intrinsics, thus lacking a theoretical maximum recoverable improvement to compare with. Instead it is compared against the raw depth maps and LiDAR-anchored depth maps.

1.4 Study Objectives

- To implement a SAM3 based prompt to instance segmentation pipeline for generating per-category per-object masks on the KITTI Eigen-Split dataset, with 8 semantic super-categories including orientation based vehicle handling.

- To evaluate and the benchmark some of the SOTA (at the time of this study) MMDE foundation models (UD2, DepthPro, DA2, DA3-G, DA3-M) under four

scale-alignment strategies, and to conduct an in-depth per-category and per-range error analysis on the best-performing model (DA3-M).

- To implement and evaluate a vehicle-anchored predicted intrinsic based RANSAC + WLS scale/shift correction with orientation-aware prior selection, and to quantitatively monitor the achievable improvements through training-free mathematical correction.

- To set primary dataset as KITTI Eigen-Split for all the reported experiments and maintain a configurable projection template for nuScenes, IDD and TIAND for future use; these additional datasets are not experimentally evaluated.

- To conduct a comprehensive fully supervised fine-tuning ablation study covering 13 loss functions, 3 architecture variants, 4 augmentation strategies and 5 optimizer/scheduler combinations. Then using its result to formulate the best loss combination recipes for the final fine-tuning task.

- To implement a comprehensive self-supervised fine-tuning ablation study from the consecutive frame data in-order to improve the temporal depth consistency, covering 14 pure SSL, 3 OXTS assisted, 5 stationary object tracking based loss functions and few loss combinations, all of them using a mixture of photometric inverse warping with predicted and oracle pose data with temporal flow consistency and stationary object tracking without LiDAR depth maps.

- To sequentially combining the self-supervised with the supervised fine-tuning strategy and doing a comprehensive evaluation of the fine-tuned model to compare its final performance with standalone strategies.

1.5 Thesis Organization

Chapter 2 is the literature review of all the relevant research papers from classical depth estimation to dense prediction transformers to foundation models bench-marked here and identifying the specific research gaps.

Chapter 3 describes the 9-step methodology used in this thesis, along with the mathematical formulations and engineering decisions.

Chapter 4 discusses the datasets, evaluation protocols and some terminologies.

Chapter 5 presents and analyses all the experimental results.

Chapter 6 finally summarizes all the findings, contributions, limitations, and future works.

Chapter 2

Literature Review

2.1 Overview

This chapter follows the history of monocular depth estimation from its classical beginnings through the self-supervised learning era to the current era of large-scale foundation models. The goal is not to list out every research papers in the field but to show how each of them help create the condition for the next one, and to arrive at a clear understanding of why the specific foundation model and the loss functions are chosen for this thesis. The chapter ends with a summary of the research gaps that this thesis work addresses.

2.2 Classical Methods and Their Limits

Before the era of deep learning, monocular depth estimation were based on hand-crafted image cues like texture gradient, atmospheric perspective, de-focus blur, and geometric constraints like vanishing points. All these cues were combined through

Markov Random Field (MRF) energy minimization over super-pixel grids. But this method worked in controlled conditions and failed in all other conditions. Across different scene types, it performed very poorly and it was not able to produce metric-scale outputs.

2.3 The Beginning of Deep Learning Era

With the introduction of Eigen et al. [10], the deep learning era for this field began. They were the first ones to successfully introduce a deep convolutional neural network for per-pixel depth prediction, using a coarse to fine two scale architecture which was trained end to end. More notably for this thesis, they introduced two things that are still being used till date. First, the training and test split of the KITTI dataset, famously named after them as KITTI Eigen Split dataset. It includes 23,488 training frames and 697 test frames from the 28 non overlapping drive sequences, all evaluated with a top and bottom crop and a depth cap up to 80 m. Every MMDE paper since has reported their results on this split and this thesis primarily using this dataset. Secondly, the Scale-Invariant Logarithmic (SI-Log) loss function, which has become one of the standard loss function for metric depth estimation and is one of the 13 loss functions candidates (Stage 7) evaluated in this thesis.

2.4 Self-Supervised Learning: Removing the LiDAR Dependency

The most obvious bottleneck of the supervised depth estimation is the need for ground-truth depth (dense or semi dense) at the time of training. This means it

will be needing LiDAR setups with careful projection pipelines. Godard et al. [16] introduced a way to remove this dependency for relative depth estimation. They used rectified stereo pairs as their training data, where the left image can be used to reconstruct the right image through predicted disparity map and the photometric reconstruction error is used as the differential training signal. They introduced SSIM + L1 photometric residual:

$$\mathrm{pe}(\hat{I}, I) = \alpha \cdot \frac{1 - \mathrm{SSIM}(\hat{I}, I)}{2} + (1 - \alpha) \left\| \hat{I} - I \right\|_1, \quad \alpha = 0.85 \tag{2.1}$$

This became the standard photometric loss in self-supervised depth estimation field and is one of the 14 pure SSL loss functions candidates (Stage 8) evaluated in this thesis. In their next paper Godard et al. [17] (Monodepth2), they extended this for monocular video without any stereo. In this a separate 6-DOF pose network (PoseNet) [21] predicts the relative ego camera motion between consecutive frames which is combined with the predicted depth map allowing the synthesis of one frame from another frame using differentiable inverse warping. The photometric error between the actual frame and the synthesized frame becomes the training signal. From Monodepth2, two specific contribution are incorporated in this thesis (stage 8). First is the per-pixel minimum re-projection which only takes the minimum photometric error over multiple source frames in order to reduce the occlusion artifacts which occurs when one object blocks the view of another object from a different perspective. Second is the auto-masking which ignores pixels where the unwarped frame is already a better photometric match to the target frame than the synthesized warp, thus blocking the training signal from the static scene pixels when the camera is stationary and from objects moving at the same speed as the camera.

2.5 Dense Prediction Transformers

Earlier CNN-based architectures had a limitation: features in the deeper layers were formed from progressively larger, but still local, receptive fields. Ranftl et al. [33] showed that Vision Transformer (ViT) [9] features can be combined with a dense-prediction head to produce effective spatial predictions, using global attention while recovering detailed output maps. DPT is an influential design in monocular depth estimation, but it should not be treated as the common backbone of all five benchmarked models. DA2 uses a DPT-style decoder with a transformer encoder [43]; DepthPro combines a multi-scale ViT with a DPT-like feature-fusion stage [5]; and DA3 uses a plain transformer backbone with a dual-DPT prediction head [25]. UniDepth V2, in contrast, uses its own camera-conditioned architecture [31]. The full architectures therefore differ, even where DPT-style components appear in the prediction head.

Ranftl et al. [34] introduced a DPT-based architecture to train their model MiDaS on a mixture of heterogeneous datasets. To handle the inconsistent absolute depth scales, they introduced an Affine-Invariant SILog loss function which normalizes the predictions by their median and mean absolute deviation before computing the scale-invariant error. This ultimately removes the global scale and shift ambiguity, enabling training on mixed datasets. This affine-invariant loss function is also one of the 13 loss function candidates (Stage 7) evaluated in this thesis.

2.6 Foundation Models for MMDE

The term foundation model refers to a large-scale model trained on large and diverse datasets at significant computational cost, which is then either directly applied to

downstream task or fine-tuned for a specific task. In MMDE, 5 such models are used in this thesis.

2.6.1 DepthPro

DepthPro [5] is a zero shot depth model developed by Apple Research, which uses a multi-scale ViT architecture with path level slicing in-order to preserve sharp geometric details at the object's boundary. This model was trained in two stages: First training the model on diverse dataset to learn the relative depth estimation, then fine-tuning the model on calibrated dataset to learn metric depth estimation. To predict the camera intrinsics from a single image, a separate focal-length estimation head is also introduced which enable the metric output without any calibration information.

MAE-Canonical loss is the training objective used in DepthPro [5], which computes mean absolute error in canonical inverse depth space with a multi scale gradient term. This loss function is also included in the 13 loss function candidates (Stage 7) Supervised ablation study, and is also one of the best performers of that ablation study. But the implementation has one unique requirement which is that the prediction must be clamped to a minimum of 0.5 m before inversion, because without this floor, the inverse depths near zero reach magnitudes of $\sim 10^6\,\mathrm{m}^{-1}$, which causes the training to diverge by overwhelming every other loss term.

2.6.2 UniDepth V2 (UD2)

UniDepth [30] architecture is engineered to resolve the reciprocal scale ambiguity, where a small near object looks identical to large far object, by directly deriving

lens parameters from the pixel content. It uses a typical ViT-style backbone like DINOv2 [9, 29] as its encoder to extract fine local features and contextual global features. A stop gradient operation is applied to the encoded features flowing into the camera module, preventing camera parameter optimization from dominating the encoder gradients and ensuring encoder remains focused on visual representations. It introduced a self-promptable camera module which predicts a dense camera representation to condition depth features by disentangling depth from the camera model via a pseudo-spherical output representation.

UniDepth V2 [31] simplified the architecture: First, UniDepth V2 makes camera embeddings more parameter efficient with the help of simplified pinhole sinusoidal embeddings, second, it changes the decoder loop from previously used heavy attention-based layers to now with integrated multi-resolution features and convolutional layers, third, it adds an uncertainty map with the output, and finally introduces the Edge-Guided Scale-Shift Invariant (EG-SSI) loss which is an affine alignment of the predicted output to the ground truth, followed by L1 supervision on both global valid pixels and the top 5% highest RGB gradient pixels for fine boundary supervision. In this thesis work, ViTL14 UD2 variant is benchmarked.

2.6.3 Depth Anything V2 (DA2)

Yang et al. [43] introduced a data engine approach, where a large teacher model first annotates unlabeled real world images with pseudo depth labels, which are then combined with photorealistic synthetic images to train the student model. A multi-scale gradient loss is used to improve boundary sharpness. The Metric-Outdoor-Large variant (DA2) is used in this thesis for benchmarking because it outputs in metric depth directly via the HuggingFace AutoModelForDepthEstimation API. Though it

is an older model relative to DA3, it provides a meaningful baseline for comparing the benefit of the newer model.

2.6.4 Video Depth Anything (VDA)

Video Depth Anything (VDA) by Chen et al. [8], extends DA2 for long videos by introducing spatiotemporal head and a temporal consistency loss to reduce frame to frame variations in predicted depth while a key frame based inference strategy enables processing of extended video sequences. These capabilities make VDA relevant to dashcam applications by providing stable depth estimation across consecutive frames which help with scene reconstruction and Bird's Eye View mapping. Its Temporal Gradient Matching (TGM) loss, which enforces logarithmic temporal consistency between consecutive depth predictions for static scene regions, is included as a candidate in the Supervised ablation study (Stage 7).

Temporal consistency does not by itself guarantee metric accuracy, although it can improve stability across long videos. A direct comparison with the image-based models evaluated in this thesis is also not straightforward because VDA is designed for contiguous video sequences, whereas the canonical KITTI Eigen test split is commonly described as 697 selected images drawn from 28 non-overlapping drives. The experiments reported in this thesis used 652 usable KITTI test frames, as described in Chapter 4; these frames do not constitute a single continuous drive sequence. The released VDA implementation considered in this study was intended primarily for inference, which also made downstream fine-tuning less straightforward. For these reasons, VDA was not evaluated in this thesis.

2.6.5 Depth Anything V3 (DA3)

Lin et al. [25] extends upon the Depth Anything framework to any-view inputs like monocular, stereo and multi-view through a unified Transformer architecture. Their one of the key contribution includes a depth-ray prediction target which implicitly encodes camera geometry supervision via surface normals. DA3 outperforms DA2 on monocular benchmarks in both indoor and outdoor domains.

This thesis uses two variants of DA3. **DA3-Metric-Large (DA3-M)** is the primary inference and fine-tuning model in the thesis. The metric depth is computed from the raw output using:

$$D_{\text{metric}} = \frac{f_x \times D_{\text{raw}}}{300}, \quad f_x = 721.5377 \text{ px (KITTI default)} \tag{2.2}$$

where 300 is the fixed focal-length normalisation factor used by the model's metric-depth conversion. For the KITTI experiments in this thesis, the conversion uses the KITTI reference focal length $f_x = 721.5377$ pixels. Applying the same conversion to a different camera requires that camera's focal length to be known or estimated; the conversion is therefore not independent of camera intrinsics. All inputs are resized to dimensions compatible with the ViT patch size of 14. **DA3-Nested-Giant-Large-1.1 (DA3-G)** is a higher-capacity model that produces affine-invariant relative depth and can predict camera intrinsics per sequence. In this thesis, DA3-G is used for benchmarking and camera intrinsic estimation for stage 5. Its relative depth output requires oracle scale/shift alignment before it can be used as a metric estimate and is not suitable for direct deployment.

2.7 Instance Segmentation: SAM3

SAM3 [7] is the prompt-able concept-segmentation model that takes natural language text prompt and image as input and returns binary instance masks of the corresponding image regions with associated IoU confidence scores, making it well-suited for this thesis to do object-category-level evaluation. The masks are generated once and reused across all the downstream tasks without re-running the model.

SAM3 is loaded via `build_sam3_image_model` and `Sam3Processor` from Facebook Research's `sam3` package. Eight semantic super-categories are defined, each with text prompts engineered for dashcam conditions. Masks are saved as binary PNGs with a JSON manifest per frame and a JPG overview.

2.8 Loss Functions for Supervised Fine-Tuning

Stage 7 of this thesis evaluates 13 loss functions for supervised fine-tuning. Understanding why this ablation is necessary requires understanding a specific gap in the literature: *there is no published research work on which loss function works best for the supervised fine-tuning of DA3 models under the LiDAR supervision.* In all these literature there are loss functions developed for training from scratch (SILog, BerHu), those designed for mixed-dataset relative depth training (Affine-Invariant SILog), those specific to particular models (MAE-Canonical for DepthPro, EG-SSI for UD2), those designed for geometric or temporal consistency (VNL, TGM), and other loss individual functions with their combinations, so all of them have different properties that may or may not work for the fine-tuning LiDAR depth maps. Thus a systematic comparison of all these configurations a controlled environment is the gap this ablation addresses.

A brief overview of the loss functions from the literature are given below:

- **MAE-Canonical** [5]:
  - MAE-Canonical Loss is a modified Mean Absolute Error (MAE) loss used in depth estimation.
  - It measures the absolute difference between predicted depth and ground truth depth after converting them into a canonical (normalized) scale.
  - Canonicalization means converting depth values into a common standardized scale before computing error.
  - Requires a 0.5 m depth floor before inversion.
  - This reduces errors caused by: different camera settings , scene scale variations , global depth shifts.
  - This helps the model learn depth more consistently across different scenes and cameras.
- **SILog - Scale-Invariant Log Loss** [10]:
  - SILog Loss is used in depth estimation to compare the predicted depth with the ground truth depth while being less sensitive to overall scale differences.
  - It focuses more on the relative depth relationships between objects rather than exact absolute values.
  - SILog Loss checks whether the model understands which objects are nearer or farther, instead of only checking exact meter-level accuracy.
- **BerHu - Reverse Huber Loss** [24]:
  - BerHu Loss combines the advantages of:

    - L1 Loss for small errors: stable training and less sensitive to noise.
    - L2 Loss for large errors: strongly penalizes large mistakes and helps fix bad depth predictions.
  - It is widely used in depth estimation because it handles both normal pixels and large depth mistakes effectively.

- **SSIM + L1** [16]:
  - It combines: L1 Loss which preserves accurate pixel-wise depth values & SSIM Loss which preserves structural and edge information.
  - This helps the model produce depth maps that are both: numerically accurate and visually consistent.
  - L1 Loss: Measures absolute pixel-wise error.
  - SSIM Loss: SSIM (Structural Similarity Index) measures similarity in structure, contrast and brightness between predicted and ground truth depth maps.
  - SSIM value ranges from: 1→ perfect similarity to 0→ very different.

- **Masked Depth Loss** [25]:
  - Masked Depth Loss computes the depth error only on valid pixels and ignores invalid or missing regions.
  - This is very useful in datasets like LiDAR depth maps where many pixels contain missing depth, zero values and sparse measurements.
  - This prevents the model from learning incorrect information from missing depth regions.

- **Robust Cauchy Loss**[1]:

  - Cauchy Loss is a robust loss function that reduces the effect of very large errors (outliers) during training.
  - It behaves: like L2 loss for small errors but grows slowly for large errors
  - This makes training more stable in noisy depth estimation tasks.

- **Log-L1 Loss**[3]:
  - Log-L1 Loss computes the absolute error in logarithmic depth space instead of normal depth space.
  - It helps the model focus more on: relative depth differences and proportional depth changes rather than only absolute meter-level errors.

- **Virtual Normal Loss (VNL)** [44]:
  - Virtual Normal Loss (VNL) helps the model learn the 3D geometric structure of a scene instead of only pixel-wise depth values.
  - It compares the surface normals computed from predicted depth and ground truth depth.
  - This improves geometric consistency, planar surfaces and object structure
  - Surface Normal Computation: Using three sampled 3D points from the depth map: $n = (p_2 - p_1) \times (p_3 - p_1)$ where p_1,p_2,p_3 = 3D points, $\times$ = cross product, n = surface normal vector (Normals are computed for predicted depth and ground truth depth)

- **Affine-Invariant SILog** [34]:
  - Affine-Invariant SILog Loss is an improved version of SILog Loss that becomes robust to both: scale changes and shift changes between predicted and ground truth depth.

  - It helps the model focus more on the true scene geometry instead of absolute depth alignment.
  - Affine Transformation: Before computing loss, the predicted depth is aligned using: $\hat{y}' = s\hat{y} + t$ where s= scale factor , t= shift factor , $\hat{y}'$= aligned predicted depth. This compensates for global depth mismatch.
  - SILog Computation: After alignment: $d_t = log(\hat{y_t}') - log(y_t)$

- **TGM** [8]:
  - Temporal Gradient Matching (TGM) Loss helps maintain temporal consistency between consecutive video frames in depth estimation.
  - It ensures that depth changes smoothly over time instead of producing flickering or unstable predictions.
  - This loss is mainly useful for sequential datasets and video-based depth estimation
  - Temporal Gradient : The temporal depth change between two consecutive frames is:$G_t = D_t - D_{t-1}$
    Where D_t= depth at current frame and D_{t-1}= depth at previous frame

- **EG-SSI** [31]:
  - Edge-Guided SSI Loss helps the model preserve: object boundaries, sharp edges and structural details, while also being scale and shift invariant (SSI).
  - It is mainly used in depth estimation to improve depth discontinuities around objects.
  - SSI Part handles scale mismatch and improves relative depth learning

- Scale and Shift Invariant (SSI): Before computing loss, the predicted depth is aligned to the ground truth using: $\hat{y}' = s\hat{y} + t$ where s=scale factor, t=shift factor. This removes global scale mismatch.
- Edge Guidance focuses more on boundaries and preserves sharp object transitions.
- Edge-Guided Weighting: Pixels near edges receive higher importance. The edge weight is usually computed from image gradients: $w_i = |\Delta I_i|$ where ΔI_i= image gradient magnitude (stronger edges $\rightarrow$ larger weights)

Complete mathematical definitions of all loss function variants are given in Appendix A and the summary for ablation and hyperparameter study is given in Appendix B.

2.9 Loss Functions for Self-Supervised Fine-Tuning

Unlike supervised fine-tuning, which directly compares predicted depth with LiDAR measurements, self-supervised fine-tuning does not require ground-truth depth maps as training targets. It instead exploits the geometric relationship between consecutive frames of a video sequence. The underlying principle is that given accurate depth predictions and relative camera motion, a source frame can be geometrically transformed to reconstruct the target frame from its viewpoint. This reconstructed image should then agree with the original target image, and the difference becomes the loss signal that needs to be minimized. This framework follows established approaches to monocular self-supervised depth estimation [16, 17, 47].

In this thesis, Stage 8 investigates 22 self-supervised loss functions through an ablation study on the DA3-M model. All experiments follow a common training and view-synthesis setup. Each training sample consists of a triplet of consecutive

frames (I_{t-1}, I_t, I_{t+1}) from the same drive, where I_t is the target frame and I_{t-1} and I_{t+1} serve as source frames. DA3-M predicts the target depth map $\hat{D}_t$ clamped to the range $[10^{-3}, 80]$ m. The relative camera pose between the target and each source frame is obtained either from a learned PoseNet or from the KITTI OXTS GPS/IMU unit.

The PoseNet follows the architecture used in the approaches of Zhou et al. [47] and Godard et al. [17]. It uses a ResNet-18 encoder from He et al. [20], initialized with ImageNet pretrained weights, and takes a concatenated pair of frames as input and outputs a six-degree-of-freedom (6-DoF) relative camera transformation. Although its name follows the pose-regression network introduced by Kendall et al. [21] but it estimates relative camera motion rather than absolute camera pose.

For each source frame, the target frame pixels are first back-projected into 3D using the predicted depth map and camera intrinsic matrix $\mathbf{K}$. The resulting 3D points are then transformed into the source-camera coordinate system using the corresponding relative transformation $\mathbf{T}_{t\to s}$ and re-projected onto the source image plane. Differentiable bilinear sampling is subsequently used to warp the source frame into the target viewpoint, producing a reconstructed target image. A valid-pixel mask V identifies pixels that are projected within the image boundaries and in front of the camera. The photometric discrepancy between the reconstructed and original target images, together with other spatial, temporal, geometric, feature-space, and stationary-object motion constraints, provides the supervisory signals used by the different loss functions investigated in this study.

All geometric transformations, image warping operations, and loss computations are performed in single precision. Furthermore, the self-supervised training pipeline is explicitly isolated from ground-truth depth supervision means the training loop does

not pass LiDAR measurements to any loss function. This ensures that ground-truth depth measurements cannot influence the self-supervised experiments.

The principal loss functions investigated in the ablation study and their respective roles are described below:

- **Photometric L_1 Loss** [13]:
  - This loss measures the pixel-wise absolute difference between the reconstructed target image and the original target image. Only pixels with valid projections inside the source image are considered. The channel-wise absolute differences are averaged and subsequently reduced over the valid image region. This provides a direct image-reconstruction objective and serves as the simplest photometric baseline in the ablation study.
- **Photometric SSIM Loss** [41]:
  - Instead of relying only on absolute pixel differences, this loss measures the structural similarity between the reconstructed and target images. It compares local luminance, contrast, and structural information, making the reconstruction objective less dependent on exact pixel-wise intensity agreement. The SSIM dissimilarity is evaluated over local image windows and averaged over valid projected pixels.
- **Combined SSIM–L_1 Photometric Loss**[16]:
  - This loss combines structural similarity with pixel-wise L_1 reconstruction error. The SSIM component captures local structural differences, whereas the L_1 component penalizes absolute intensity differences. The

implementation uses a weighting coefficient of $\alpha = 0.85$ for the SSIM component. Combining the two terms provides a more balanced reconstruction objective than either component alone.

- **Minimum Re-projection Photometric Loss** [17]:
  - When multiple source frames are available, this loss computes the photometric reconstruction error for each source and selects the minimum error at every pixel, rather than averaging the errors across all source frames. This allows the model to select the source view that provides the most reliable reconstruction for a particular pixel. The implementation also incorporates auto-masking, which excludes pixels for which the identity mapping (without camera-motion-based warping) produces a lower reconstruction error than the warped reconstruction. This helps suppress the influence of pixels affected by camera-independent appearance changes or objects moving independently of the camera.
- **Multi-Scale Photometric Loss**[17]:
  - This variant evaluates the minimum re-projection photometric objective at multiple image resolutions. A four-level image pyramid is constructed by progressively down-sampling the target and source images, while the predicted depth maps and camera intrinsics are adjusted to the corresponding resolutions. The photometric losses obtained at the four scales are then averaged. This encourages reconstruction consistency across different spatial resolutions and allows the loss to capture both coarse scene structure and finer image details. The implementation uses an image-pyramid formulation rather than multiple depth-decoder outputs.
- **Edge-Aware Smoothness Loss**[16]:

  - Photometric reconstruction alone may not provide sufficient constraints in texture-less regions, where several depth configurations can produce similar image appearances. To regularize such regions, this loss penalizes spatial variations in the mean-normalized inverse depth (disparity). The regularization is weighted by image gradients so that smoothing is encouraged in homogeneous image regions but reduced near strong intensity boundaries. This helps preserve depth discontinuities corresponding to object boundaries and scene structure. It is used as an additional regularization term rather than an independent depth-supervision objective.

- **Geometric Consistency Loss**[4]:

  - This loss enforces consistency between depth predictions obtained from consecutive frames. The predicted depth of a target-frame pixel is used with the relative camera pose to project the corresponding 3D point into the next frame. The depth sampled from the next-frame prediction at the projected location is compared with the geometrically expected depth after applying the camera transformation. A normalized depth discrepancy is computed over valid projections. This provides a geometric constraint across frames and reduces inconsistencies between depth predictions that cannot be explained by the estimated camera motion.

- **Temporal Warp Consistency Loss** [4, 47]:

  - This loss also compares depth predictions across consecutive frames using pose-based re-projection. The predicted depth from the next frame is sampled at the re-projected pixel locations and compared with the expected depth obtained by transforming the corresponding 3D points. Invalid projections and non-positive depth values are excluded from the

computation. The implementation supports an additional photometric reconstruction component, however, the temporal-only configuration sets its photometric contribution to zero so that the objective focuses on depth consistency across the two frames.

- **Naive Temporal Depth Difference Loss**:
  - This variant directly penalizes differences between depth predictions from consecutive frames. It is included as a diagnostic baseline to investigate whether temporal depth regularization without explicit camera-motion compensation can improve consistency. However, it assumes that corresponding pixels should maintain identical depth values between frames. This assumption is geometrically incorrect when the camera is moving, even if the observed scene is static. Therefore, this loss does not account for the apparent depth changes caused by camera motion and is treated as a deliberately simplified temporal baseline rather than a physically consistent geometric constraint.
- **Rigid-Flow Consistency Loss**[45]:
  - This loss derives a dense rigid-flow field from the predicted depth map, camera intrinsics, and relative camera pose. The projected displacement of each target-frame pixel into the next frame defines the rigid flow. The loss combines the SSIM+L_1 photometric reconstruction error of the warped source frame with an edge-aware smoothness penalty on the magnitude of the rigid-flow field. The implementation uses flow induced by depth and camera motion but does not contain a separately learned optical-flow branch and should therefore not be interpreted as a complete implementation but an inspiration of GeoNet.

- **Feature-Metric Loss**[37]:
  - Pixel-level photometric reconstruction may be sensitive to illumination changes, local appearance variations, and imperfect image alignment. To introduce a feature-space constraint, this loss compares representations extracted from the target and warped source frames using a frozen ImageNet-pretrained ResNet-18 feature extractor, up to its first residual stage. The objective combines the masked L_1 difference between the extracted feature maps with the pixel-space photometric reconstruction error, assigning equal weights to both components. This is a simplified feature-metric implementation inspired by FeatDepth and does not reproduce its learned feature encoder exactly.

Stationary-object losses:

Dashcam scenes contain many objects that are fixed in the world, such as poles, signs, trees and parked vehicles, and their 3D positions between frames must follow the ego-motion exactly. These losses use the SAM3 masks and require each object to keep an identity over time. Three mask trackers were implemented: a greedy association by mask IoU, centroid distance and area ratio, following the association step of SORT [2]; a globally optimal Hungarian assignment whose cost also includes a colour-histogram appearance term, in the spirit of DeepSORT [42]; and a two-stage variant with Kalman-predicted centroids that retains low-confidence detections, adapted from ByteTrack [46]. The Hungarian tracker was used in the experiments. This was a configuration choice and not the outcome of a tracking evaluation, as no tracking ground truth exists for these masks.

- **Stationary Depth-Motion Loss**:

  - This loss investigates whether objects identified as stationary in the world can provide additional geometric constraints. Matched object instances across consecutive frames are used to obtain representative object locations and median predicted depths within their masks. The current-frame 3D object point is transformed using the estimated camera motion, and its expected depth in the next frame is compared with the observed depth obtained from the corresponding object in that frame. The resulting depth discrepancy is aggregated across the matched stationary objects.

- **Stationary Uniform-Motion Loss**:

  - This loss assumes that objects stationary in the world should exhibit motion consistent with the camera movement. It evaluates the dispersion of signed depth-motion residuals across the matched stationary objects within a sample. In the final ablation implementation, the dispersion is measured using the mean absolute deviation from the median. This provides an object-level consistency constraint, while the earlier variance-based formulation remains available as an alternative configuration.

- **Full 3D Ego-Motion Loss**:

  - Rather than comparing only the depth component, this loss evaluates the complete 3D displacement of matched stationary objects. The 3D point of an object in the current frame is transformed into the next camera coordinate system using the relative camera pose. It is then compared with the independently reconstructed 3D point obtained from the corresponding object in the next frame. The Euclidean distance between these two points provides the consistency measure.

- **Mask Point-Cloud Chamfer Loss**:

  - This loss extends stationary-object consistency from representative object points to the point-cloud level. Valid depth pixels within the matched instance masks are back-projected to form 3D point clouds for the current and next frames. The current-frame point cloud is transformed into the next camera coordinate system, after which a bidirectional nearest-neighbor Chamfer distance is computed between the transformed and observed point clouds. The number of sampled points is capped to control computational cost.

Oracle-pose variants:

In addition to the above loss functions, the ablation includes pose-assisted variants in which the relative camera transformation is supplied by KITTI OXTS measurements rather than estimated by the PoseNet-style network. They do not use LiDAR depth as a training target, however, since the camera motion is supplied from external measurements, they are treated as separate pose-assisted diagnostic experiments rather than predicted-pose self-supervision.

- **GT-pose photometric re-projection**:
  - The minimum re-projection loss with OXTS poses.
- **GT-pose rigid-flow consistency**:
  - The rigid-flow loss with the OXTS pose to frame $t + 1$.
- **GT-pose temporal warp consistency**:
  - The warped-depth consistency with the OXTS pose and the dynamic-object mask.

Complete mathematical definitions of all the self-supervised loss functions are given in Appendix C, and the corresponding hyperparameter and ablation summary in Appendix D.

2.10 Metric Depth, Relative Depth, and Evaluation

A central difference in monocular depth estimation is whether the output is metric or relative. Relative depth systems may preserve ordering and scene structure while returning values whose scale is not tied to meters. Metric depth systems attempt to predict physical distance, but their output can still be affected by camera intrinsics, training-domain bias, and the range represented in the training data. This difference is important in driving applications where a visually plausible depth map is not sufficient if downstream calculations require distances in meters.

The KITTI benchmark is especially relevant to this thesis because it pairs forward-facing images with LiDAR measurements and supports evaluation using the Eigen split [10, 15]. Since projected LiDAR is sparse, evaluation is performed at valid reference pixels rather than assuming that every image pixel has a ground-truth value.**AbsRel** normalizes each residual by its reference depth, and threshold accuracy such as δ_1 measures the fraction of predictions within a multiplicative tolerance.

For RMSE, this thesis adopts a **pixel-weighted aggregation** strategy, referred to as Pixel-Weighted RMSE (PW-RMSE), instead of calculating a simple arithmetic mean of the per-image RMSE values. Let RMSE_i denote the root mean squared error computed over the valid LiDAR pixels of the i-th evaluation image, and let n_i represent the corresponding number of valid reference pixels. The aggregate metric

is computed as

$$\text{RMSE}_i = \sqrt{\frac{1}{n_i}\sum_{j=1}^{n_i}(\hat{D}_{ij} - D_{ij})^2}, \tag{2.3}$$

$$\text{PW-RMSE} = \sqrt{\frac{\sum_{i=1}^{N} n_i \,\text{RMSE}_i^2}{\sum_{i=1}^{N} n_i}}, \tag{2.4}$$

where D_i is the LiDAR-derived reference depth, $\hat{D}_i$ is the prediction, and N is the total number of evaluation images. Consequently, images containing more valid reference pixels contribute proportionally more to the final metric, avoiding the equal weighting of images with lower numbers of valid LiDAR measurements. PW-RMSE therefore emphasizes large absolute depth errors while providing a dataset-level measure that accounts for the varying density of valid reference pixels. Reporting multiple metrics avoids ranking models using one statistic alone.

Post scale alignment must also be interpreted carefully. Median scaling and affine scale-shift fitting can improve numerical scores by using reference LiDAR depth at evaluation time. Such oracle alignment is useful for diagnosing whether an error is largely global scale or shift, but it is not equivalent to a model predicting metric depth without reference measurements. This thesis therefore treats unscaled results as the primary comparison and reports oracle-aligned scores separately.

2.11 Geometric Priors and Mathematical Correction

Monocular depth estimation from a single image is inherently challenging because the appearance of a scene depends on both its geometry and the camera parameters.

One possible way to estimate distance is to use objects of known dimensions as reference points. Under the pinhole camera model, the apparent size of an object decreases as its distance from the camera increases, although this relationship is also influenced by focal length and object orientation. In practice, however, relying on object dimensions can be difficult because of inaccurate segmentation boundaries, variations in object sizes within the same category, occlusions, and uncertainties in camera intrinsics. Techniques such as RANSAC can help identify and reject inconsistent correspondences, while weighted least squares can limit the influence of less reliable reference objects [12]. These considerations provide the motivation for the vehicle-anchored experiment in this thesis. However, whether this approach actually improves depth estimation needs to be determined through experimental evaluation as done in this thesis (Stage 5).

2.12 Instance-Level Evaluation and Video Consistency

Whole-image evaluation metrics do not always reveal how well a model performs across different object categories. For example, a model may predict road depth reasonably well while producing larger errors for pedestrians, vehicles, or thin structures. To examine these differences, instance masks can be used to calculate the same depth metrics over specific semantic regions of an image. Promptable segmentation models from the SAM family make it possible to generate such masks using text prompts. However, the quality of the segmentation masks and the way object categories are defined need to be considered when interpreting the resulting metrics [7, 35]. In this thesis, these masks are used only to analyze category-wise depth estimation performance and are not used to provide depth supervision during training.

Video sequences offer another way to constrain depth estimation, as consecutive frames capture overlapping parts of a scene from slightly different camera positions. Photometric self-supervision takes advantage of this relationship by using the predicted depth and relative camera pose to reconstruct a target frame from a neighboring source frame. The difference between the reconstructed and original images is then used to calculate the photometric loss. Monodepth2 introduced minimum re-projection and auto-masking strategies to reduce errors arising from occlusions, moving objects, and pixels whose appearance cannot be explained by camera motion alone [17]. Although these objectives help maintain consistency across frames, their effectiveness depends on accurate pose estimation and can be affected by changes in illumination, dynamic objects, and violations of the static-scene assumption. Therefore, a reduction in photometric loss does not necessarily mean that metric-depth accuracy has improved. Both aspects need to be evaluated independently.

Some video-based approaches also focus on maintaining consistency between depth maps predicted for consecutive frames. Temporal Gradient Matching is one such objective explored in the broader Video Depth Anything work [8] and it is included in the ablation study of the supervised fine-tuning experiment. Also temporal and photometric loss terms are examined as part of the self-supervised fine-tuning experiments, including configurations that use KITTI OXTS-derived poses. Since the pose source and the combination of loss functions vary across experiments, they are treated as part of the experimental setup rather than as a common configuration applicable to every self-supervised run.

2.13 Reflection Suppression in Dash-Cam Imagery

Internal reflections, particularly those caused by the vehicle windshield, can affect the visual quality of dash-cam footage and introduce unwanted patterns into subsequent depth estimation. To address this issue, this thesis considers DC-Gaussian [40], a 3D Gaussian Splatting method designed to handle reflections and obstructions in dash-cam videos. In the implemented pipeline, COLMAP is first used for feature extraction, image matching, camera pose estimation through Structure-from-Motion (SfM), and image undistortion. The resulting data are then used to train DC-Gaussian with its obstruction modeling configuration, followed by rendering the reconstructed scene from the original camera viewpoints. This produces reflection-suppressed images while retaining the original frame correspondence and filenames, making them suitable for subsequent depth estimation. However, this pre-processing step is not necessary for every dataset. For instance, the KITTI dataset used in this thesis employs a roof-mounted, forward-facing camera setup rather than an in-cabin dash camera recording through the windshield. So, the windshield reflections addressed by DC-Gaussian are not a relevant source of corruption in the KITTI Eigen Split images used in these experiments. The reflection-removal stage is therefore optional and is primarily intended for datasets collected using cameras positioned inside vehicles, where windshield reflections are visibly present.

2.14 Research Gaps

The four gaps that this thesis directly addresses are:

- Systematic per-object-category and per-distance-range depth error characterization of current MMDE foundation models on dashcam data, using instance segmentation masks, is absent from the literature.

- The LiDAR-free scale correction using semantic vehicle-size priors with orientation-aware anchor selection has not been evaluated quantitatively against oracle scale-alignment bounds on the KITTI benchmark.

- No comprehensive ablation exists for supervised and self-supervised fine-tuning of DA3 models on sparse KITTI LiDAR ground truth, covering different loss functions, architecture depth, augmentation strategies, and optimizer choices jointly.

- The interaction between supervised (metric accuracy) and self-supervised (temporal consistency) fine-tuning regimes for MMDE foundation models-and whether stacking them yields additive gains-has not been studied.

Chapter 3

Methodology

3.1 Pipeline Overview

The repository workflow is organized into nine stages: (1) SAM3 instance segmentation, (2) zero-shot model benchmarking, (3) error-trend analysis, (4) optional reflection removal, (5) vehicle-anchored correction, (6) optional LiDAR projection templates for other datasets, (7) supervised fine-tuning, (8) self-supervised fine-tuning, and (9) sequential combined fine-tuning. The quantitative results reported in this thesis use KITTI Eigen-Split; the reflection-removal and additional-dataset projection notebooks are included as optional or reusable templates, not as completed experiments in the reported evaluation.

- **Stage 1:** SAM3 instance segmentation on the KITTI Eigen-Split dataset with per-category mask generation for eight semantic super-categories.
- **Stage 2:** Zero-shot benchmarking of five MMDE foundation models under four scale-alignment strategies, with per-category and per-range error analysis.

- **Stage 3:** In-depth signed-error characterization of DA3-M to identify the directional underestimation bias and its range dependence.
- **Stage 4:** Optional DC-Gaussian/COLMAP-based reflection removal for reflection-affected dashcam sequences; not used in the KITTI experiments.
- **Stage 5:** Vehicle-anchored RANSAC + WLS training-free scale/shift correction using known vehicle physical dimensions.
- **Stage 6:** Configurable LiDAR projection templates for nuScenes, IDD, and TIAND; these datasets were not used in the reported experiments.
- **Stage 7:** Comprehensive supervised fine-tuning ablation on DA3-M (13 loss functions, 3 architectures, 4 augmentation strategies, 5 optimizer/scheduler combinations), then final fine-tuning on the complete KITTI training set.
- **Stage 8:** Self-supervised photometric re-projection fine-tuning with a 6-DoF ResNet-18 PoseNet and an ablation of photometric, geometric, and temporal loss formulations.
- **Stage 9:** Combined fine-tuning that initializes from the selected self-supervised checkpoint and continues with supervised losses using projected LiDAR depth.

3.2 Stage 1: SAM3 Instance Segmentation

3.2.1 Model Loading and Offline Mask Generation

We load SAM3 via Facebook Research's `sam3` package:

```
from sam3.model_builder import build_sam3_image_model
```

```
from sam3.model.sam3_image_processor import Sam3Processor
model = build_sam3_image_model()
processor = Sam3Processor(model)
```

For each frame, `processor.set_text_prompt(state, prompt)` returns binary masks with IoU confidence scores. We generate masks *offline, once*, saving them as binary PNGs with a JSON manifest per frame. They are never regenerated during evaluation. This clears the SAM3's inference overhead across all downstream stages.

3.2.2 Prompt Engineering

Finding suitable prompts for SAM3 required testing different descriptions and observing how the model responded to them on dashcam images. General prompts were not always sufficient, particularly when the objective was to segment a specific vehicle part or an object under different viewing and lighting conditions. During this process, a few changes were made to the prompt descriptions to obtain more consistent segmentation results. The main principles followed during prompt design are discussed below:

1. **Part-as-subject for vehicle components.**

   When segmenting vehicle lights and wheels, the prompts are written to describe the specific component rather than the entire vehicle. For example, the headlight cluster or wheel is made the main subject of the prompt instead of using a general description such as "car". This helps SAM3 focus on the required part rather than segmenting the complete vehicle, which is important when these regions are later used for geometric measurements and depth anchoring.

2. **Combination of illumination and housing descriptions.**

   For vehicle lights, both illumination-related descriptions and physical housing or assembly descriptions are included in the prompts. Terms describing active light sources are useful in conditions such as nighttime or dusk, whereas descriptions of the headlight housing and assembly are more suitable for daytime images. Using both types of descriptions helps the segmentation process work across the different lighting conditions present in the KITTI dataset.

3. **Use of static geometric descriptions.**

   Motion-related terms such as "approaching", "driving away", and "receding" are avoided in favour of descriptions based on the vehicle's visible orientation, such as "front view of", "rear view of", and "in profile". Since SAM3 processes individual images without any temporal information, the prompts are designed around the visible appearance and geometry of objects rather than their movement across consecutive frames. This also makes the descriptions applicable to stationary vehicles.

4. **Use of camera-relative directions.**

   Directional terms such as "nearside" and "offside" are replaced with "left-side" and "right-side", respectively, based on the camera's viewpoint. These descriptions remain consistent regardless of the driving side followed in a particular country. This is useful when applying the same segmentation pipeline to footage from different regions, such as India and European countries, where the traffic direction differs.

5. **Noun-first descriptions for animals.**

   For animal segmentation, action-based terms such as "pecking" and "running" are avoided. Instead, the prompts describe the animal using its name along with its location or surrounding context. This encourages SAM3 to identify

the animal as an object rather than focusing mainly on a particular posture or action. It also makes the prompts more suitable for segmenting animals in different poses.

3.2.3 Semantic Categories Used for Segmentation

Based on the different objects identified in dashcam images, the segmentation pipeline groups the detected instances into eight main categories. Each category contains one or more sub-categories, which are defined through individual text prompts given to SAM3. This grouping makes it easier to identify and process different objects separately, particularly when the segmented regions are used for geometric measurements or further analysis. The eight super-categories, along with some of their corresponding sub-categories and their intended use in the pipeline, are listed below.

Table 3.1: Segmentation categories and their use cases

Super-category	**Example sub-categories**	**Pipeline use**
Vehicle	`personal_vehicle_headlights`	Vehicle detection and segmentation
Vehicle lights	`vehicle_both_headlights`	Width/length measurement
Vehicle wheels	`vehicle_both_front_wheels`	Width/length measurement
Cyclist	`bicycle_with_rider`	Cyclist and bicycle segmentation
Pedestrian	`pedestrian`	Pedestrian segmentation
Road	`road_surface`	Road-width estimation
Animal	`wildlife_large_small`	Animal segmentation
Stationary object	`building`, `traffic_sign`	SSL training signal

The vehicle category represents complete vehicle instances, whereas vehicle lights and vehicle wheels are treated separately because their segmented regions are used for geometric measurements. Similarly, bicycles with riders and bicycles without riders are grouped under the cyclist category. Pedestrians, animals, and road surfaces are also handled as separate categories. The stationary object category includes objects

such as trees, poles, road barriers, buildings, and traffic signs, which generally remain fixed with respect to the surrounding road environment.

3.2.4 NMS and Mask Quality Filtering

Since multiple prompts belonging to the same super-category can sometimes detect the same physical object, duplicate masks may be generated. For example, prompts related to "headlights" and "front of vehicle" may both produce masks for the same vehicle. To handle such cases, greedy Non-Maximum Suppression (NMS) is applied with an IoU threshold of 0.60. This threshold was selected after testing different values. With a threshold of 0.50, several valid masks corresponding to different sub-categories were suppressed, whereas values above 0.70 allowed more duplicate masks to remain. Based on a manual inspection of 50 frames, a threshold of 0.60 provided a reasonable balance between removing duplicates and retaining valid detections.

After NMS, the remaining masks are further checked using the following quality criteria:

- The SAM3 IoU confidence must be greater than 0.40 for general categories. For vehicle anchoring, a higher threshold of 0.50 is used because accurate mask boundaries are more important for the subsequent geometric measurements.
- The mask must cover an area of at least $500\,\mathrm{px}^2$. This helps remove very small detections that may not be useful for further processing.
- The bounding-box fill ratio, calculated as the ratio of the mask area to the area of its bounding box, must be less than 0.85:

$$\text{Fill Ratio} = \frac{\text{Mask Area}}{h_{\text{bbox}} \times w_{\text{bbox}}} < 0.85 \tag{3.1}$$

This condition helps filter out false-positive masks that cover nearly the entire image or extend across a large portion of the scene.

3.3 Stage 2: Multi-Model Inference and Evaluation

3.3.1 Models and Inference APIs

We evaluate five models on all valid LiDAR pixels within the Eigen crop intersected with each instance mask:

DepthPro:

HuggingFace `DepthProForDepthEstimation` (`apple/DepthPro-hf`); returns metric depth and predicted focal length.

UD2:

`lpiccinelli/unidepth-v2-vitl14` via UniDepth custom API; returns metric depth map and the full predicted camera model.

DA2:

HuggingFace `AutoModelForDepthEstimation` (`depth-anything/Depth-Anything-V2-Metric-Outdoor-Large-hf`); output in metres directly.

DA3-G:

Same API as DA3-M with ‘‘`depth-anything/DA3NESTED-GIANT-LARGE`’’; affine-invariant relative depth only; used exclusively for camera intrinsic estimation in Stage 5.

DA3-M:

`DepthAnything3.from_pretrained(‘‘depth-anything/DA3METRIC-LARGE’’)`; raw

output scaled by Equation 2.2; input resized to the nearest multiple of 14 px for ViT patch alignment.

3.3.2 Evaluation Metrics

The performance of the depth estimation models is evaluated using standard depth estimation metrics on valid LiDAR pixels within the Eigen crop and the corresponding instance masks, wherever applicable. Only ground-truth depth values within the range of 0.001 to 80.0 m are considered during evaluation. The metrics are first calculated separately for each evaluation frame and are then aggregated across the dataset. Two different approaches are used for reporting RMSE: frame-averaged RMSE and pixel-weighted RMSE (PW-RMSE). The remaining metrics are averaged across frames without applying pixel weighting.

Let F denote the total number of valid evaluation frames, n_i the number of valid LiDAR pixels in the i-th frame, and $D_{\mathrm{gt},ij}$ and $\hat{D}_{ij}$ represent the ground-truth and predicted depth values, respectively, at the j-th valid pixel of that frame.

The RMSE for an individual frame is calculated as

$$\mathrm{RMSE}_i = \sqrt{\frac{1}{n_i}\sum_{j=1}^{n_i}\left(D_{\mathrm{gt},ij} - \hat{D}_{ij}\right)^2}. \tag{3.2}$$

The overall frame-averaged RMSE is then obtained by taking the arithmetic mean of the RMSE values across all valid evaluation frames:

$$\mathrm{RMSE}_{\mathrm{frame}} = \frac{1}{F}\sum_{i=1}^{F}\mathrm{RMSE}_i. \tag{3.3}$$

In contrast, the pixel-weighted RMSE (PW-RMSE) considers the number of valid LiDAR pixels in each frame while combining the squared errors. It is calculated as

$$\text{PW-RMSE} = \sqrt{\frac{\sum_{i=1}^{F} n_i \, \text{RMSE}_i^2}{\sum_{i=1}^{F} n_i}}. \tag{3.4}$$

Thus, frame-averaged RMSE gives equal importance to every evaluated frame, whereas PW-RMSE gives more weight to frames containing a larger number of valid LiDAR measurements.

The other evaluation metrics are calculated for each frame and subsequently averaged across all valid evaluation frames. These include absolute relative error (AbsRel), logarithmic RMSE, threshold accuracy (δ), and depth Bias. Their definitions are given below:

$$\text{AbsRel} = \frac{1}{F} \sum_{i=1}^{F} \left[\frac{1}{n_i} \sum_{j=1}^{n_i} \frac{|D_{\text{gt},ij} - \hat{D}_{ij}|}{D_{\text{gt},ij}} \right] \tag{3.5}$$

$$\text{RMSE}_{\log} = \frac{1}{F} \sum_{i=1}^{F} \sqrt{\frac{1}{n_i} \sum_{j=1}^{n_i} \left(\log D_{\text{gt},ij} - \log \hat{D}_{ij} \right)^2} \tag{3.6}$$

$$\delta_k = \frac{1}{F} \sum_{i=1}^{F} \left[\frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{1} \left(\max \left(\frac{D_{\text{gt},ij}}{\hat{D}_{ij}}, \frac{\hat{D}_{ij}}{D_{\text{gt},ij}} \right) < 1.25^k \right) \right], \quad k \in \{1, 2, 3\} \tag{3.7}$$

$$\text{Bias} = \frac{1}{F} \sum_{i=1}^{F} \left[\frac{1}{n_i} \sum_{j=1}^{n_i} (D_{\text{gt},ij} - \hat{D}_{ij}) \right]. \tag{3.8}$$

Here, $\mathbf{1}(\cdot)$ is an indicator function that returns 1 when the specified condition is satisfied and 0 otherwise. The threshold accuracy measures the proportion of

predictions for which the ratio between the predicted and ground-truth depth is below the specified threshold.

Bias represents the average signed difference between the ground-truth and predicted depths. A positive Bias indicates that the model tends to underestimate depth, meaning that the predicted distances are smaller than the corresponding LiDAR measurements. A negative Bias indicates overestimation of depth.

It is important to distinguish between these aggregation methods when interpreting the results. Frame-averaged RMSE and the other metrics give equal importance to each evaluation frame, regardless of the number of valid LiDAR measurements it contains. PW-RMSE, on the other hand, accounts for the variation in valid LiDAR pixel counts across frames. This provides an additional measure of overall pixel-level depth estimation error without allowing frames with sparse LiDAR coverage to have the same influence as frames containing a larger number of valid measurements.

3.3.3 Scale-Alignment Strategies

To understand how much of the depth estimation error is caused by differences in scale and shift, four different scale-alignment strategies are considered during evaluation. These methods help compare the model's original predictions with the results obtained after applying different types of corrections.

Unscaled:

The raw model prediction is used directly without applying any correction:

$$\hat{D} = D_{\text{raw}} \tag{3.9}$$

This represents the zero-shot metric-depth performance of the model and serves as the primary reference for evaluation.

Median:

A scale factor is calculated using the ratio between the median ground-truth depth and the median predicted depth:

$$s = \frac{\text{median}(D_{\text{gt}})}{\text{median}(D_{\text{raw}})}, \qquad \hat{D} = s \cdot D_{\text{raw}} \tag{3.10}$$

This provides a scale-only correction based on the ground-truth depth values and is therefore considered an oracle-based comparison.

Mean:

Similar to median scaling, the scale factor is calculated using the mean values of the ground-truth and predicted depths:

$$s = \frac{\text{mean}(D_{\text{gt}})}{\text{mean}(D_{\text{raw}})}, \qquad \hat{D} = s \cdot D_{\text{raw}} \tag{3.11}$$

Affine:

In this approach, both scale and shift are estimated by minimizing the squared difference between the corrected predictions and the ground-truth depth values:

$$[s, t] = \arg\min_{s,t} \|sD_{\text{raw}} + t - D_{\text{gt}}\|^2 \tag{3.12}$$

The corrected depth is then obtained as:

$$\hat{D} = sD_{\text{raw}} + t \tag{3.13}$$

Since both parameters are fitted directly using ground-truth values, this method provides an oracle-based upper bound for linear post-hoc correction.

The difference between the Unscaled and Affine results gives an indication of how much the prediction error can be reduced through a global scale and shift correction. This also provides a reference for assessing the scope for further improvement in Stage 5.

3.4 Stage 3: DA3-M Error Pattern Analysis

In this stage, the error patterns of DA3METRIC-LARGE are examined in greater detail by analysing the signed difference between the predicted depth and the corresponding LiDAR ground truth at the pixel level. The difference is calculated for each valid LiDAR pixel as:

$$\text{diff}_i = D_{\text{gt}}^{(i)} - \hat{D}^{(i)} \tag{3.14}$$

A positive difference indicates that the model predicts a smaller depth value than the LiDAR measurement, meaning that the depth is underestimated. Similarly, a negative difference indicates overestimation. Based on the magnitude and sign of these differences, the pixels are divided into seven categories, as listed below.

The analysis is also carried out using three different depth-clipping settings, with maximum depth values ($D_{\max}$) of 40.0 m, 80.0 m, and no upper limit (∞). This comparison helps examine how restricting the evaluated depth range affects the error distribution and the overall evaluation metrics. For the clipped variants, LiDAR

TABLE 3.2: Error pattern analysis table with range and interpretations

Category	**Range**	**Interpretation**
Zero	$\lvert\text{diff}\rvert \leq 0.10\,\text{m}$	Negligible error
Positive	$0.10 < \text{diff} \leq 2.0\,\text{m}$	Mild underestimation
Too-positive	$2.0 < \text{diff} \leq 5.0\,\text{m}$	Significant underestimation
Extreme-positive	$\text{diff} > 5.0\,\text{m}$	Severe underestimation
Negative	$-2.0 \leq \text{diff} < -0.10\,\text{m}$	Mild overestimation
Too-negative	$-5.0 \leq \text{diff} < -2.0\,\text{m}$	Significant overestimation
Extreme-negative	$\text{diff} < -5.0\,\text{m}$	Severe overestimation

points beyond the corresponding maximum depth are excluded from evaluation, while all valid LiDAR points are considered in the no-clipping case.

Finally, the error distribution and depth evaluation metrics are examined across all eight SAM3 super-categories. This provides a category-wise view of the model's performance and helps identify whether certain object categories exhibit more frequent or larger depth estimation errors than others.

3.5 Stage 4: Internal-Reflection Removal (Optional)

The repository also includes a separate notebook for removing windshield reflections from dashcam image sequences using DC-Gaussian. The pipeline first uses COLMAP for feature extraction, image matching, sparse reconstruction, and image un-distortion. The resulting data is then used to train the DC-Gaussian model, which renders the input frames with reflections removed. This step is included as an optional part of the overall pipeline, but it was not used in the KITTI Eigen-Split experiments presented in this thesis. Since the notebook focuses on generating reflection-free images and does not include a quantitative evaluation of the removal process, no numerical results or performance claims related to this step are reported in the results chapter.

3.6 Stage 5: Vehicle-Anchored Scale/Shift Correction

3.6.1 Motivation

The error analysis in Stage 3 showed that DA3-M tends to underestimate depth, particularly in the 0–40 m range. Based on this observation, we investigate whether this bias can be reduced using the known physical dimensions of vehicles, without requiring LiDAR measurements during inference. Vehicles are considered suitable reference objects for this purpose because their approximate dimensions are known, they can be detected using SAM3, and they are commonly found within the near-field region where the underestimation is more noticeable. The main objective of this stage is to estimate a correction using these vehicle measurements and apply it to the predicted depth maps.

3.6.2 Camera Intrinsic Estimation via DA3-G

Camera intrinsics are estimated using DA3-G (DA3NESTED-GIANT-LARGE). Instead of predicting the camera parameters independently for every frame, the images from each drive sequence are processed in chunks of 20 frames at a resolution of 504 pixels. The predicted intrinsic matrices $\mathbf{K}$ from all the chunks are combined, and their mean is used to obtain the sequence-level camera parameters. Since the model processes resized images, the focal lengths are rescaled according to the original image dimensions:

$$\tilde{f}_x = \bar{K}_{00} \times \frac{\max(W, H)}{504} \tag{3.15}$$

The variation in the predicted focal lengths is also considered while assigning a quality weight to the estimated intrinsics. This weight is kept within the range $[0.20, 1.00]$, with lower values assigned when the predictions show greater variation across the sequence. If all intrinsic prediction chunks fail, a fallback based on a 65° field of view is used.

For the KITTI experiments, the intrinsic estimation follows the selected focal-length policy specified in the implementation. In the configuration used here, the `as_predicted` policy is selected, meaning that the predicted focal lengths are used without modification. The implementation also provides other options for adjusting the relationship between f_x and f_y when required.

3.6.3 Vehicle Prior Specification

The vehicle dimensions used for anchor estimation are defined through two sub-categories: front-facing vehicles detected through their headlights and rear-facing vehicles detected through their taillights. The corresponding physical dimensions and reliability values are given below.

TABLE 3.3: Vehicle prior sub-category specification

Sub-category	H_r (m)	σ_H	W_r (m)	σ_W	Reliability
`personal_vehicle_headlights`	1.50	0.12	1.82	0.12	0.90
`personal_vehicle_taillights`	1.50	0.12	1.82	0.12	0.92

These values represent approximate dimensions of passenger cars, covering a typical range from sedans to compact SUVs. Standard deviations of 0.12 m are assigned to both height and width to account for variations in vehicle dimensions. The reliability values are slightly different for the two sub-categories and are also considered when calculating the weight of each anchor.

3.6.4 Orientation-Aware Anchor Depth Estimation

For every detected vehicle instance, a tight bounding box is obtained directly from the mask pixels. This is done using the minimum and maximum pixel coordinates of the mask, rather than relying on the bounding box provided in the SAM3 manifest. The resulting pixel height and width are denoted by $h_p = y_2 - y_1$ and $w_p = x_2 - x_1$, respectively.

The aspect ratio $\alpha = w_p/(h_p + \varepsilon)$ is then used to determine the approximate viewing orientation of the vehicle. Depending on this ratio, the height and width measurements are assigned different weights or excluded from the anchor calculation.

Table 3.4: Orientation-Aware anchor usability

α range	Orientation	Height usable?	Width usable?
$\alpha < 1.8$	Frontal/rear-on	Yes	Yes (full weight)
$1.8 \leq \alpha < 2.5$	Oblique	Yes	Yes ($\times 0.4$ weight)
$\alpha \geq 2.5$	Side-on	Yes	No

This distinction is important because the apparent width of a side-facing vehicle corresponds more closely to its length (approximately 4.5 m) rather than its actual width (1.82 m). Using the vehicle width prior in such cases would result in a considerably incorrect depth estimate. Therefore, the width-based estimate is completely excluded for side-on vehicles, while the height-based estimate is retained since the vertical dimension remains usable across different viewing orientations.

The depth estimates from the vehicle height and width are calculated using the pinhole camera projection model:

$$d_h = \frac{f_y \times H_r}{h_p}, \quad h_p \geq 8\,\text{px},\ 0.001 < d_h < 80\,\text{m} \tag{3.16}$$

$$d_w = \frac{f_x \times W_r}{w_p}, \quad w_p \geq 8\,\text{px},\ 0.001 < d_w < 80\,\text{m} \tag{3.17}$$

Here, H_r and W_r represent the assumed real-world vehicle height and width, respectively, while f_x and f_y are the camera focal lengths. Measurements with insufficient pixel dimensions or depth estimates outside the specified range are not considered valid.

For each valid vehicle instance, the corresponding raw DA3-M depth is obtained by taking the median of the predicted depth values within the vehicle mask. The median is preferred over the mean because it is less affected by incorrect depth values near the mask boundaries, where background pixels may also be present.

When both height-based and width-based depth estimates are available, their consistency is checked before combining them. If the ratio between the two estimates exceeds 1.25, only the estimate with the higher weight is retained. Otherwise, the two estimates are combined using a weighted average:

$$d_{\text{anchor}} = \frac{\sum_j w_j d_j}{\sum_j w_j} \tag{3.18}$$

The weights are derived from the prior reliability and the standard deviation associated with each physical dimension. The width-based weight is additionally adjusted according to the estimated vehicle orientation. The final anchor weight also considers the camera intrinsic quality and SAM3 mask confidence:

$$w_h = \frac{r}{\sigma_H^2}, \qquad w_w = \frac{r \cdot c_w}{\sigma_W^2} \tag{3.19}$$

$$w_{\text{anchor}} = r \cdot w_{\mathbf{K}} \cdot s_{\text{mask}} \cdot \sum_j w_j \tag{3.20}$$

where r denotes the reliability of the vehicle sub-category, $w_{\mathbf{K}}$ is the intrinsic quality weight, s_{mask} is the SAM3 mask confidence, and c_w is the orientation-dependent width coefficient, which takes values of 0, 0.4, or 1.0.

3.6.5 RANSAC + WLS Global Scale/Shift Fit

After extracting the vehicle anchors, the raw DA3-M depth values and their corresponding metric anchor depths are collected. These pairs are used to estimate a global correction for the dataset. The correction follows a linear relationship between the raw predicted depth and the anchor depth:

$$s \cdot D_{\text{raw},i} + t \approx d_{\text{anchor},i} \tag{3.21}$$

Here, s represents the scale factor and t represents the depth shift. The implementation supports two configurations: scale-and-shift correction, where both parameters are estimated, and scale-only correction, where the shift is fixed at zero. The latter is controlled through the `ONLY_SCALE` parameter and is enabled in the configuration used for the present experiments.

RANSAC is used first to identify a suitable set of inlier anchors. For the scale-and-shift configuration, each iteration randomly selects two anchor pairs to estimate the initial values of s and t. For the scale-only configuration, a single anchor is selected to estimate the scale factor. A total of 1,000 iterations are performed, and solutions with scale values outside the range $[0.2, 5.0]$ are rejected. An anchor is considered an inlier when the difference between its estimated depth and its reference anchor depth is less than 2.5 m. The hypothesis with the highest number of inlier is selected for the next step.

The selected inlier are then used for Weighted Least Squares (WLS) refinement. The weights assigned during anchor extraction are retained so that more reliable vehicle measurements have a greater influence on the final correction. For the scale-and-shift case, the parameters are obtained by solving:

$$[s, t]^* = \arg\min_{s,t} \sum_{i \in \mathcal{I}} w_i \left(s D_{\text{raw},i} + t - d_{\text{anchor},i} \right)^2 \tag{3.22}$$

where $\mathcal{I}$ represents the set of RANSAC inlier and w_i denotes the corresponding anchor weight. When scale-only correction is selected, the shift is fixed at zero and the scale is estimated as:

$$s^* = \frac{\sum_{i \in \mathcal{I}} w_i D_{\text{raw},i} d_{\text{anchor},i}}{\sum_{i \in \mathcal{I}} w_i D_{\text{raw},i}^2} \tag{3.23}$$

A minimum of four valid anchors is required before estimating the correction. If this requirement is not met, the implementation raises an error instead of applying an unreliable correction. The estimated parameters, together with the number of anchors and inlier, are also saved for subsequent use.

RANSAC combined with WLS was selected because it provides a relatively simple way to estimate the correction without introducing additional training or learnable parameters. RANSAC helps remove inconsistent anchor measurements, while WLS gives greater importance to the more reliable anchors during refinement. However, the accuracy of the final correction still depends on the quality of the vehicle masks, physical dimension assumptions, and estimated camera intrinsics. The effect of these factors is examined further in Chapter 5.

3.6.6 Two-Pass Evaluation

The vehicle-anchored correction is implemented in two passes so that the correction parameters can be estimated before being applied during evaluation.

Pass 1 (Anchor Collection and Calibration): The first pass processes the KITTI frames and loads the corresponding SAM3 vehicle masks. DA3-M is used to

obtain the raw depth maps, while DA3-G estimates the camera intrinsics for each drive sequence. The tight bounding boxes are then calculated from the vehicle mask pixels, and the corresponding anchor depths are obtained using Equations 3.16–3.17. After collecting the valid anchor pairs, RANSAC and WLS are used to estimate the global scale and shift parameters.

Pass 2 (Depth Correction and Evaluation): In the second pass, the estimated correction is applied to the raw DA3-M depth maps of all frames. The corrected predictions are then evaluated against the LiDAR ground truth for each valid SAM3 mask. Along with the vehicle-anchored results, four other scale variants (unscaled, median, mean, and affine) are also evaluated for comparison. The evaluation metrics and associated depth statistics are finally saved in a CSV file for further analysis.

3.7 Stage 6: LiDAR-to-Depth-Map Generation Templates

For datasets that provide LiDAR point clouds along with the required camera calibration, the notebook follows a common procedure to generate depth maps aligned with the corresponding front-camera images. The procedure is applied to nuScenes, TIAND, and IDD, with the point cloud loading and calibration steps adjusted according to the format and structure of each dataset.

1. **Load the LiDAR point cloud:**

   The point clouds are read from `.bin` files for nuScenes, `.pcd` files for TIAND, and `.npy` files for IDD. In each case, only the x, y, and z coordinates are retained for further processing.

2. **Apply the LiDAR-to-camera transformation:**

   The points are transformed from the LiDAR coordinate system to the camera coordinate system using the extrinsic transformation matrix $\mathbf{T}_{\text{cam}\leftarrow\text{lidar}}$. The corresponding camera intrinsics are also loaded from the dataset calibration files or, when necessary, from the configured manual calibration values. For nuScenes, the transformation can additionally account for the ego-motion between the LiDAR sweep and camera exposure.

3. **Filter points based on depth:**

   After transformation, the z-coordinate of each point in the camera frame is used to determine its depth. Points outside the configured range of $[0.1, 80.0]\,\text{m}$ are removed.

4. **Project points onto the image plane:**

   The remaining 3D points are projected into pixel coordinates using the camera intrinsic matrix $\mathbf{K}$. Points falling outside the image boundaries are discarded.

5. **Generate the depth map:**

   The depth assigned to each projected pixel is the z-coordinate of the corresponding point in the camera frame. This represents the camera-axis (optical-axis Z) depth, rather than the Euclidean distance from the camera centre, which would be calculated as $\sqrt{x^2 + y^2 + z^2}$.

6. **Handle multiple points at the same pixel:**

   Since more than one LiDAR point may project onto a single image pixel, the point with the smallest z-value is retained. This ensures that the closest visible surface is represented in the resulting depth map. Pixels without any projected LiDAR return are assigned a value of zero.

7. **Save the depth map:**

   The projected depth array is exported as a single-channel `uint8` PNG by scaling valid depths relative to the 80 m maximum and mapping them to $[1, 255]$, while zero is reserved for invalid pixels. This 8-bit representation is useful for compact visual inspection, but it quantises the metric depth values and should not be treated as a precision-preserving training or evaluation target. Quantitative downstream processing should use the floating-point projected depths or a precision-preserving format such as 16-bit or floating-point storage.

The same projection procedure is used for nuScenes, TIAND, and IDD, while the point cloud loaders and calibration handling are adapted to the respective dataset formats. The notebook also provides an option to save colourised versions of the generated depth maps, which are used for visual inspection and quality checking.

KITTI pre-projected depth PNGs are used directly (no re-projection) since KITTI distributes high-quality calibrated projections. For nuScenes, TIAND, and IDD, we run the full projection pipeline above.

3.8 Stage 7: Supervised Fine-Tuning

3.8.1 Why Decoder-Only

The architecture choice for the later supervised experiments was made mainly with the available GPU memory and the size of the training set in mind. DA3-M uses a ViT-Large encoder, and updating the complete network on the available 32 GB V100 would substantially increase the memory required for training. Freezing the encoder and training the decoder and depth head reduces the number of trainable

parameters from approximately 334.2 M to 29.8 M. With this configuration, the final training setup could use a batch size of 8 and accumulate gradients over four steps, giving an effective batch size of 32.

The architecture ablation, however, does not show that decoder-only training gives the best result on every evaluation metric. On the 46-frame held-out test set, full fine-tuning achieved a pixel-weighted RMSE of 2.5056 m and an AbsRel of 0.0713, compared with 2.5714 m and 0.0718 for decoder-only training. Decoder-only gave a slightly higher δ_1 value (0.9690 compared with 0.9662 for full fine-tuning). The saved ablation output also shows a steadier validation-RMSE trajectory for decoder-only, while the full fine-tuning run was less stable over the five-epoch search.

These results therefore show a trade-off rather than a consistent performance advantage for decoder-only training. Full fine-tuning has lower RMSE and AbsRel in this comparison, whereas decoder-only has the higher δ_1 and requires substantially fewer trainable parameters. Decoder-only was retained for the subsequent full-set experiments because it was more practical under the available hardware constraints and avoided updating the pretrained encoder. The ablation does not, by itself, establish that full fine-tuning caused catastrophic forgetting, so that effect is treated as a possible risk rather than a measured result.

The encoder is identified by parameter-name matching: parameters whose names contain `encoder`, `pretrained`, `backbone`, or `net.net` are frozen, while the remaining decoder and head parameters are updated.

3.8.2 Loss Functions

The supervised ablation notebook implements 13 loss configurations as PyTorch modules. The equations below describe the implementations used in the individual-loss ablation. Where the later full-set training notebook changes an implementation, that difference is stated separately so that the ablation and final-training results are not treated as if they used identical code.

SILog [10]

$$\mathcal{L}_{\text{SILog}} = \sqrt{\text{mean}_M(d_i^2) - 0.85\,[\text{mean}_M(d_i)]^2 + \varepsilon}, \quad d_i = \log(\max(\hat{D}_i, \varepsilon)) - \log(\max(D_{\text{gt}}^{(i)}, \varepsilon)), \tag{3.24}$$

where mean_M denotes the mean over pixels with valid LiDAR depth.

BerHu [24]

$$\mathcal{L}_{\text{BerHu}}(e_i) = \begin{cases} |e_i| & |e_i| \le c, \\ \dfrac{e_i^2 + c^2}{2c} & |e_i| > c, \end{cases} \qquad e_i = \hat{D}_i - D_{\text{gt}}^{(i)}, \quad c = 0.2 \max_{i:M_i=1} |e_i|, \tag{3.25}$$

where the threshold is calculated from the maximum valid absolute residual in the batch and is detached from the computational graph. The resulting loss is averaged over valid LiDAR pixels.

Gradient Loss

Let M_i denote the valid-depth mask. For each horizontal and vertical neighbouring pair, define

$$M_i^x = M_i M_{i+1_x}, \qquad M_i^y = M_i M_{i+1_y}, \tag{3.26}$$

where $i+1_x$ and $i+1_y$ are the adjacent pixels in the horizontal and vertical directions. The HPT implementation first multiplies the prediction and ground truth by the validity mask, then calculates their finite differences. Its loss is

$$\begin{aligned}\mathcal{L}_{\text{grad}}^{\text{HPT}} = \ & \text{mean}\Big(\Big|\Delta_x(\hat{D}\odot M) - \Delta_x(D_{\text{gt}}\odot M)\Big|\, M^x\Big) \\ & + \text{mean}\Big(\Big|\Delta_y(\hat{D}\odot M) - \Delta_y(D_{\text{gt}}\odot M)\Big|\, M^y\Big)\,,\end{aligned} \tag{3.27}$$

where the tensor means are taken over the full horizontal and vertical difference arrays; masked-out pairs contribute zero but remain in the averaging denominator. The final full-set training notebook uses a revised denominator and averages only over valid neighbouring pairs:

$$\mathcal{L}_{\text{grad}}^{\text{final}} = \frac{\sum_i M_i^x |g_i^x|}{\sum_i M_i^x + \varepsilon} + \frac{\sum_i M_i^y |g_i^y|}{\sum_i M_i^y + \varepsilon}, \tag{3.28}$$

with the residuals defined as $g_i^x = (\Delta_x \hat{D})_i - (\Delta_x D_{\text{gt}})_i$ and $g_i^y = (\Delta_y \hat{D})_i - (\Delta_y D_{\text{gt}})_i$. The masks M_i^x and M_i^y ensure that only pairs with valid depth at both endpoints contribute.

MAE-Canonical [5]

The ablation implementation applies an L1 loss to inverse depth. The prediction and reference depth are first clamped to the positive numerical range $[\varepsilon, 80]$ m:

$$p_i = \frac{1}{\text{clamp}(\hat{D}_i, \varepsilon, 80)}, \qquad q_i = \frac{1}{\text{clamp}(D_{\text{gt}}^{(i)}, \varepsilon, 80)}. \tag{3.29}$$

The masked inverse-depth term and its four-scale gradient component are combined as

$$\mathcal{L}_{\text{MAE}}^{\text{HPT}} = \frac{\sum_i M_i |p_i - q_i|}{\sum_i M_i + \varepsilon} + \frac{0.5}{4} \sum_{k=0}^{3} \mathcal{G}_{\text{grad}}^{\text{HPT}}(p^{(k)}, q^{(k)}, M^{(k)}), \tag{3.30}$$

where the inverse-depth maps and validity mask are downsampled by average pooling between scales, and the pooled mask is thresholded at 0.5 when the gradient term is evaluated. The HPT implementation does not use per-image min–max normalisation of inverse depth.

The final full-set notebook makes two changes to this loss. First, it clamps both predicted and reference depths to $[0.5, 80]$ m before inversion. Second, its lower-resolution inverse-depth maps are pooled using the validity mask, so invalid pixels do not enter the pooled depth values. If $P^{(k)}$ and $Q^{(k)}$ denote the prediction and reference inverse-depth maps at scale k, and $M^{(k)}$ the corresponding pooled mask, the downsampling is

$$M^{(k+1)} = \mathrm{Pool}(M^{(k)}), \qquad P^{(k+1)} = \frac{\mathrm{Pool}(P^{(k)} M^{(k)})}{M^{(k+1)} + \varepsilon}, \qquad Q^{(k+1)} = \frac{\mathrm{Pool}(Q^{(k)} M^{(k)})}{M^{(k+1)} + \varepsilon}. \tag{3.31}$$

The final full-set MAE-Canonical term otherwise retains the same inverse-depth L1 term and the 0.5-weighted average of the four gradient scales. Thus, the 0.5 m floor applies to the final full-set run, not to all earlier HPT trials.

Masked Depth [25]

In the HPT ablation implementation, the confidence weight is formed from inverse ground-truth depth and normalised by its mean over valid pixels:

$$c_i = \frac{1}{\max(D_{\mathrm{gt}}^{(i)}, \varepsilon)}, \qquad \bar{c}_i = \frac{c_i}{\mathrm{mean}_M(c_i) + \varepsilon}. \tag{3.32}$$

The implemented loss includes both a confidence-weighted metric residual and a logarithmic confidence regulariser:

$$\mathcal{L}_D^{\mathrm{HPT}} = \frac{1}{\sum_i M_i + \varepsilon} \sum_i M_i \left[\bar{c}_i |\hat{D}_i - D_{\mathrm{gt}}^{(i)}| - 0.5 \log(\max(\bar{c}_i, \varepsilon)) \right]. \tag{3.33}$$

This is not a weighted SILog loss.

The final full-set notebook uses a simpler weighted metric L1 form. The reference depth is clamped to $[0.5, 80]$ m, and the inverse-depth weights are normalised to sum to one over the valid pixels:

$$D_{i,c} = \mathrm{clamp}(D_{\mathrm{gt}}^{(i)}, 0.5, 80), \qquad w_i = \frac{M_i / D_{i,c}}{\sum_j M_j / D_{j,c} + \varepsilon}, \qquad \mathcal{L}_D^{\mathrm{final}} = \sum_i w_i |\hat{D}_i - D_{i,c}|. \tag{3.34}$$

The final version does not include the logarithmic confidence regulariser used by the HPT implementation. Recipe A4 in the 100-epoch full-set notebook uses this final version.

EG-SSI [31]

The implemented Edge-Guided SSI loss selects image regions with strong RGB gradients and compares locally standardised depth patches. RGB is converted to grayscale and Sobel filters are used to obtain the gradient magnitude. For each image, candidate patch centres are selected from the top 5% of gradient magnitudes. Up to 64 patches of size 16×16 are sampled; a patch is retained when at least 25% of its pixels have valid LiDAR depth. The validity mask is used for this patch-acceptance test; after a patch is accepted, the patch loss is calculated over the complete patch.

For a patch P, the implementation uses the median and the mean absolute deviation from the median:

$$\mathcal{N}(P)_i = \frac{P_i - \text{median}(P)}{\text{mean}_j \, |P_j - \text{median}(P)| + \varepsilon}. \tag{3.35}$$

The loss is the mean L1 difference between the independently standardised predicted and ground-truth patches:

$$\mathcal{L}_{\text{EG-SSI}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{mean}_{i \in P} \left| \mathcal{N}(\hat{D}_P)_i - \mathcal{N}(D_{\text{gt},P})_i \right|, \tag{3.36}$$

where $\mathcal{P}$ is the set of accepted patches. If RGB is not supplied, the implementation falls back to image-region selection using ground-truth depth gradients. The code does not perform a global least-squares scale-and-shift fit over the complete depth map.

Temporal Gradient Matching (TGM) [8]

TGM compares the change in predicted depth at the same pixel location between two adjacent frames with the corresponding change in LiDAR depth. The joint valid mask is further restricted to pixels whose ground-truth depth change is below 0.05 m:

$$M_{s,i} = M_{t,i} M_{t+1,i} \mathbf{1} \left[|D^{\text{gt}}_{t+1,i} - D^{\text{gt}}_{t,i}| < 0.05\,\text{m} \right]. \tag{3.37}$$

Define

$$r^{\text{pred}}_i = |\hat{D}_{t+1,i} - \hat{D}_{t,i}|, \qquad r^{\text{gt}}_i = |D^{\text{gt}}_{t+1,i} - D^{\text{gt}}_{t,i}|. \tag{3.38}$$

The temporal matching component and the standalone TGM objective are

$$\mathcal{L}^{\text{match}}_{\text{TGM}} = \frac{\sum_i M_{s,i} |r^{\text{pred}}_i - r^{\text{gt}}_i|}{\sum_i M_{s,i} + \varepsilon}, \qquad \mathcal{L}^{\text{standalone}}_{\text{TGM}} = \mathcal{L}_{\text{SILog}}(\hat{D}_t, D^{\text{gt}}_t) + 10 \mathcal{L}^{\text{match}}_{\text{TGM}}. \tag{3.39}$$

If sequential data or valid static pixels are unavailable, the standalone module falls back to the current-frame SILog term. In the DA3-Video combined loss, the temporal matching component is used with weight 10 directly, without adding the standalone TGM SILog term a second time.

The remaining variants (Affine-Invariant SILog, VNL, Log-L1, SSIM+L1, Robust Cauchy, Combined-Base, and DA3-Video-Combined) are defined in full in Appendix A.

3.8.3 Ablation Design

The ablation study was initially carried out on a smaller subset of the KITTI dataset to examine the effect of different training configurations without requiring the complete training dataset. This subset was divided into 488 training samples and 85 validation samples, while the separate KITTI Eigen test split contained 46 frames. The train-validation split was generated using a fixed random seed to maintain consistency across the experiments.

Four aspects of the training setup were examined separately: the loss function, model architecture, data augmentation, and optimiser-scheduler combination. During each experiment, only one of these aspects was changed at a time, while the remaining settings were kept at their default values. The different configurations used for this study are listed in Table 3.5.

For each configuration, a small hyperparameter search was performed using three learning-rate values: $\{1 \times 10^{-4}, 5 \times 10^{-5}, 2 \times 10^{-5}\}$. Each candidate was trained for a maximum of five epochs, with early stopping enabled after three consecutive epochs without improvement in validation RMSE. The learning rate that produced the lowest validation RMSE was selected, and its corresponding best checkpoint

Table 3.5: Ablation Design for Supervised Fine-tuning

Dimension	**Variants (default in bold)**
Loss	13 variants, with `combined_base` used as the baseline
Architecture	`decoder_only`, `last2_blocks`, `full`
Augmentation	`none`, `flip`, `base` **(flip + jitter)**, `full` (flip + jitter + crop)
Optimiser/scheduler	`AdamW + StepLR`, `Adam + cosine_warmup`, `SGD + cosine_warmup`, `AdamW + OneCycleLR`, `AdamW + cosine_warmup`

was then evaluated on the 46-frame held-out test set. Thus, the test evaluation was performed only after the learning-rate selection had been completed using the validation data.

The main training settings were kept consistent across the ablation experiments. The input images were resized to 378×1246 px, with both dimensions chosen to be compatible with the ViT patch size of 14. A batch size of 4 was used along with gradient accumulation over 2 steps, resulting in an effective batch size of 8. Training was performed using mixed-precision (FP16) with `GradScaler`, and gradient clipping was applied with a maximum norm of 1.0. For the default AdamW and cosine-warmup configuration, the learning rate was increased linearly during the first epoch and then reduced using cosine annealing. The minimum learning rate was set to 1% of the configured base learning rate.

3.8.4 Combined Recipe Evaluation

After examining the individual loss functions, different combinations of loss components were defined for further experiments. The intention was to combine the advantages of MAE-Canonical and SILog with additional terms such as gradient,

BerHu, and MaskedDepth loss. The final fine-tuning pipeline includes eight such combinations, as shown in Table 3.6.

Table 3.6: Combined Loss Recipes for Supervised Fine-tuning

Tag	**Composition**
A1	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + 0.1\mathcal{L}_{\text{grad}}$
A2	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}}$
A3	A1 $+0.5\mathcal{L}_{\text{BerHu}}$
A4	A1 $+0.5\mathcal{L}_{D}$
A5	A1 $+1.0\mathcal{L}_{D}$
B1	$0.5\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + \mathcal{L}_{\text{BerHu}} + 0.5\mathcal{L}_{L1} + 0.1\mathcal{L}_{\text{grad}}$
B2	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + \mathcal{L}_{\text{BerHu}} + 0.5\mathcal{L}_{L1} + 0.1\mathcal{L}_{\text{grad}}$
C	$\mathcal{L}_{\text{MAE}}$ alone (SGD with Nesterov momentum, LR= 10^{-4})

Here, $\mathcal{L}_D$ represents the MaskedDepth loss. Recipes A1–A5 are based on the combination of MAE-Canonical, SILog, and gradient loss, with additional components introduced in A3, A4, and A5. Recipes B1 and B2 retain the BerHu and L1 terms from the original combined loss, but use different weights for MAE-Canonical. Recipe C uses MAE-Canonical alone and is trained with SGD and Nesterov momentum instead of the default AdamW optimiser.

These combinations provide different ways of combining the individual loss components during supervised fine-tuning. The corresponding recipe can be selected directly in the training pipeline, allowing the configurations to be trained and evaluated under the same general framework.

3.8.5 Final Fine-Tuning

For the final fine-tuning stage, the training configuration was adjusted based on the observations made during the ablation experiments. In the supplied final training notebook, Recipe A4 is selected. It combines MAE-Canonical, SILog, and gradient

loss with an additional MaskedDepth term weighted by 0.5. The final full-set notebook uses the revised MAE-Canonical, Gradient and MaskedDepth implementations described above: MAE-Canonical applies a 0.5 m depth floor and mask-aware pooling, Gradient Loss is normalised by the number of valid neighbouring pairs, and MaskedDepth uses inverse-depth-normalised weighted L1 without the logarithmic confidence regulariser used in the earlier HPT ablation. The model is trained using the complete set of available KITTI training samples, with 10% of the training data reserved for validation. This split is generated using a fixed random seed of 42, while the separate KITTI Eigen test set is retained for evaluation.

The main changes made to the training configuration are as follows:

- **Batch size:** A batch size of 8 is used, with gradient accumulation over 4 steps. This gives an effective batch size of 32.
- **Learning rate:** The learning rate is set to 1×10^{-6}. This is lower than the earlier value of 2×10^{-6}, as the notebook configuration was further adjusted to avoid the instability observed with larger learning rates.
- **Learning-rate scheduler:** A cosine learning-rate schedule with one epoch of linear warmup is used. The minimum learning rate is set to 1% of the base learning rate, giving $\eta_{\min} = 1 \times 10^{-8}$ for the selected configuration. The warmup duration in steps is calculated from the number of training batches and the gradient accumulation setting.
- **Optimiser:** AdamW is used with a weight decay of 10^{-4}. Gradient clipping is retained, with the maximum gradient norm set to 1.0.
- **Mixed precision and training duration:** Training uses AMP in FP16 mode with gradient scaling. The maximum training duration is 100 epochs, with

early stopping patience set to 40 epochs based on the pixel-weighted validation RMSE.

- **Input resolution:** The input images are maintained at 378×1246 px, consistent with the ablation experiments.

The best model checkpoint is saved whenever the validation RMSE improves. In addition, the latest checkpoint is updated after every epoch so that training can be resumed in case of an interruption. Periodic checkpoints are also retained for subsequent evaluation and analysis.

3.9 Stage 8: Self-Supervised Fine-Tuning

3.9.1 Motivation and Formulation

A large amount of dashcam footage is available without corresponding LiDAR measurements. This makes it difficult to use conventional supervised fine-tuning for every collected video. Self-supervised fine-tuning provides an alternative by using the geometric relationship between consecutive frames to obtain a training signal. Instead of directly comparing the predicted depth with ground-truth depth, the model is trained by checking how well the scene can be reconstructed when one frame is projected into the viewpoint of another.

In the implemented pipeline, each training sample contains three consecutive frames: the current frame I_t and two neighboring frames, I_{t-1} and I_{t+1}. The depth model estimates the depth of the current frame, while the relative camera motion is obtained either from a PoseNet or from the KITTI OXTS measurements, depending on the

experiment. The predicted depth and camera motion are then used to establish the pixel correspondences between frames and calculate the reconstruction error.

An important consideration in this process is that the depth of a scene point does not necessarily remain the same in two consecutive frames, even when the object itself is stationary. This is because the camera moves between exposures, changing the point's position relative to the camera. Therefore, directly penalizing the difference between two depth maps at the same pixel locations does not properly describe the underlying geometry.

The notebook retains a simple depth-difference formulation, `naive_temporal_delta`, as a negative-control experiment. However, this loss only encourages the depth difference to remain spatially consistent; it does not explicitly account for camera motion. A more appropriate approach is to use geometric warping, where the predicted depth and relative camera transformation are used to compare the corresponding 3D points across frames. This follows the general principle of photometric reconstruction introduced in Monodepth2 [17].

3.9.2 Pose Estimation

The pipeline supports two ways of obtaining the relative camera pose. The first uses a PoseNet to predict camera motion directly from consecutive RGB frames, while the second uses the relative poses derived from the KITTI OXTS measurements. The latter is used as an oracle-pose reference in the ablation experiments, allowing the effect of pose-estimation errors to be examined separately.

3.9.2.1 PoseNet

For experiments that use predicted camera motion, a 6-DoF PoseNet is implemented with a ResNet-18 backbone. It receives a pair of RGB images, concatenated along the channel dimension to form a six-channel input, (I_t, I_s), where I_s represents one of the neighboring source frames. The network predicts a six-dimensional pose vector containing an axis-angle rotation $\boldsymbol{\omega} \in \mathbb{R}^3$ and a translation vector $\mathbf{t} \in \mathbb{R}^3$.

The predicted pose is converted into a homogeneous transformation matrix:

$$\mathbf{T}_{t\to s} = \begin{bmatrix} \mathbf{R}(\boldsymbol{\omega}) & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathrm{SE}(3), \tag{3.40}$$

where $\mathbf{R}(\boldsymbol{\omega})$ is obtained from the axis-angle representation using Rodrigues' rotation formula. The translation and rotation together describe the relative motion from the current camera frame to the selected source frame.

The PoseNet is not jointly trained with the depth model from the very beginning. In the implemented version, it first undergoes a one-epoch warm-up in which the depth model is kept frozen and only the PoseNet is updated using the photometric reconstruction loss. After this initial stage, the predicted-pose experiments train the depth network and PoseNet together. A separate learning rate of 10^{-4} is assigned to the PoseNet, while the learning rate for the depth model is selected independently.

This initialization is important because starting with an exactly identity transformation can make the photometric warp independent of the predicted depth, resulting in little or no useful depth gradient. The PoseNet output is therefore scaled by 0.1, and its final convolution is not initialized with zero weights.

3.9.2.2 Relative Pose from KITTI OXTS

For the oracle-pose experiments, relative camera motion is obtained from KITTI's OXTS GPS/IMU measurements and calibration files. The transformation from the IMU coordinate system to the rectified camera-02 coordinate system is constructed as:

$$\mathbf{T}_{\text{cam02}\leftarrow\text{imu}} = \mathbf{T}_{\text{02-offset}} \cdot \mathbf{R}_{\text{rect00}} \cdot \mathbf{T}_{\text{velo}\rightarrow\text{cam0}} \cdot \mathbf{T}_{\text{imu}\rightarrow\text{velo}}. \tag{3.41}$$

Let $\mathbf{W}_i$ and $\mathbf{W}_j$ represent the world transformations obtained from OXTS for two frames. The relative transformation between the corresponding camera coordinate systems is calculated by:

$$\mathbf{T}_{i\rightarrow j}^{\text{cam02}} = \mathbf{T}_{\text{imu}\rightarrow\text{cam02}} \cdot \mathbf{W}_j^{-1} \cdot \mathbf{W}_i \cdot \mathbf{T}_{\text{cam02}\rightarrow\text{imu}}. \tag{3.42}$$

This conversion is important because the relative transformation must be expressed in the camera coordinate system used by the depth and warping operations. The resulting OXTS pose is used in place of the PoseNet prediction for the oracle-pose experiments. It provides an externally measured camera-motion reference, but does not provide ground-truth depth to the self-supervised loss.

3.9.3 Differentiable Inverse Warping

The main idea behind inverse warping is to use the predicted depth of the current frame to determine where each pixel should appear in a neighboring frame. The process starts by back-projecting a pixel $\mathbf{p} = [u, v, 1]^\top$ from the current image into

3D camera coordinates using the predicted depth and the inverse camera intrinsic matrix:

$$\mathbf{P}_t = D_t(\mathbf{p}) \cdot \mathbf{K}^{-1}\mathbf{p}. \tag{3.43}$$

The resulting 3D point is transformed into the coordinate system of the source frame using the relative camera pose:

$$\mathbf{P}_s = \mathbf{R}_{t\to s}\mathbf{P}_t + \mathbf{t}_{t\to s}. \tag{3.44}$$

It is then projected back onto the source image plane:

$$\mathbf{p}_s = \mathbf{K}\mathbf{P}_s / P_{s,z}. \tag{3.45}$$

The RGB value at the projected location is obtained using bilinear grid sampling. In this way, the source image is sampled at the locations corresponding to the pixels in the current frame, producing a reconstructed image $\hat{I}_{t\leftarrow s}$. The operation remains differentiable, allowing the reconstruction error to contribute gradients to the predicted depth.

A validity mask is also generated during projection. It excludes points that fall outside the source image or lie behind the camera. The implementation performs the back-projection, transformation, projection, and loss calculations in FP32 to avoid numerical problems during the geometric operations.

3.9.4 Photometric Loss and Auto-Masking

The photometric loss measures the difference between the reconstructed image and the actual current frame. The implementation combines the Structural Similarity Index Measure (SSIM) and the L_1 pixel difference:

$$\mathrm{pe}(\hat{I}, I) = 0.85 \cdot \frac{1 - \mathrm{SSIM}(\hat{I}, I)}{2} + 0.15 \cdot \|\hat{I} - I\|_1. \tag{3.46}$$

The SSIM component gives more importance to local structural similarity, while the L_1 term measures the direct pixel-level difference. Their combination provides the photometric residual used by the reconstruction-based losses.

The pipeline also uses the auto-masking strategy from Monodepth2 [17]. This is useful because not every pixel provides a reliable reconstruction signal. For example, a pixel may have a lower photometric error in the unwarped images than after applying the predicted camera transformation. Such a pixel does not provide useful evidence for the estimated motion and is excluded from the loss.

For the three-frame setup, the implementation calculates the re-projection error for both neighboring source frames and uses the lower error at each pixel. The auto-mask compares this minimum re-projection error against the minimum identity error obtained directly from the unwarped source images:

$$\mu(\mathbf{p}) = \mathbf{1}\left[\min_{s\in\{t-1,t+1\}} \mathrm{pe}(\hat{I}_{t\leftarrow s}, I_t) < \min_{s\in\{t-1,t+1\}} \mathrm{pe}(I_s, I_t)\right]. \tag{3.47}$$

Only valid pixels that pass this comparison contribute to the final photometric loss. A small random perturbation is also added to the identity errors before the minimum operation to handle ties, following the Monodepth2 implementation.

3.9.4.1 Edge-Aware Smoothness

Photometric reconstruction alone may not provide sufficient constraints in textureless areas. To address this, the pipeline also includes an edge-aware smoothness term. The implementation applies this regularisation to the inverse depth, normalised by its mean value, rather than directly to the depth map.

Let $d_t = 1/D_t$ denote the inverse depth and let $\tilde{d}_t$ denote its mean-normalised form. The smoothness loss is then written as:

$$\mathcal{L}_{\text{smooth}} = \text{mean}\left(|\partial_x \tilde{d}_t| e^{-|\partial_x I_t|} + |\partial_y \tilde{d}_t| e^{-|\partial_y I_t|}\right). \tag{3.48}$$

This term encourages the predicted inverse depth to vary smoothly in regions where the image intensity changes only slightly. At the same time, the image-gradient weighting reduces the smoothness penalty near strong image edges, helping to preserve depth discontinuities.

3.9.4.2 Temporal and Geometric Consistency

In addition to photometric reconstruction, the notebook implements losses that directly compare depth predictions across neighboring frames. For this purpose, the depth map predicted for frame $t+1$ is warped into the coordinate system of frame t using the relative camera pose. Two depth values can then be compared at the corresponding locations: the warped depth from the next frame and the z-coordinate obtained by transforming the current frame's 3D point into the next camera coordinate system.

The resulting geometric consistency error is defined as:

$$\mathcal{L}_{\text{geom}} = \text{mean}_{\mathbf{p}\in\mathcal{V}} \left(\frac{|D_{\text{proj}}(\mathbf{p}) - D_{\text{compute}}(\mathbf{p})|}{D_{\text{proj}}(\mathbf{p}) + D_{\text{compute}}(\mathbf{p}) + \epsilon} \right), \tag{3.49}$$

where D_{proj} is the depth sampled from the neighboring frame after warping, D_{compute} is the transformed z-coordinate of the current 3D point, and $\mathcal{V}$ contains valid correspondences. Where applicable, pixels belonging to potentially moving objects are removed using the dynamic-object mask.

The `temporal_warp_consistency` loss uses this depth-consistency calculation and can optionally include an additional photometric term. In the recommended combined SSL formulation, the photometric contribution inside this temporal term is disabled, so that it provides a separate geometric consistency signal.

3.9.5 SSL Loss Registry

The ablation notebook contains a loss registry that allows different photometric, geometric, and regularization terms to be tested independently. In the current v6 implementation, the main registry contains 18 variants. Five additional variants are included for the stationary-object experiments, bringing the total number of registered losses to 23. These losses can be grouped according to their role:

- **Photometric and feature-based losses:** `photo_l1`, `photo_ssim`, `photo_ssim_l1`, `min_reproj_photo`, `multi_scale_photo`, and `feature_metric`.
- **Geometric and temporal losses:** `geometric_consistency`, `rigid_flow_consistency`, and `temporal_warp_consistency`. The `naive_temporal_delta` loss is retained separately as a negative control because it does not explicitly model camera motion.

- **Regularisation and combined SSL recipes:** `edge_aware_smooth`, `combined_monodepth2`, and `combined_recommended_ssl`. The first combined recipe uses multi-scale photometric reconstruction with smoothness, while the recommended SSL formulation combines photometric reconstruction, smoothness, and temporal consistency.

- **OXTS-based oracle-pose variants:** `gt_photo_reproj`, `gt_rigid_flow_consistency`, and `gt_temporal_warp_consistency`. These use the camera motion obtained from OXTS instead of the PoseNet prediction.

- **Hybrid and supervised reference losses:** `combined_recommended` and `lidar_anchor_only`. The former combines self-supervised terms with a sparse LiDAR-based SILog anchor, while the latter uses the sparse depth measurements as a supervised control. These are not pure self-supervised losses.

- **Stationary-object losses:** `stationary_depth_motion`, `stationary_uniform_motion`, `full_3d_ego_motion`, `mask_chamfer`, and `combined_stationary`. These use SAM3 instance masks and object tracking to obtain additional geometric consistency signals from objects classified as stationary.

The recommended pure self-supervised combination uses the following loss:

$$\mathcal{L}_{\text{combined,SSL}} = \lambda_{\text{photo}}\mathcal{L}_{\text{photo}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}} + \lambda_{\text{temporal}}\mathcal{L}_{\text{temporal}}, \tag{3.50}$$

with the default weights $\lambda_{\text{photo}} = 1.0$, $\lambda_{\text{smooth}} = 10^{-3}$, and $\lambda_{\text{temporal}} = 0.1$.

The registry also provides a hybrid version, `combined_recommended`, which adds a sparse LiDAR-based SILog term with a weight of 0.5. This is kept as a separate hybrid experiment and should not be treated as pure self-supervised fine-tuning.

Similarly, `lidar_anchor_only` is used as a supervised reference. The geometric consistency loss is available as an independent registry entry rather than as an additional VNL term in the combined recipe.

3.9.6 Data Augmentation and Experimental Setup

Since the training losses depend on the geometric relationship between consecutive frames, augmentation must be applied consistently to all frames in a sample. The implemented pipeline uses horizontal flipping and colour jitter. The same augmentation parameters are applied to the current frame and its neighbouring frames. When a horizontal flip is applied, the camera principal point is mirrored, and the relative camera pose is transformed accordingly.

Random cropping is not used in the SSL augmentation pipeline because it changes the image coordinate system and shifts the camera principal point. Unless the intrinsic matrix is updated to account for that change, the geometric warping operation would use incorrect camera parameters.

The ablation notebook uses an input resolution of 378×1246 px and a canonical set of samples for which the required OXTS pose and camera calibration are available. In the execution recorded in the notebook, this set contains 1,609 temporal samples, divided into 1,449 training samples and 160 validation samples. The separate test split contains 652 frames. Each HPT trial is configured for a maximum of five epochs, with early stopping patience of four epochs. The learning-rate candidates are 10^{-5} and 10^{-6}.

3.9.7 Combined Recipe Evaluation

Following the individual loss experiments, six different combinations were selected for the final self-supervised fine-tuning experiments. These recipes were designed to examine the effect of photometric reconstruction, rigid-flow consistency, feature-based reconstruction, and temporal depth consistency, both individually and in combination.

The recipes are divided into three groups based on how the relative camera motion is obtained. Groups A and B use the relative camera poses derived from KITTI OXTS measurements, whereas Group C uses the poses predicted by the PoseNet. It is important to note that the OXTS poses provide only the camera-motion information required for geometric reconstruction; they are not used as ground-truth depth targets. None of the six recipes uses LiDAR depth supervision or a sparse-depth anchor during training.

The loss compositions and their respective roles are summarised in Table 3.7.

Among these configurations, A3 produced the lowest validation RMSE and the lowest unscaled test RMSE in the supplied final comparison. The addition of edge-aware smoothness helps regularize the predicted depth, while the temporal consistency term provides an additional constraint by comparing depth predictions at geometrically corresponding locations in consecutive frames. The dynamic-object mask further limits this comparison to regions that are more likely to belong to stationary parts of the scene. Based on these results, A3 was selected as the best-performing recipe for self-supervised fine-tuning.

TABLE 3.7: Combined Loss Recipes for Self-Supervised Fine-Tuning

Tag	Loss Composition	Description
A1	$\mathcal{L}_{\text{GT-photo}}$	Photometric reprojection using relative camera poses obtained from KITTI OXTS. The loss uses the minimum reprojection error and auto-masking to exclude unreliable pixel correspondences.
A3*	$\mathcal{L}_{\text{GT-photo}} + 10^{-3}\mathcal{L}_{\text{smooth}} + 0.1\mathcal{L}_{\text{GT-temporal}}$	Combines OXTS-based photometric reprojection with edge-aware inverse-depth smoothness and temporal depth-warp consistency. A dynamic-object mask is also used to exclude potentially moving objects from the temporal consistency calculation.
B1	$\mathcal{L}_{\text{GT-rigid}-\text{flow}}$	Uses OXTS-derived camera motion to calculate rigid flow from the predicted depth. The loss combines photometric reconstruction with edge-aware smoothness of the flow magnitude.
C1	$\mathcal{L}_{\text{PoseNet-rigid}-\text{flow}}$	Uses PoseNet-predicted camera motion instead of OXTS poses. The rigid-flow consistency loss combines photometric reconstruction and flow smoothness.
C3	$\mathcal{L}_{\text{photo-L1}} + 0.1\mathcal{L}_{\text{flow-smooth}}$	Combines an L_1 photometric reconstruction loss with an edge-aware smoothness term applied to the magnitude of the predicted rigid flow. Camera motion is estimated using PoseNet.
C5	$\mathcal{L}_{\text{photo-L1}} + 0.1\mathcal{L}_{\text{feature}}$	Combines pixel-level L_1 reconstruction with feature-space consistency. The feature loss compares warped and target image features extracted using a frozen ResNet-18 backbone, while also retaining a pixel-level reconstruction component. PoseNet provides the relative camera motion.

* A3 achieved the best reported performance among the completed runs in the final comparison.

3.9.8 Final Fine-Tuning

Based on the results of the combined recipe evaluation, A3 was identified as the best-performing configuration for self-supervised fine-tuning. This recipe combines photometric reprojection, edge-aware smoothness, and temporal depth consistency using camera poses obtained from KITTI OXTS measurements. The final training setup uses the larger KITTI dataset and the following configuration:

- **Training dataset:**

  The canonical dataset contains 23,092 temporal samples with complete OXTS pose and camera calibration information. A 90/10 split, generated using random seed 42, results in 20,783 training samples and 2,309 validation samples. The separate test set contains 652 frames.

- **Loss configuration:**

  Recipe A3 combines three loss components: OXTS-based photometric reprojection, edge-aware smoothness, and temporal depth-warp consistency. Their respective weights are kept fixed throughout training:

$$\mathcal{L}_{\text{A3}} = 1.0\mathcal{L}_{\text{GT-photo}} + 10^{-3}\mathcal{L}_{\text{smooth}} + 0.1\mathcal{L}_{\text{GT-temporal}}. \tag{3.51}$$

  The temporal consistency term uses the relative camera motion from OXTS measurements. A dynamic-object mask derived from SAM3 is also applied to exclude potentially moving regions from this term.

- **Depth model and architecture:**

  The DA3METRIC-LARGE model is used for fine-tuning. Only the decoder is trainable, while the backbone remains frozen. This corresponds to the `decoder_only` architecture used throughout the final training experiments.

- **Input data and augmentation:**

  Three consecutive frames are used to establish the temporal relationship, with source-frame offsets of $(-1, +1)$. The input images are resized to 378×1246 px. The base augmentation strategy is retained, consisting of horizontal flipping and colour jitter. Camera intrinsics and relative poses are also adjusted consistently when horizontal flipping is applied.

- **Batch size:**

  A batch size of 4 is used with gradient accumulation over 8 steps, giving an effective batch size of 32. This keeps the per-step memory demand lower while retaining the same effective batch size.

- **Learning rate and optimizer:**

  AdamW is used with a learning rate of 10^{-6} and weight decay of 10^{-4}. The learning rate follows a cosine-warmup schedule, with linear warmup during the first epoch followed by cosine decay. Gradient clipping is applied with a maximum norm of 1.0.

- **Training duration and early stopping:**

  The maximum training duration is set to 50 epochs, with early-stopping patience of 15 epochs. The validation RMSE is monitored after each epoch, and the checkpoint with the lowest validation RMSE is retained as the best model. The recorded run completed 42 epochs, and its best-validation checkpoint was selected at epoch 27.

- **Numerical precision:**

  Mixed-precision training is enabled using FP16 and gradient scaling. However, the geometric operations involved in back-projection, camera transformation,

projection, and image warping, along with the loss calculations, are performed in FP32 to maintain numerical stability.

- **LiDAR usage and evaluation:**

  LiDAR depth is not loaded by the training data loader and is not used in any of the self-supervised loss terms. It is used only during validation to calculate RMSE for checkpoint selection and early stopping, and during the final evaluation on the held-out test set. The test set is evaluated using the best-validation checkpoint after training, and is not involved in model selection.

In the recorded final comparison, recipe A3 achieved its best validation RMSE of 2.5035 m at epoch 27, with an unscaled test RMSE of 2.5266 m. These results were used to identify A3 as the best-performing self-supervised configuration among the evaluated recipes.

3.10 Stage 9: Sequential Combined Fine-Tuning

The combined experiment starts from the selected checkpoint produced by Stage 8 self-supervised fine-tuning Recipe A3 and continues training with the supervised fine-tuning Recipe A4 objective using KITTI LiDAR depth. This ordering is intended to retain temporal/geometric priors from the self-supervised phase while adapting the model to metric ground truth. The combined run is evaluated with the same held-out KITTI test protocol.

Chapter 4

Datasets and Evaluation Protocol

4.1 Scope of the Dataset Used

The experiments presented in this thesis are carried out exclusively on the KITTI Eigen-Split benchmark. This choice was mainly due to the limited computational resources available for the work, which restricted the possibility of conducting experiments on multiple large-scale datasets. Although the repository includes configurable templates for processing other datasets, their presence does not imply that they were used for training or evaluation in this thesis. Datasets such as nuScenes, the India Driving Dataset (IDD), and TIAND are mentioned as possible extensions of the proposed pipeline. However, no experimental results from these datasets are reported or included in the comparisons presented in Chapter 5.

4.2 KITTI Eigen-Split

The KITTI dataset was collected in and around Karlsruhe, Germany, using a vehicle fitted with a forward-facing stereo camera system, a Velodyne HDL-64E LiDAR sensor, and an inertial/GPS navigation system [15]. For monocular depth estimation, the Eigen split introduced by Eigen et al. [10] is commonly used as a benchmark for training and evaluating depth estimation models. In this work, the RGB images are used as input to the model, while the corresponding projected LiDAR points provide sparse metric depth values that are used for supervision and evaluation.

4.2.1 Depth representation and valid pixels

The KITTI depth maps store depth values as unsigned 16-bit integers. To obtain the actual depth in metres, the stored values are divided by 256, as shown below:

$$D_{\text{gt}} = \frac{D_{\text{stored}}}{256.0} \quad [\text{m}], \tag{4.1}$$

A stored value of zero indicates an invalid pixel. During evaluation, only valid ground-truth depth values within the range specified for the experiment (typically 0.001–80 m) are considered. The Eigen crop is also applied wherever required, following the evaluation protocol introduced by Eigen et al. [10]. This crop excludes the upper and lower portions of the image from the evaluation. Since LiDAR provides depth measurements only at certain pixels, the evaluation metrics are calculated using these valid ground-truth locations rather than treating the predicted dense depth maps as ground truth at every pixel.

4.2.2 Training, validation, and test data

The KITTI Eigen-Split training frames are used for fine-tuning and validation, while the held-out test frames are reserved for the final evaluation. According to the saved evaluation summaries, 652 test frames were successfully used for the final evaluation. Any frames without valid LiDAR depth measurements are excluded when calculating the evaluation metrics. It is also worth noting that the number of samples and the train–validation split vary across different ablation studies and final training experiments. Therefore, the dataset sizes should be considered in the context of their respective experimental configurations rather than assumed to follow one fixed split. The model checkpoint is selected based on its performance on the validation set, whereas the test set is used only for the final evaluation and does not influence hyperparameter selection.

For fine-tuning experiments that use supervised learning, sparse LiDAR measurements are used as depth supervision. In contrast, the self-supervised experiments make use of temporally adjacent KITTI frames and, in configurations where pose information is required, the corresponding motion data from the KITTI OXTS records. Therefore, these self-supervised experiments are not completely independent of additional sensor information during training. However, the final deployment objective remains the same: to estimate metric depth from a single RGB frame without requiring LiDAR input.

4.3 Other Dataset Pipelines: Implemented Templates, Not Evaluated Data

Apart from KITTI, the repository also contains a configurable notebook for preparing depth maps from LiDAR point clouds for use with other driving datasets. These implementations were included to make it easier to extend the proposed pipeline to different datasets in future work. However, their inclusion in the repository does not mean that they were used in the training or evaluation experiments presented in this thesis.

- **nuScenes.** The nuScenes dataset provides multi-camera driving sequences along with LiDAR sweeps collected in Boston and Singapore [6]. A template is available in the repository for loading point clouds stored in binary format and projecting them onto camera images using the corresponding calibration parameters. However, this dataset was not used in the experiments reported in this thesis.

- **India Driving Dataset (IDD).** IDD contains driving data from Indian roads and covers a wide range of road users and traffic conditions [39]. The repository includes a configurable pipeline for point-cloud processing and depth-map preparation for this dataset. However, no training or evaluation was carried out using IDD for the results presented in this work.

- **TIAND.** The Traffic and Indian Autonomous Navigation Dataset is another potential source of driving data from Indian road environments. The repository contains a configurable pipeline for processing point clouds stored in PCD format and preparing the corresponding depth maps. However, TIAND was also not included in the reported experiments.[23]

Similarly, the additional RGB dashcam datasets and driving sequences mentioned in the exploratory configuration files were not used to generate the quantitative results presented in this thesis, unless explicitly mentioned in a saved experimental result. These datasets and sequences are therefore considered possible sources of data for future work rather than part of the experiments completed in this study.

4.4 Evaluation Protocol

The evaluation of the models and different experimental configurations follows the steps given below:

1. All pretrained models are evaluated using the same KITTI Eigen-Split test frames and valid-depth criteria, wherever their output formats allow a consistent comparison.

2. SAM3 masks are generated in advance and reused for the category-wise evaluation. For each category, the evaluation considers only those pixels that fall within the corresponding mask and also have valid ground-truth depth values.

3. For supervised fine-tuning, KITTI LiDAR depth maps are used as the training targets. The validation set is used to select the best-performing checkpoint, while the held-out test set is kept separate and used only for reporting the final evaluation results.

4. In the self-supervised experiments, consecutive RGB frames are used along with the pose information specified in the respective training configuration. For experiments that use KITTI OXTS poses, this dependency is explicitly mentioned.

5. The results obtained without any scale correction are reported separately from those obtained using oracle-based scale alignment and vehicle-anchored post-processing. This allows the direct metric-depth performance of the models to be distinguished from the improvements obtained through additional corrections.

6. Each numerical result is reported with reference to its corresponding training run, the rule used for checkpoint selection, and the evaluation method used for aggregating the metrics.

Chapter 5

Analysis and Results

5.1 Overview

Results are reported stage by stage from saved evaluation outputs. Unless stated otherwise, the fine-tuning RMSE values in this chapter are unscaled pixel-weighted metrics on the 652-frame KITTI Eigen-Split test set. The tested recipes and checkpoint-selection metrics are identified explicitly so that supervised, self-supervised, and sequential combined runs are not confused.

5.2 Stage 1: SAM3 Segmentation Quality

Across the 652-frame KITTI Eigen-Split test set, the SAM3 pipeline generated 12,808 valid mask instances across all eight super-categories.

- **Vehicles:** 2,108 vehicle masks, 1,819 vehicle-wheel masks, and 642 vehicle-light masks. Vehicle and wheel masks provide candidate regions for the orientation-aware anchor selection in Stage 5.
- **Pedestrians:** 482 masks. Detection degrades beyond 20 m because the pixel footprint becomes too small for SAM3 confidence to exceed 0.40.
- **Road:** 1,070 masks. Large contiguous regions with dense LiDAR coverage. Reliable for range-wise error analysis.
- **Cyclists and animals:** 204 cyclist masks and 2 animal masks, the least represented categories in the saved mask set.
- **Stationary objects:** 6,481 masks (trees, poles, buildings, signs). This is the largest category by count.
- **NMS:** The configured IoU threshold is 0.60; the saved evaluation contains 12,808 mask instances across eight super-categories.

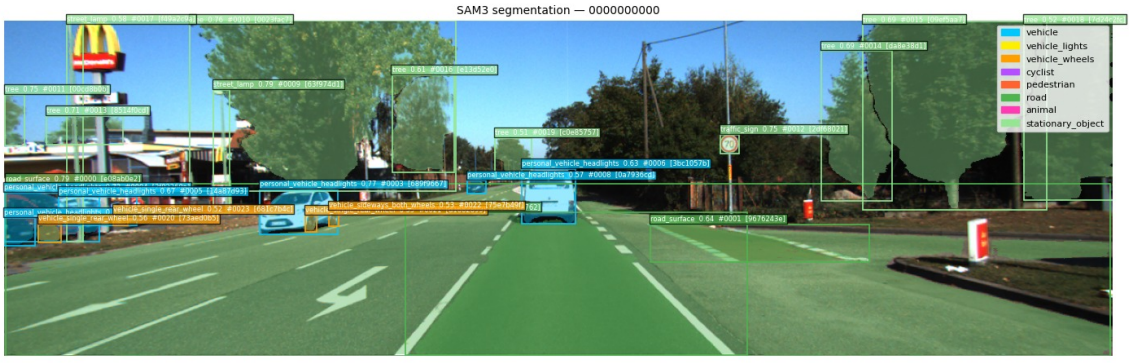


FIGURE 5.1: Overview of the SAM3 instance segmentation mask of one of the KITTI dataset imagery to visualize the scene segmentation

5.3 Stage 2: Baseline Multi-Model Comparison

5.3.1 Unscaled Zero-Shot Results

The first stage of the evaluation involved comparing all five depth estimation models without applying any additional scale correction. Each model was evaluated in its original zero-shot configuration on the KITTI Eigen-Split test set, consisting of 652 frames. The evaluation was performed using the Eigen crop, with the predicted depth values restricted to the range $D \in [0.001, 80]\,\text{m}$. The results obtained from the final whole-frame evaluation are presented in Table 5.1.

TABLE 5.1: Unscaled zero-shot performance on the KITTI Eigen-Split test set (652 frames, Eigen crop, $D \in [0.001, 80]\,\text{m}$). All metrics are computed using pixel-weighted whole-frame evaluation.

Model	**RMSE (m)↓**	**AbsRel↓**	$\boldsymbol{\delta_1}$ **↑**
DA3-M (Metric-Large)	**2.9379**	0.1001	0.9346
UD2 (UniDepth V2 ViTL14)	2.9944	**0.0761**	**0.9535**
DepthPro (apple/DepthPro-hf)	3.7375	0.1446	0.8276
DA2 (Metric-Outdoor-Large)	4.0539	0.1451	0.8217
DA3-G (Nested-Giant-1.1)	5.4960	0.2078	0.5134

From the results, DA3-M achieves the lowest unscaled RMSE of 2.9379 m, with UniDepth V2 following closely at 2.9944 m. However, UniDepth V2 produces the lowest AbsRel of 0.0761 and the highest δ_1 of 0.9535, showing better performance in terms of relative depth accuracy and threshold-based accuracy. Although DA3-M does not achieve the best result across all three metrics, its performance is relatively balanced. More importantly, its decoder architecture allows it to be fine-tuned using a decoder-only training strategy, while its available open-source implementation

provides access to the computational graph required for gradient-based optimization. For these reasons, DA3-M was selected as the primary model for the subsequent fine-tuning experiments.

DA3-G shows the highest error among the five models in the unscaled evaluation, with an RMSE of 5.4960 m. However, its performance improves considerably after applying oracle affine alignment, reducing the RMSE to 2.7564 m. This indicates that a considerable portion of its initial error is associated with scale and shift differences. Since this alignment requires LiDAR ground truth, it cannot be directly used during normal inference. Therefore, DA3-G is used only for camera intrinsic estimation in the proposed pipeline rather than as the primary depth estimation model.

It is also worth mentioning that the separate trend-analysis results show a positive signed bias for DA3-M. Under the sign convention used in that analysis, this indicates a tendency to predict smaller depth values than the corresponding reference depths. These bias measurements are discussed separately because the whole-frame evaluation summary used in Table 5.1 reports only RMSE, AbsRel and δ_1.

5.3.2 Scale-Aligned Comparison

To understand how much of the error in the zero-shot predictions is caused by differences in depth scale, the models were also evaluated using three additional alignment strategies: median scaling, mean scaling and affine alignment. In all three cases, LiDAR ground-truth depths are used to estimate the required correction. Therefore, these results should be treated as oracle-aligned references rather than results that can be obtained directly during inference.

Table 5.2 presents the RMSE, AbsRel and δ_1 values for all five models under the four evaluation settings, including the original unscaled predictions.

TABLE 5.2: Performance of the five depth estimation models under different scale-alignment strategies (652 frames, pixel-weighted whole-frame evaluation). Median, mean and affine alignment use LiDAR ground truth as oracle references.

Model	**Strategy**	**RMSE (m)↓**	**AbsRel↓**	$\boldsymbol{\delta_1}$ **↑**
DA3-M	Unscaled	2.9379	0.1001	0.9346
	Median	2.7109	0.0677	0.9669
	Mean	2.5678	0.0675	0.9694
	Affine	2.4620	0.0601	0.9719
UD2	Unscaled	2.9944	0.0761	0.9535
	Median	2.5836	0.0504	0.9769
	Mean	2.4654	0.0513	0.9789
	Affine	2.3821	0.0521	0.9770
DepthPro	Unscaled	3.7375	0.1446	0.8276
	Median	3.0477	0.0647	0.9573
	Mean	2.7451	0.0718	0.9619
	Affine	2.5362	0.0717	0.9565
DA2	Unscaled	4.0539	0.1451	0.8217
	Median	3.6611	0.1022	0.8999
	Mean	3.4054	0.1015	0.9060
	Affine	3.3124	0.1042	0.8993
DA3-G	Unscaled	5.4960	0.2078	0.5134
	Median	3.1221	0.0654	0.9587
	Mean	2.8916	0.0699	0.9606
	Affine	2.7564	0.0711	0.9598

The results show that all five models achieve a lower RMSE after affine alignment. For example, DA3-M improves from 2.9379 to 2.4620 m, while UniDepth V2 improves from 2.9944 to 2.3821 m. A similar reduction can be observed for DepthPro, DA2

and DA3-G. Among the five models, DA3-G shows a particularly large improvement over its original unscaled result.

These observations suggest that the difference in predicted depth scale and shift contributes considerably to the overall error. However, the aligned results rely on LiDAR ground truth and cannot be considered equivalent to a practical inference-time solution. Instead, they provide an oracle reference for understanding how much improvement is theoretically possible through global depth correction. This comparison is particularly useful when evaluating the effectiveness of the training-free correction approach introduced in Stage 5.

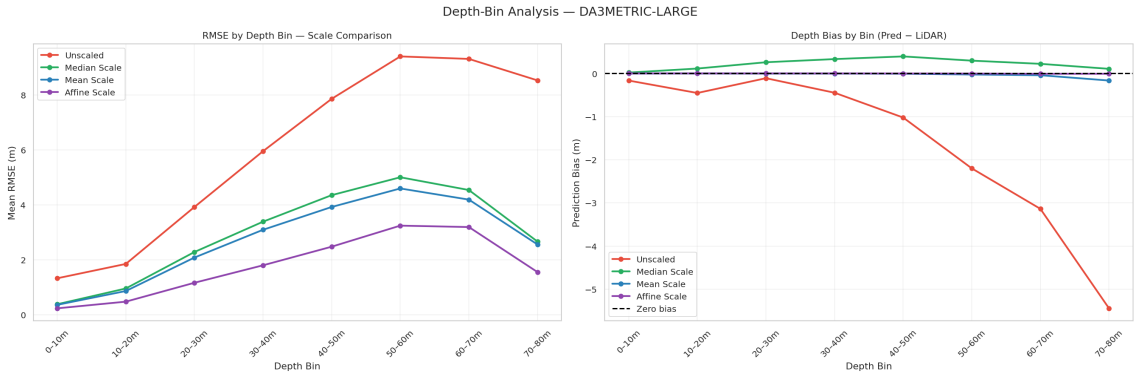


FIGURE 5.2: Scale-Aligned comparison and Depth bin analysis for DA3-M

5.3.3 Per-Category Analysis (DA3-M)

Apart from evaluating the models over the entire image, an additional evaluation was performed using the object masks generated by SAM3. This allows the depth estimation performance to be examined separately for different object and scene categories rather than considering all valid pixels together. For this analysis, DA3-M was evaluated over all 12,808 valid mask instances obtained from the 652 test frames.

Table 5.3 presents the unscaled pixel-weighted RMSE for each super-category. The RMSE for each category is calculated using the valid LiDAR depth measurements falling within its corresponding SAM3 masks.

TABLE 5.3: Unscaled per-category RMSE of DA3-M from the final 652-frame mask-level evaluation. Each category is evaluated over the valid LiDAR pixels within its SAM3 masks.

Category	**#Masks**	**Unscaled RMSE (m)↓**
animal	2	1.3987
cyclist	204	1.9167
pedestrian	482	2.6805
road	1,070	1.3594
stationary_object	6,481	2.8404
vehicle	2,108	1.8537
vehicle_lights	642	0.9569
vehicle_wheels	1,819	1.2177
All masks	12,808	1.9391

From the category-wise results, stationary objects have the highest RMSE of 2.8404 m, followed by pedestrians at 2.6805 m. Stationary objects also account for the largest number of masks, with 6,481 instances out of the total 12,808. On the other hand, vehicle lights and vehicle wheels show considerably lower errors, with RMSE values of 0.9569 m and 1.2177 m, respectively. The animal category also has a relatively low RMSE, although this result should be interpreted carefully because it is based on only two masks.

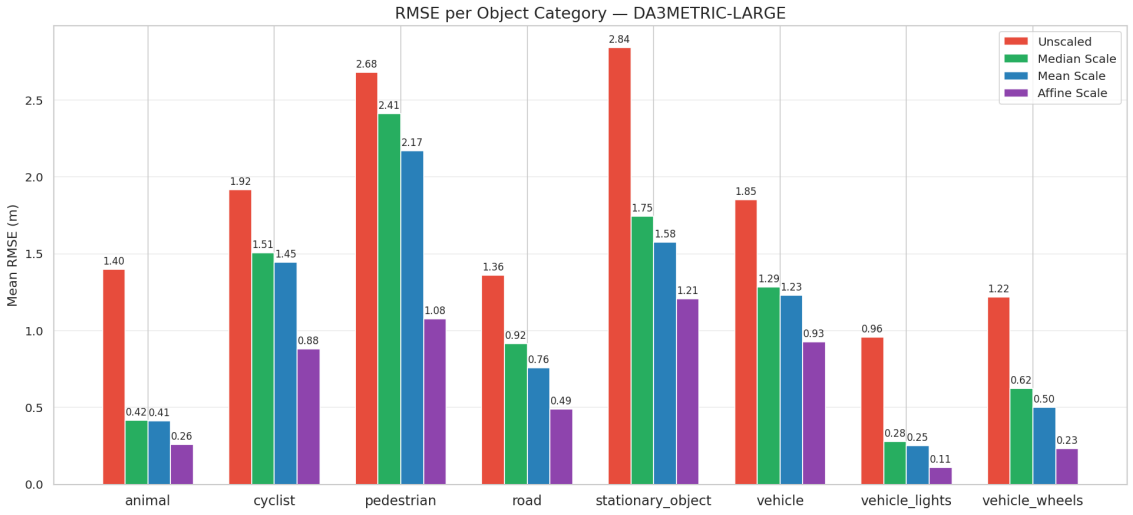


FIGURE 5.3: Multi scale-aligned Object-category RMSE across scale-alignment strategies

The overall unscaled RMSE across all valid masks is 1.9391 m. This value is lower than the corresponding whole-frame RMSE of 2.9379 m because the two evaluations consider different pixel regions. The mask-level evaluation is restricted to valid LiDAR pixels within the SAM3 masks, whereas the whole-frame evaluation considers the complete valid region of the image.

An additional experiment was carried out to study the effect of limiting the evaluation depth range on the mask-level performance of DA3-M. Three configurations were considered: clip80, clip40 and noclip. The results obtained from these experiments are presented below.

TABLE 5.4: Results of different Clip variants for DA3-M on the mask-level performance

Clip Variant	**#Masks**	**RMSE (m)**	**MAE (m)**	**AbsRel**	$\boldsymbol{\delta_1}$
clip80	12,420	1.9372	2.9531	0.1222	0.9035
clip40	10,715	1.4537	1.8910	0.1149	0.9233
noclip	12,439	1.9747	3.0540	0.1239	0.9018

Among the three configurations, clip40 gives the lowest RMSE of 1.4537 m, compared to 1.9372 m for clip80 and 1.9747 m for noclip. Its MAE also decreases from 2.9531 to 1.8910 m. A similar improvement can be seen in AbsRel and δ_1, which change from 0.1222 to 0.1149 and from 0.9035 to 0.9233, respectively, when moving from clip80 to clip40. This indicates that limiting the evaluation to depths up to 40 m improves the overall mask-level results. The higher errors observed in the other two configurations also suggest that predictions at larger distances contribute considerably to the total error.

The signed-bias results provide another useful observation. Even after restricting the evaluation to 40 m, the frame-level mean bias remains positive at +0.8452 m, although it is lower than the +1.0357 m observed for clip80. This indicates that the underestimation tendency is not limited to distant objects and is also present within the 0–40 m range.

5.4 Stage 3: DA3-M Error Pattern Analysis

To understand the nature of the errors produced by DA3-M, a separate signed-difference analysis was performed on the KITTI Eigen-Split test set. Instead of considering only the magnitude of the error, this analysis also examines whether the predicted depth is smaller or larger than the corresponding LiDAR depth. For this purpose, the difference is calculated as LiDAR depth − Predicted depth. Therefore, a positive value indicates depth underestimation, whereas a negative value indicates depth overestimation.

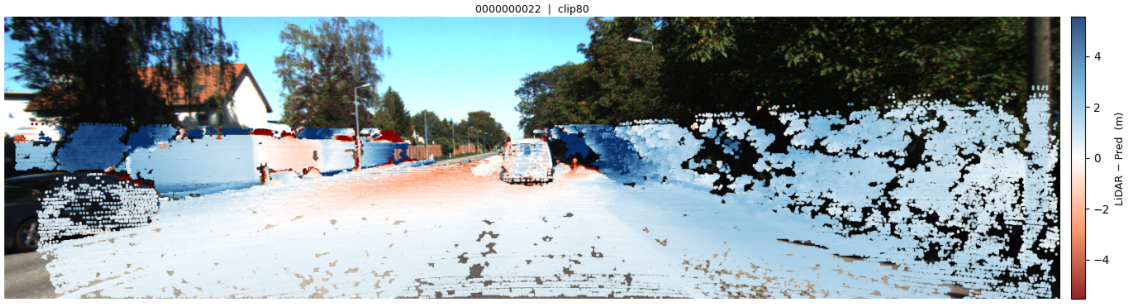


FIGURE 5.4: Image Overview for DA3-M signed-difference distribution at frame level (652 frames, clip80)

Table 5.5 shows the distribution of these differences across the valid LiDAR pixels for the clip80 configuration.

TABLE 5.5: DA3-M signed-difference distribution at frame level (652 frames, clip80). Positive differences indicate depth underestimation, while negative differences indicate overestimation.

Difference Category	**Pixel %**	**Direction**
Extreme Positive ($> 5.0\,\text{m}$)	4.23%	Severe underestimation
Too Positive (2.0–5.0 m)	9.71%	Significant underestimation
Positive (0.1–2.0 m)	**67.60%**	Mild underestimation
Zero ($\leq 0.1\,\text{m}$)	3.84%	Negligible error
Negative (-2.0 to $-0.1\,\text{m}$)	11.09%	Mild overestimation
Too Negative (-5.0 to $-2.0\,\text{m}$)	2.26%	Significant overestimation
Extreme Negative ($< -5.0\,\text{m}$)	1.26%	Severe overestimation

Underestimation: 81.54%; Overestimation: 14.61%; Negligible: 3.84%.

The distribution shows that the errors are mostly on the positive side, with 81.54% of the valid LiDAR pixels falling into the underestimation categories. Within this group, the largest contribution comes from the 0.1–2.0 m range, which alone accounts for 67.60% of all evaluated pixels. In comparison, only 14.61% of the pixels fall into

the overestimation categories, while 3.84% have a negligible difference of at most 0.1 m.

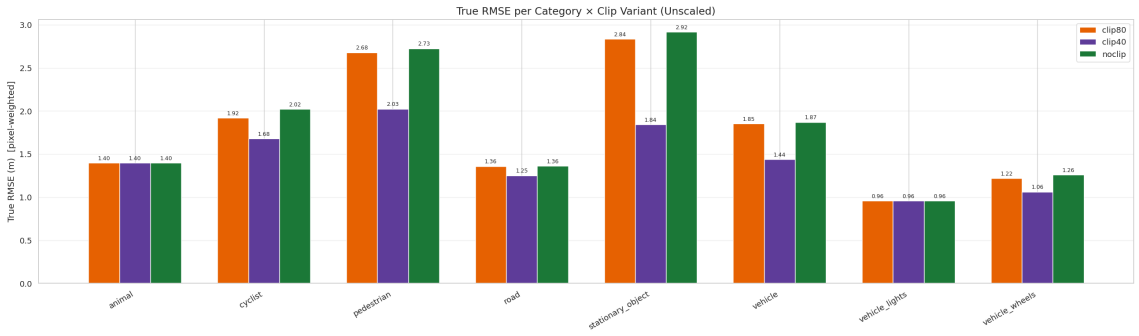


FIGURE 5.5: Unscaled RMSE by category and clip variant for DA3-M

A similar pattern can also be observed in the category-wise results. All eight evaluated mask categories have a positive mean bias under the adopted sign convention. The vehicle category has the highest mean bias of +1.9279 m, followed by animal masks at +1.3728 m and vehicle wheels at +1.1244 m. Pedestrians show the lowest category-wise bias at +0.3802 m. The relatively high bias for vehicle masks is particularly relevant to the vehicle-anchor-based correction method explored later in Stage 5.

The frame-level results also show that the mean bias decreases from +1.0357 m for clip80 to +0.8452 m for clip40, while remaining positive in both cases. Thus, restricting the depth range reduces the magnitude of the bias but does not eliminate the overall tendency towards underestimation.

Taken together, these results indicate that the depth estimation errors produced by DA3-M are not distributed evenly between underestimation and overestimation. A large proportion of the predictions show a consistent tendency to produce smaller depth values than the LiDAR reference. This directional behaviour suggests that the error is partly systematic rather than being entirely random. Therefore, simply averaging the errors is unlikely to address the underlying problem. These observations

provide the motivation for the subsequent correction approach in Stage 5, where global depth correction and vehicle-based depth anchors are investigated.

5.5 Stage 5: Vehicle-Anchored Correction Results

5.5.1 Anchor Fit Quality

The vehicle-anchoring method was evaluated on the same 652-frame KITTI Eigen-Split test set. A total of 1,980 vehicle anchors were used for fitting the correction parameters. For the scale-and-shift configuration, RANSAC retained 1,249 anchors as inliers (63.1%), resulting in a scale of $s = 1.0499$ and a shift of $t = +0.3885\,\text{m}$. The corresponding anchor-fit RMSE was 1.199 m. For the scale-only configuration, 1,243 anchors were retained as inliers, giving a scale factor of $s = 1.0673$ with no shift.

It should be noted that the anchor-fit RMSE represents the error between the selected vehicle anchors and their geometric reference values. It is different from the average RMSE reported in the final test-set evaluation and should not be interpreted as the overall depth estimation error.

5.5.2 Correction Results

TABLE 5.6: Comparison of vehicle-anchored correction with oracle-based alignment on the 652-frame KITTI Eigen-Split test set. RMSE represents average RMSE, not a pixel-weighted aggregate. Oracle methods use LiDAR ground truth for alignment.

Method	**Scale**	**Shift (m)**	**Avg RMSE (m)↓**	**Avg AbsRel↓**	**Avg δ_1 ↑**
Unscaled DA3-M	1.0000	–	3.7131	0.1342	0.8955
Median scale (oracle)	1.0673	–	1.9082	0.0394	0.9713
Mean scale (oracle)	1.0597	–	1.7429	0.0421	0.9726
Affine scale+shift (oracle)	0.4770	+13.8862	1.0682	0.0257	0.9873
Vehicle-anchored scale+shift	**1.0499**	**+0.3885**	**3.4897**	**0.1164**	**0.9099**
Vehicle-anchored scale-only	1.0673	0.0000	3.5278	0.1197	0.9080

The vehicle-anchored correction provides a modest improvement over the unscaled predictions. The scale-and-shift configuration reduces the average RMSE from 3.7131 to 3.4897 m (approximately 6.0%), while AbsRel decreases from 0.1342 to 0.1164. The scale-only configuration achieves an average RMSE of 3.5278 m, corresponding to an improvement of approximately 5.0%. Overall, scale-and-shift performs slightly better than scale-only across all three metrics.

The oracle affine alignment achieves a much lower RMSE of 1.0682 m. However, this correction requires LiDAR ground truth and is therefore included only as a reference. The gap between the oracle and vehicle-anchored results indicates that the estimated vehicle geometry does not fully capture the depth variations present in the scene.

The category-wise results also show that the improvement is not consistent across all objects. With scale-and-shift correction, RMSE decreases for animals (1.42 to 0.69 m), roads (1.54 to 1.13 m), vehicles (3.23 to 2.76 m), vehicle lights (0.95 to 0.51 m) and vehicle wheels (1.34 to 0.83 m). However, pedestrian RMSE increases slightly

from 3.29 to 3.36 m, while stationary objects show only a small improvement from 5.26 to 5.22 m.

These results suggest that vehicle anchoring can partially correct the depth estimates, although its effectiveness depends on the object category. Errors in vehicle dimensions, segmentation boundaries, orientation estimation and camera intrinsics may also affect the accuracy of the estimated anchors. Therefore, the method provides a modest training-free improvement on the KITTI test set, but its performance remains considerably below oracle alignment.

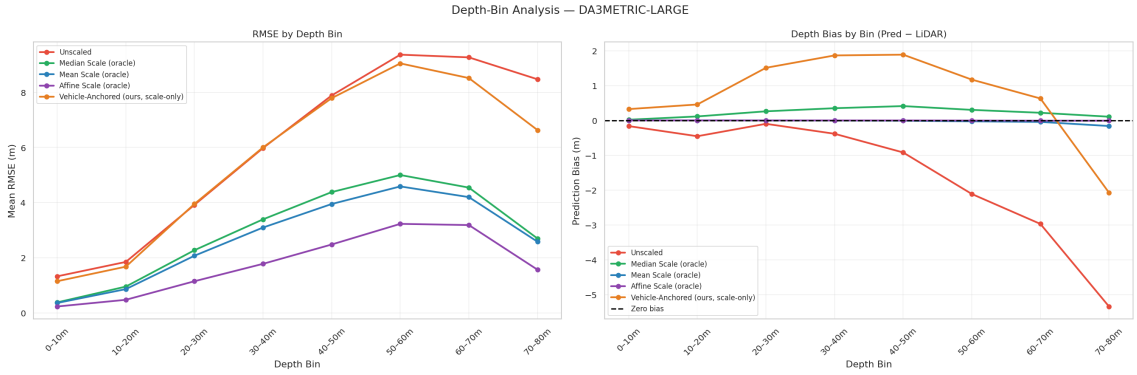


FIGURE 5.6: Only-Scale vehicle anchored Depth Bin Analysis for DA3-M

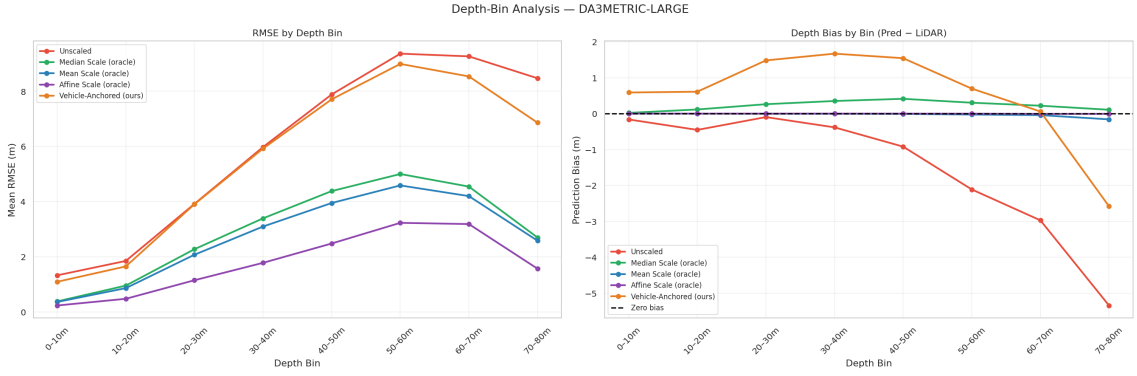


FIGURE 5.7: Scale-Shift vehicle anchored Depth Bin Analysis for DA3-M

5.6 Stage 7: Supervised Fine-Tuning Results

5.6.1 Individual Loss Function Ablation

The first experiment focused on comparing different loss functions for supervised fine-tuning of DA3-M. A total of 13 loss functions were tested while keeping the model architecture, augmentation and optimiser settings unchanged. The experiments were performed on the smaller KITTI training subset, with 488 training samples, 85 validation samples and a separate 46-frame held-out test set. The results are presented in Table 5.7.

TABLE 5.7: Comparison of individual loss functions using DA3-M decoder-only fine-tuning, base augmentation and AdamW with cosine warmup. Three learning-rate candidates were tested for each configuration, with a maximum of five epochs and early stopping patience of three epochs. RMSE represents pixel-weighted RMSE on the 46-frame held-out test set.

Loss Function	**PW-RMSE (m)↓**	**AbsRel↓**	**δ_1 ↑**	**Bias (m)**
MAE-Canonical (DepthPro)	**2.4997**	**0.0719**	**0.9702**	+0.0317
Combined-Base (SILog+BerHu+L1+Grad)	2.5262	0.0725	0.9676	−0.1498
SILog (Eigen 2014)	2.5376	0.0731	0.9641	+0.0514
BerHu (Laina 2016)	2.5868	0.0787	0.9614	−0.1651
SSIM + L1 (Godard 2017)	2.6201	0.0786	0.9620	−0.0739
Masked Depth (DA3 $\mathcal{L}_D$)	2.6725	0.0756	0.9624	−0.0044
EG-SSI (UniDepth V2)	2.6739	0.0832	0.9687	+0.0488
Robust Cauchy (Barron 2019)	2.7602	0.0830	0.9636	−0.0532
Log-L1 (AdaBins)	2.8042	0.0949	0.9482	−0.3947
DA3-Video Combined	2.9101	0.0753	0.9583	+0.2729
Affine-Invariant SILog (MiDaS)	2.9870	0.1150	0.9272	−0.9160
VNL (Yin 2019)	4.3563	0.1712	0.7175	+2.5846
TGM (VideoDA, sequential)	6.5545	0.1696	0.6792	+2.7955

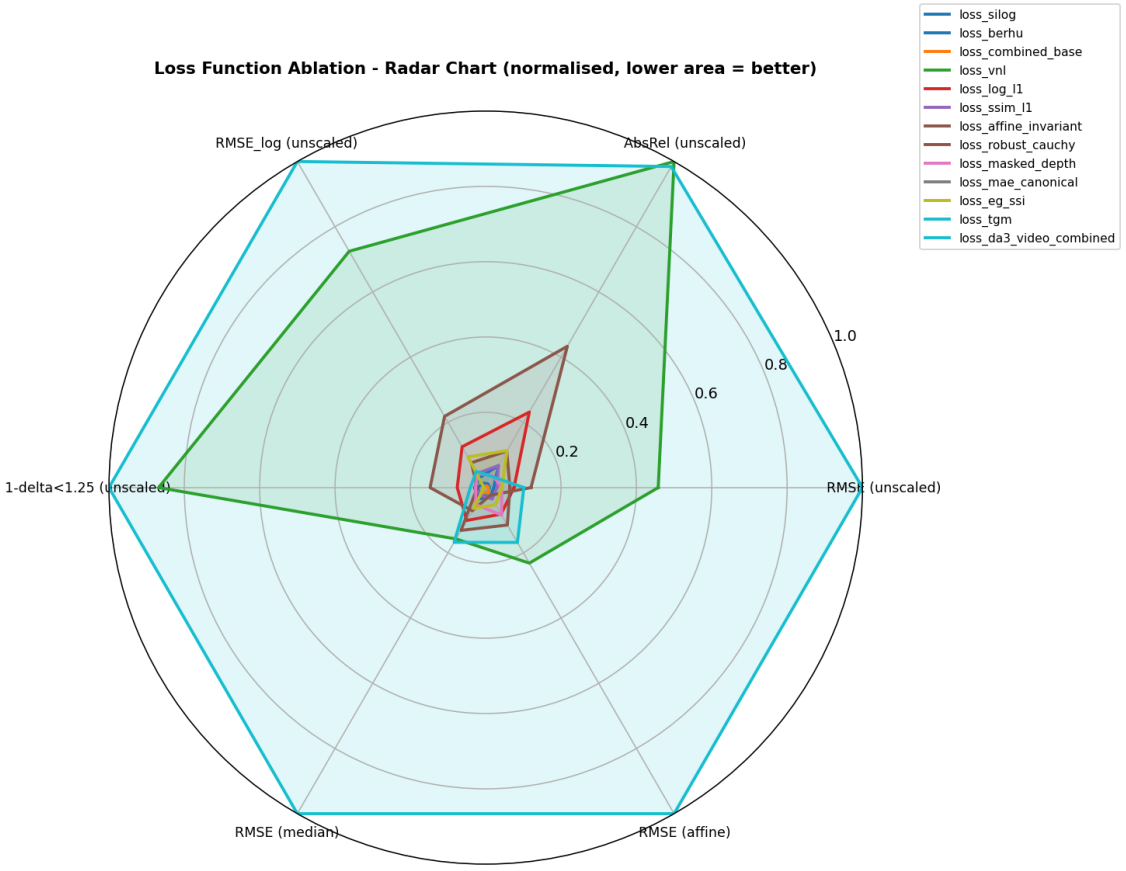


FIGURE 5.8: Radar chart for all individual loss function ablation study for supervised fine-tuning, normalized, lower area is better

MAE-Canonical gives the lowest RMSE of 2.4997 m, along with the best AbsRel and δ_1 among the tested loss functions. SILog and Combined-Base also give reasonably close results, with RMSE values of 2.5376 m and 2.5262 m, respectively. The performance of VNL and TGM is considerably poorer, particularly in terms of RMSE and threshold accuracy. The limited availability of valid LiDAR points may affect the geometric constraints used by VNL, while TGM depends on temporal consistency between consecutive frames, which is difficult to exploit with the limited sequential training samples.

Affine-Invariant SILog also performs worse than the other main loss functions. Since the objective is to learn metric depth, removing scale dependence during supervision

does not provide the same constraint on absolute depth values. Based on these results, MAE-Canonical was selected for the subsequent architecture, augmentation and optimiser experiments.

5.6.2 Architecture Ablation

The next experiment examined the effect of updating different parts of the DA3-M network during fine-tuning. Three configurations were compared: training the complete model, a partial decoder/head-unfreezing configuration, and training only the decoder and depth head. The intermediate configuration retains the code label `last2_blocks`, but it does not literally unfreeze the last two encoder blocks. In the supplied implementation, it unfreezes a selected later portion (approximately 30%) of the decoder/head-related parameters according to the parameter-name ordering. It is therefore described here as partial decoder/head unfreezing rather than as a two-encoder-block experiment.

TABLE 5.8: Architecture ablation using MAE-Canonical loss, base augmentation and AdamW with cosine warmup on the 46-frame test set.

Architecture	**PW-RMSE (m)↓**	**AbsRel↓**	$\boldsymbol{\delta_1}$ **↑**
Full (all parameters)	2.5056	0.0713	0.9662
Partial decoder/head	2.5124	0.0793	0.9660
Decoder-only	2.5714	0.0718	**0.9690**

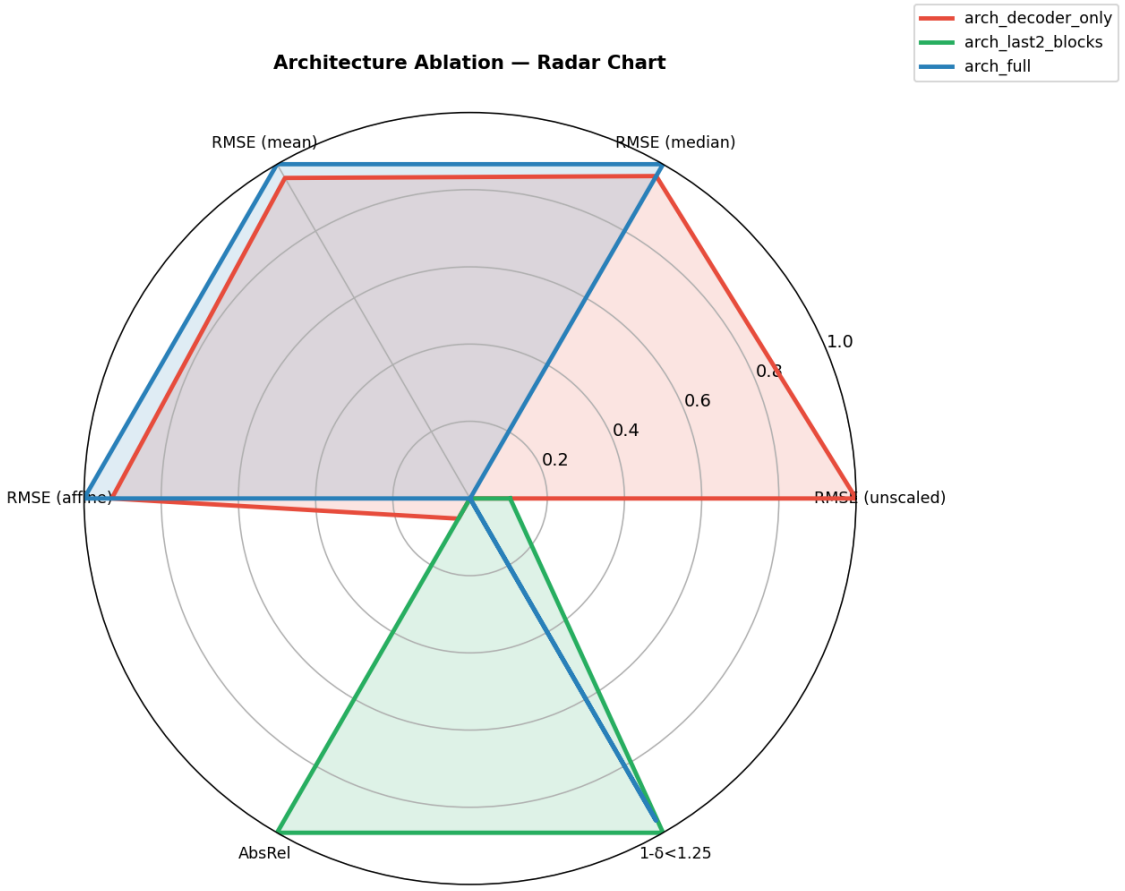


FIGURE 5.9: Radar Chart for Architecture ablation study for Supervised fine-tuning, normalized, lower area is better

Full fine-tuning produces the lowest RMSE of 2.5056 m, while decoder-only gives the highest δ_1 of 0.9690. The differences between these configurations are relatively small. The intermediate row should be interpreted as the partial decoder/head-unfreezing implementation described above, not as an experiment that selectively unfroze two encoder blocks. Decoder-only training was retained for the later experiments because it requires considerably less GPU memory and leaves the pretrained encoder unchanged. This was particularly important for training on the larger KITTI dataset using the available GPU resources.

5.6.3 Augmentation Ablation

Different augmentation strategies were also tested to determine their effect on metric depth estimation. The experiments included no augmentation, horizontal flipping, the base combination of flipping and colour jitter, and a full configuration that additionally uses random cropping and resizing.

TABLE 5.9: Augmentation ablation using MAE-Canonical loss, decoder-only training and AdamW with cosine warmup on the 46-frame test set.

Augmentation	**PW-RMSE (m)↓**	**AbsRel↓**	$\boldsymbol{\delta_1}$ **↑**
No augmentation	2.7515	0.0835	0.9508
Flip only	2.5507	0.0741	0.9667
Base (flip + colour jitter)	**2.4064**	0.0743	**0.9689**
Full (flip + jitter + crop)	3.0485	0.1230	0.9344

Base augmentation gives the lowest RMSE of 2.4064 m and the highest δ_1 of 0.9689, although flip-only gives a slightly lower AbsRel. In contrast, adding random cropping and resizing increases the RMSE to 3.0485 m, which is even worse than using no augmentation. This is likely related to the change in the effective field of view and camera geometry caused by cropping, which can introduce inconsistencies between the metric depth predictions and the corresponding LiDAR supervision. Therefore, the base augmentation strategy was retained for subsequent training.

5.6.4 Optimiser and Scheduler Ablation

The effect of different optimiser and learning-rate scheduler combinations was examined while keeping the loss function, architecture and augmentation unchanged.

TABLE 5.10: Optimiser and scheduler ablation using MAE-Canonical loss, decoder-only training and base augmentation on the 46-frame test set.

Optimiser / Scheduler	**PW-RMSE (m)↓**	**AbsRel↓**	**δ_1 ↑**
AdamW + StepLR	2.4344	0.0714	0.9713
Adam + cosine warmup	2.7228	0.0774	0.9650
SGD + cosine warmup	2.4591	**0.0620**	**0.9758**
AdamW + OneCycleLR	2.6625	0.0850	0.9634
AdamW + cosine warmup	2.6078	0.0757	0.9664

AdamW with StepLR achieves the lowest RMSE of 2.4344 m, whereas SGD with cosine warmup gives the best AbsRel and δ_1. Despite these differences, AdamW with cosine warmup was retained as the default for the later combined-loss and final training experiments. This choice was based on its more stable behaviour across the different loss functions, particularly those that showed convergence difficulties with SGD at the tested learning rates.

5.6.5 Combined Loss Ablation

After selecting the individual training components, different combinations of losses were evaluated to determine whether adding further constraints could improve the depth predictions. The recipes were based mainly on MAE-Canonical and SILog, with additional gradient, BerHu, MaskedDepth and L1 terms. Their compositions and full 652-frame evaluation results are presented in Table 5.11.

TABLE 5.11: Full-frame KITTI Eigen-Split evaluation of the combined supervised loss recipes (652 frames). RMSE is pixel-weighted, while AbsRel and δ_1 are averaged across frames.

Recipe	**Composition**	**PW-RMSE (m)↓**	**AbsRel↓**	**δ_1 ↑**
A1	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + 0.1\mathcal{L}_{\text{grad}}$	**2.2335**	0.0500	**0.9799**
A2	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}}$	2.2585	0.0503	0.9796
A3	A1 $+0.5\mathcal{L}_{\text{BerHu}}$	2.3298	0.0510	0.9786
A4	A1 $+0.5\mathcal{L}_{D}$	2.2578	0.0498	0.9795
A5	A1 $+1.0\mathcal{L}_{D}$	2.2843	**0.0497**	0.9794
B1	$0.5\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + \mathcal{L}_{\text{BerHu}} + 0.5\mathcal{L}_{L1} + 0.1\mathcal{L}_{\text{grad}}$	2.3732	0.0512	0.9782
B2	$\mathcal{L}_{\text{MAE}}+\mathcal{L}_{\text{SILog}}+\mathcal{L}_{\text{BerHu}}+0.5\mathcal{L}_{L1}+0.1\mathcal{L}_{\text{grad}}$	2.3112	0.0511	0.9787
C	$\mathcal{L}_{\text{MAE}}$ alone (SGD)	2.6193	0.0637	0.9647

Among these configurations, A1 gives the lowest PW-RMSE of 2.2335 m, while A5 achieves the lowest AbsRel of 0.0497. The addition of the gradient term in A1 provides an improvement over A2, which uses only MAE-Canonical and SILog. Adding BerHu in A3 or MaskedDepth in A4 and A5 does not produce a further reduction in RMSE. However, A4 and A5 give slightly lower AbsRel values than A1. The A4 entry in Table 5.11 corresponds to the 50-epoch standalone full-set A4 run described in the next subsection. A separate 100-epoch A4 run is reported there as well; the two results refer to distinct training runs and should not be treated as measurements from the same checkpoint.

The A2 entry in Table 5.11 also corresponds to a separate standalone full-set supervised run, with a 50-epoch training horizon. Its saved configuration specifies Recipe A2, decoder-only training, base augmentation, AdamW, a learning rate of 10^{-6}, weight decay of 10^{-4}, a batch size of 8 with four-step gradient accumulation

(effective batch size 32), one warmup epoch, mixed-precision training, gradient clipping at 1.0, and a 0.5 m depth floor. The target resolution was 378×1246, and checkpoints were saved every epoch. Early-stopping patience was 30 epochs, with 10% of the data reserved for validation using seed 42. The run started at epoch 1 and was not resumed from an earlier checkpoint. The final evaluation CSV contains 652 full-frame records and 46,205,813 valid LiDAR pixels. Aggregating the per-frame squared-error totals and valid-pixel counts gives a pixel-weighted RMSE of $\sqrt{235695525.0969/46205813} = 2.25854$ m, reported as 2.2585 m in Table 5.11. The frame-averaged AbsRel and δ_1 are 0.05025 and 0.97962, respectively, which are reported as 0.0503 and 0.9796. The periodic test log records its lowest validation RMSE at epoch 44 (2.012435 m); the test RMSE recorded for that epoch is 2.258608 m. At epoch 50, the periodic test RMSE is 2.263959 m. Thus, the 2.2585 m table entry is the full-frame CSV aggregation, consistent with the best epoch in the periodic log, and should not be described as the periodic test result at the final epoch. The small difference between 2.25854 m and 2.25861 m reflects the two saved evaluation summaries; both round to 2.259 m at three decimal places. This A2 run is distinct from both standalone A4 runs reported below.

5.6.6 Final Fine-Tuning

Recipe A4 was selected for the final full-set supervised fine-tuning experiments. It combines MAE-Canonical, SILog and gradient losses with an additional MaskedDepth term weighted by 0.5. Two standalone A4 runs are reported here: one with a 50-epoch training horizon and a separate run configured for 100 epochs. Their results are listed separately because they were saved and evaluated as distinct runs.

For the 50-epoch run, training used the complete available KITTI training set, with 10% of the samples reserved for validation using random seed 42. The configuration used decoder-only optimisation, base augmentation, AdamW with a learning rate of 10^{-6} and weight decay of 10^{-4}, cosine scheduling with one warmup epoch, and a batch size of 8 with four-step gradient accumulation (effective batch size 32). Mixed-precision training and a maximum gradient norm of 1.0 were enabled, and early stopping patience was set to 30 epochs. The saved configuration records a 50-epoch training horizon and a resume start at epoch 46 from an existing A4 checkpoint; the periodic metric log contains entries for epochs 1–50.

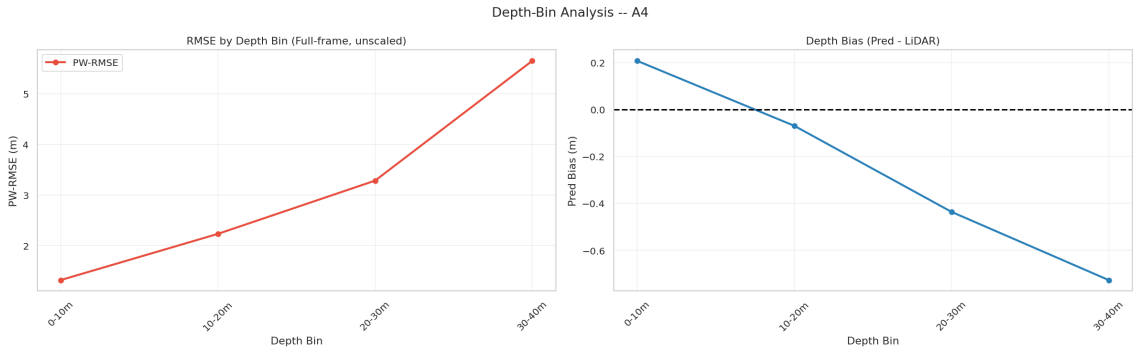


FIGURE 5.10: Depth Bin analysis of fine-tuned DA3-M on loss combination of A4 (50 epochs)

The 50-epoch model was evaluated on the complete 652-frame KITTI Eigen-Split test set. The saved full-frame results contain 46,205,813 valid LiDAR pixels. Summing the per-frame squared-error totals and dividing by the total number of valid pixels gives a pixel-weighted RMSE of 2.2578 m. The frame-averaged AbsRel and δ_1 are 0.0498 and 0.9795, respectively. These are the values reported for the 50-epoch A4 result in Table 5.11. The periodic test log records 2.2576 m at epoch 50; the final full-frame CSV aggregation gives 2.2578 m. The latter is used as the headline result because it is calculated directly from the saved per-frame squared-error totals and valid-pixel counts.

A separate A4 run was configured for up to 100 epochs. Its reported result is taken from the post-training evaluation of the checkpoint selected using validation PW-RMSE. This checkpoint achieved a pixel-weighted test RMSE of 2.2565 m, an AbsRel of 0.0494 and a δ_1 of 0.9798. This value should not be confused with the periodic test metric for the last training epoch: the reported 2.2565 m belongs to the validation-selected checkpoint, not necessarily the epoch-100 checkpoint. Thus, the 50-epoch and 100-epoch A4 results are 2.2578 m and 2.2565 m, respectively, under their corresponding saved evaluation procedures.

TABLE 5.12: Comparison of the two standalone full-set supervised A4 runs. The 50-epoch result is aggregated from the saved full-frame prediction CSV; the 100-epoch result is from the post-training evaluation of the validation-selected checkpoint.

Run	**Training horizon**	**PW-RMSE (m)**	**AbsRel**	$\boldsymbol{\delta_1}$
Standalone A4 run 1	50 epochs	2.2578	0.0498	0.9795
Standalone A4 run 2	100 epochs	2.2565	0.0494	0.9798

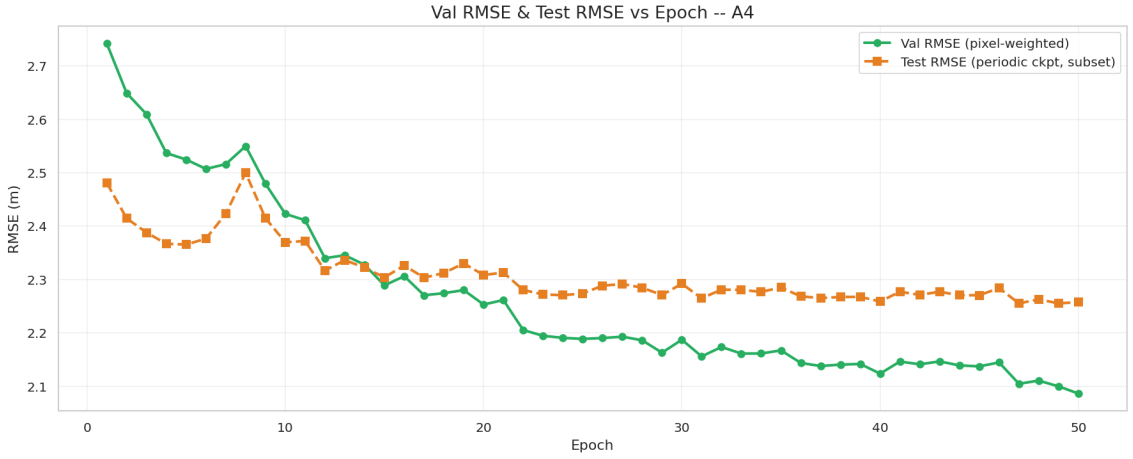


FIGURE 5.11: A4 Validation RMSE and Test RMSE per epoch (max epochs 50)

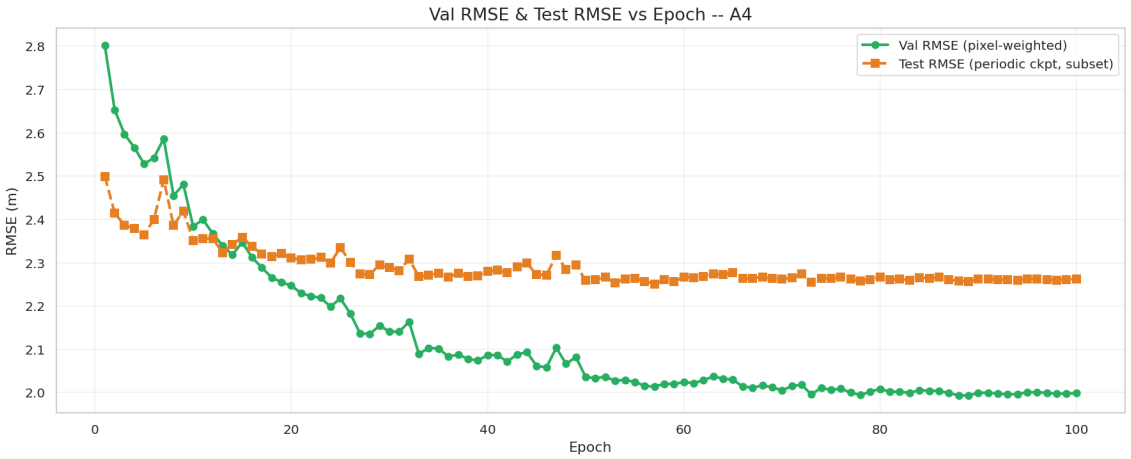


FIGURE 5.12: A4 Validation RMSE and Test RMSE per epoch (max epochs 100)

The difference between these two PW-RMSE values is 0.0013 m. Since they are results from separately recorded runs and checkpoint-selection procedures, this small difference should not be interpreted as a controlled measurement of the effect of extending training from 50 to 100 epochs. The category-wise evaluation also shows that errors remain uneven across object types, with stationary objects and pedestrians generally having higher errors than road regions and vehicle components.

Overall, the supervised experiments show that the choice of loss function, augmentation and optimiser affects the resulting depth predictions. MAE-Canonical performs well as an individual loss, while the base augmentation strategy gives the best result among the tested augmentation configurations. Recipe A4 was retained for the full-set supervised experiments, and the two training horizons are reported separately to keep their checkpoints and evaluation results distinct.

5.7 Stage 8: Self-Supervised Fine-Tuning

5.7.1 SSL Loss Function Ablation

Before selecting the final training configuration, different self-supervised loss components were evaluated to understand their individual contributions. The experiments covered photometric reconstruction, geometric consistency, rigid-flow consistency, temporal depth consistency and smoothness regularisation. The unscaled results of the main loss-registry experiments are presented in Table 5.13.

TABLE 5.13: Selected self-supervised loss-registry ablation results. RMSE is pixel-weighted true RMSE; AbsRel and δ_1 are reported as provided in the evaluation matrix.

Loss Configuration	**PW-RMSE (m)↓**	**AbsRel↓**	**δ_1 ↑**
Photo L1	2.8127	0.0824	0.9527
Photo SSIM	2.8470	0.0962	0.9395
Photo SSIM + L1	2.8415	0.0917	0.9430
Minimum Reprojection Photo	3.3454	0.1463	0.7754
Multi-Scale Photo	3.4596	0.1622	0.6677
Geometric Consistency	2.9931	0.0993	0.9345
Feature Metric	2.8151	0.0816	0.9539
Rigid-Flow Consistency	2.8179	0.0758	0.9577
Temporal Warp Consistency	3.0317	0.1025	0.9300
Combined Monodepth2	3.6405	0.1714	0.6148
Edge-Aware Smoothness	2.9541	0.0992	0.9407
Naive Temporal Delta	3.7156	0.1390	0.8436

Among the individual loss configurations, Photo L1 gives the lowest RMSE of 2.8127 m, while Rigid-Flow Consistency produces the lowest AbsRel and highest δ_1 among the

listed configurations. Minimum reprojection and multi-scale photometric losses show considerably higher errors in this experiment. The naive temporal-difference loss also performs poorly, which is consistent with the fact that it does not explicitly account for camera motion when comparing consecutive depth maps.

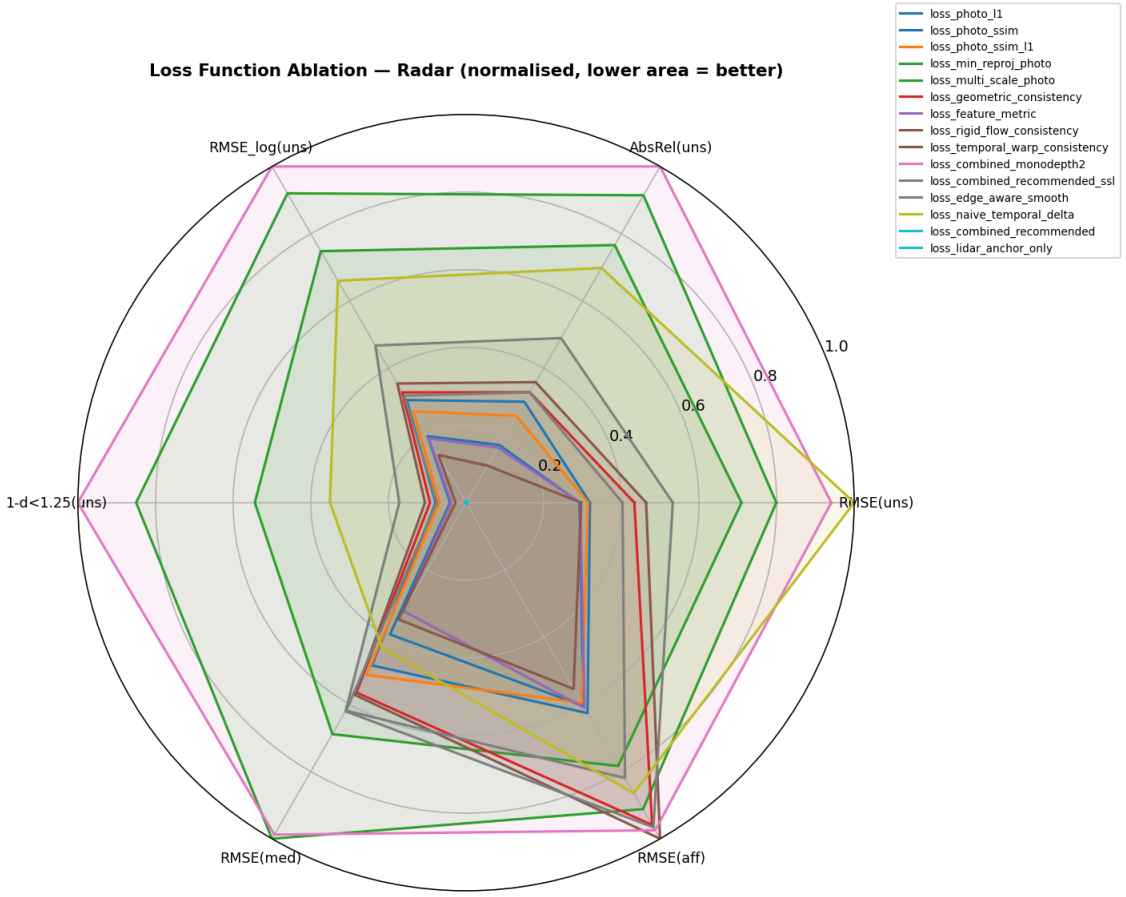


FIGURE 5.13: Radar Chart for all SSL loss functions from ablation study, normalized, lower area is better

These results show that simply adding more constraints does not necessarily improve depth estimation. The choice of loss formulation and the way temporal relationships are handled have a noticeable effect on the final predictions.

5.7.2 Stationary-Object Loss Ablation

A separate set of experiments examined the use of stationary objects for enforcing additional geometric consistency. These configurations use the object masks and geometric information obtained from the earlier stages of the pipeline. The unscaled results are shown in Table 5.14.

TABLE 5.14: Comparison of stationary-object-based self-supervised loss configurations. RMSE represents pixel-weighted true RMSE.

Configuration	**PW-RMSE (m)↓**	**AbsRel↓**	$\boldsymbol{\delta_1}$ **↑**
Stationary Depth Motion	3.6334	0.1348	0.8707
Stationary Uniform Motion	3.5815	0.1309	0.8818
Full 3D Ego Motion	3.7780	0.1419	0.8500
Mask Chamfer	4.2028	0.1558	0.7946
Combined Stationary	4.0129	0.1488	0.8249

Stationary Uniform Motion gives the lowest RMSE of 3.5815 m among these five configurations, while Mask Chamfer produces the highest error. Although these experiments explore the use of object-level geometric information, the results do not show a consistent improvement over the other self-supervised formulations. Therefore, these configurations were not selected for the final fine-tuning experiment.

5.7.3 Final Combined Recipe Comparison

Following the loss ablation experiments, five saved configurations were compared using the same 652-frame KITTI Eigen-Split test set. The experiments cover two different pose sources: relative camera poses obtained from KITTI OXTS measurements and

poses predicted by the PoseNet. None of these five configurations uses LiDAR depth as a training target.

The main loss compositions are summarised below:

TABLE 5.15: Loss compositions used in the final self-supervised recipe comparison.

Recipe	**Pose Source**	**Loss Composition**
A1	OXTS	$1.0\,\mathcal{L}_{\text{GT-photo}}$
A3	OXTS	$1.0\,\mathcal{L}_{\text{GT-photo}} + 10^{-3}\mathcal{L}_{\text{smooth}} + 0.1\mathcal{L}_{\text{GT-temporal}}$
B1	OXTS	$1.0\,\mathcal{L}_{\text{GT-rigid}-\text{flow}}$
C1	PoseNet	$1.0\,\mathcal{L}_{\text{rigid}-\text{flow}}$
C3	PoseNet	$1.0\,\mathcal{L}_{\text{photo-L1}} + 0.1\mathcal{L}_{\text{flow-smooth}}$

The best validation epoch and corresponding test results for each saved configuration are presented in Table 5.16.

TABLE 5.16: Final self-supervised fine-tuning results on the 652-frame KITTI Eigen-Split test set. Checkpoints are selected using validation RMSE. Test RMSE is pixel-weighted.

Recipe	**Best Epoch**	**Epochs Run**	**Best Val. RMSE (m)**	**Test PW-RMSE (m)**
A1	33	48	2.5421	2.6011
A3	**27**	**42**	**2.5035**	**2.5266**
B1	1	11	2.8469	2.8117
C1	1	11	2.9888	2.9990
C3	1	11	11.3948	11.6096

A3 gives the best result among the five configurations, with a test PW-RMSE of 2.5266 m. Its best validation RMSE is 2.5035 m at epoch 27, compared with 2.5421 m for A1 at epoch 33. A3 also achieves a slightly lower test AbsRel than A1. However, A1 gives a marginally higher δ_1, so the improvement is not identical across all metrics.

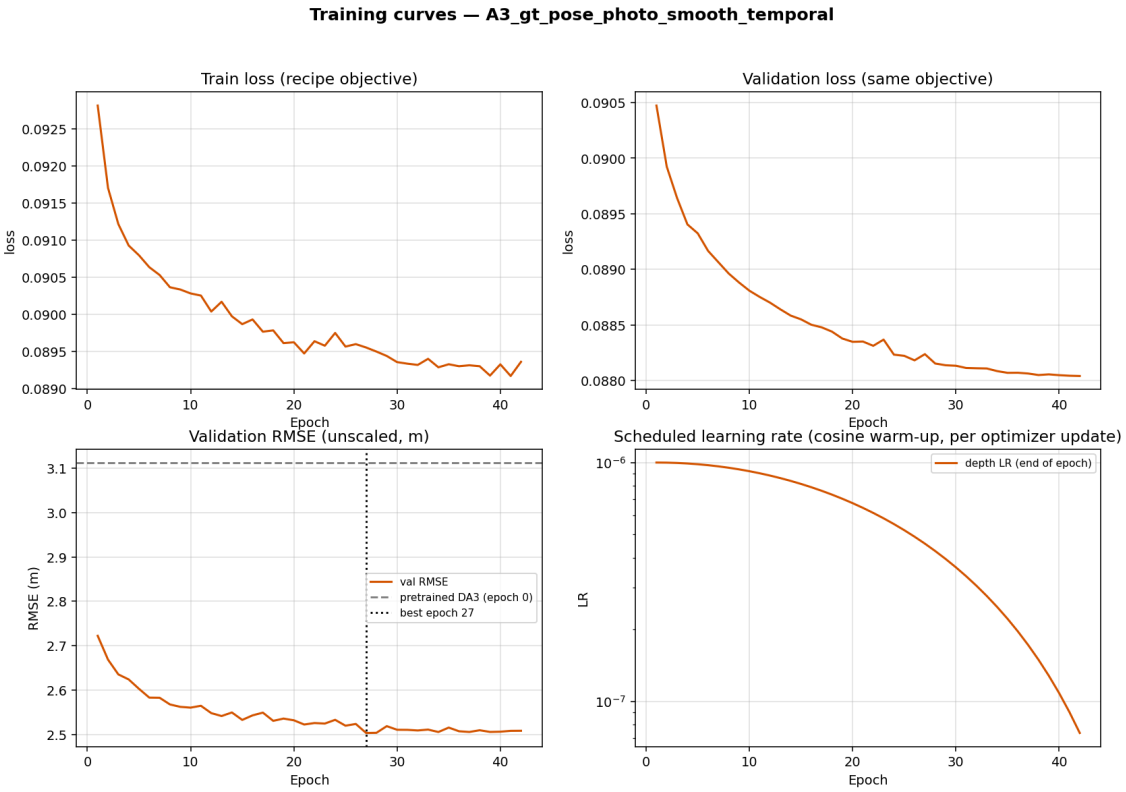


FIGURE 5.14: Various training curves of A3 loss function for self supervised fine-tuning

The training curves show that A1 and A3 converge steadily, with their validation RMSE remaining relatively stable towards the end of training. In contrast, B1, C1 and C3 achieve their best validation results at the first epoch, after which their validation RMSE increases considerably. This behaviour is particularly noticeable for C1 and C3, both of which use PoseNet-predicted camera motion. B1 also shows instability despite using OXTS poses, indicating that the choice of pose source alone does not guarantee stable optimisation.

For A3, the recorded pretrained validation baseline is 3.1125 m, which decreases to 2.5035 m after fine-tuning. Its final test result of 2.5266 m is also lower than that of A1. These results make A3 the strongest of the completed self-supervised experiments.

5.7.4 Detailed Unscaled Metric Comparison

For a closer comparison of the five saved configurations, the remaining evaluation metrics are also reported in Table 5.17. In addition to PW-RMSE, these include MAE, AbsRel, squared relative error, logarithmic error and the three threshold accuracies.

TABLE 5.17: Detailed unscaled evaluation results for the final self-supervised configurations on the KITTI Eigen-Split test set.

Recipe	PW-RMSE	MAE	AbsRel	SqRel	$\text{RMSE}_{\log}$	log10	δ_1	δ_2	δ_3	Mean Error
A1	2.6011	1.1228	0.0584	0.2375	0.0875	0.0249	0.9746	0.9957	0.9984	−0.0739
A3	2.5266	1.0968	0.0575	0.2224	0.0871	0.0248	0.9742	0.9956	0.9984	+0.1116
B1	2.8117	1.4413	0.0953	0.3218	0.1212	0.0390	0.9356	0.9936	0.9980	−0.5542
C1	2.9990	1.4145	0.0787	0.3442	0.1103	0.0327	0.9492	0.9923	0.9976	−0.4784
C3	11.6096	8.5275	0.5041	4.7387	0.7260	0.3096	0.0026	0.0237	0.4103	+8.5241

PW-RMSE is pixel-weighted. Mean Error is calculated as LiDAR depth minus predicted depth; positive values indicate underestimation and negative values indicate overestimation.

A3 gives the lowest PW-RMSE and AbsRel among the five configurations, whereas A1 achieves the highest δ_1. B1 and C1 show larger errors than the two photometric OXTS-based configurations. C3 performs particularly poorly, with an RMSE of 11.6096 m and a δ_1 of only 0.0026. Its large positive mean error of 8.5241 m also indicates substantial depth underestimation under the adopted sign convention.

5.7.5 Scale-Alignment Comparison

The final predictions were also evaluated using median, mean and affine scale alignment to examine the effect of correcting global depth-scale differences. These results are presented in Table 5.18. The alignment parameters are calculated using LiDAR

ground truth and are included as oracle references rather than practical inference-time corrections.

Table 5.18: Comparison of PW-RMSE across different scale-alignment methods for the final self-supervised configurations.

Recipe	**Unscaled (m)**	**Median (m)**	**Mean (m)**	**Affine (m)**
A1	2.6011	2.5968	2.4166	2.2654
A3	2.5266	2.5090	2.3652	2.2592
B1	2.8117	2.6376	2.5125	2.4383
C1	2.9990	2.7953	2.6181	2.4873
C3	11.6096	4.1723	3.4317	2.6310

All five configurations show a reduction in RMSE after scale alignment, with affine alignment producing the lowest error in each case. A3 achieves the lowest affine-aligned RMSE of 2.2592 m, closely followed by A1 at 2.2654 m. C3 also shows a considerable reduction after alignment, although its error remains higher than the other configurations. Since these corrections use LiDAR ground truth, they should only be treated as diagnostic references and not as the actual self-supervised inference results.

Overall, A3 provides the most consistent performance among the completed self-supervised experiments. Its combination of photometric reprojection, edge-aware smoothness and temporal depth-warp consistency gives better results than using photometric reprojection alone in A1. However, these three terms are added together in A3, so their individual contributions cannot be isolated from this comparison.

It is also important to note that A3 uses relative camera poses obtained from KITTI OXTS measurements. Thus, the experiment is self-supervised with respect to depth, but it still relies on externally supplied camera-motion information during training.

LiDAR depth is not used in the optimisation losses, although it is used for validation, checkpoint selection and final evaluation. The OXTS measurements are not required during depth inference.

The checkpoint carried forward to Stage 9 is the best-validation A3 checkpoint from epoch 27 (validation RMSE 2.5035 m), rather than the weights from the final recorded A3 epoch. The A3 run continued for 42 epochs out of the planned 50, and its final validation RMSE was 2.5084 m. The selected checkpoint is the one recorded as the A3 best model in the saved experiment summary. It was then supplied as the initialization checkpoint for the following supervised A4 run.

5.8 Stage 9: Combined Fine-Tuning

The final experiment was carried out as a sequential two-stage training run. First, the DA3-M model was fine-tuned using the A3 self-supervised objective. The best-validation checkpoint from that A3 run, selected at epoch 27, was then used to initialise the A4 supervised fine-tuning stage. Thus, the A4 run did not start again from the original pretrained DA3-M weights. This sequential A3-initialised A4 run is distinct from the standalone A4 experiments reported earlier in this chapter, even though all three use the A4 supervised recipe.

This hand-off is also recorded in the A4 training configuration. Its `init_checkpoint` field points to `best_model_SSFT_A3.pt`, while `start_epoch` is set to 1 and `resumed_from` is null. In other words, A4 began a new supervised training schedule at its own epoch 1, using the A3 fine-tuned weights as the initial model parameters; it was not a resume of an earlier A4 run.

The A3 stage used OXTS relative camera poses and the combination of photometric reprojection, edge-aware smoothness and temporal warp consistency. It used a decoder-only model with base augmentation, AdamW, cosine warmup and a learning rate of 10^{-6}. Its per-step batch size was 4 with eight gradient-accumulation steps, giving an effective batch size of 32. The run was configured for up to 50 epochs but stopped after 42; the checkpoint from epoch 27 was selected by validation RMSE (2.5035 m). Its unscaled pixel-weighted test RMSE was 2.5266 m, with AbsRel 0.0575 and $\delta_1 = 0.9742$. The subsequent A4 stage used projected LiDAR depth and Recipe A4, which combines MAE-Canonical, SILog, gradient and MaskedDepth losses:

$$\mathcal{L}_{A3}(SSL) = 1.0\,\mathcal{L}_{\text{photo}} + 10^{-3}\,\mathcal{L}_{\text{smooth}} + 0.1\,\mathcal{L}_{\text{temporal}}. \tag{5.1}$$

$$\mathcal{L}_{A4} = 1.0\,\mathcal{L}_{\text{MAECanonical}} + 1.0\,\mathcal{L}_{\text{SILog}} + 0.1\,\mathcal{L}_{\text{grad}} + 0.5\,\mathcal{L}_{\text{masked-depth}}. \tag{5.2}$$

The A4 stage used decoder-only training, base augmentation (horizontal flipping and mild colour jitter), AdamW and a cosine-warmup learning-rate schedule. The learning rate was 10^{-6}, with a batch size of 8 and four gradient-accumulation steps, giving an effective batch size of 32. Training was configured for up to 50 epochs, with a 10% validation split and an early-stopping patience of 20 epochs. The supervised implementation also used the 0.5 m depth floor for MAE-Canonical. Unlike the A3 stage, A4 directly used projected LiDAR depth as supervision.

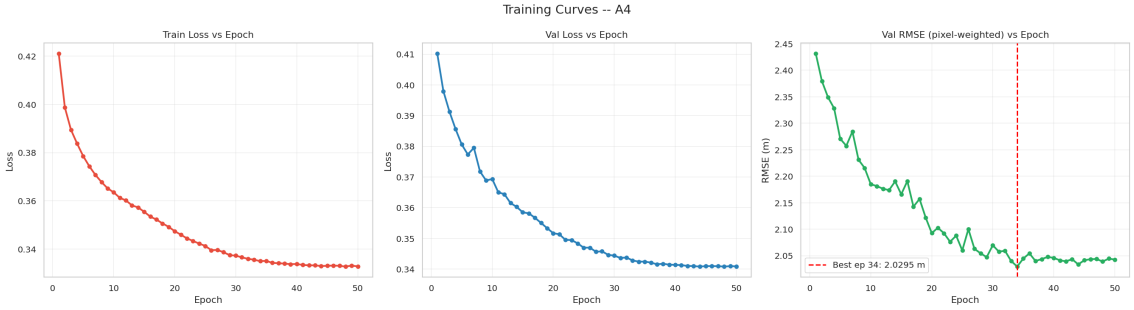


FIGURE 5.15: Training Curve for A4 Supervised loss combination with the starting checkpoint taken from the best model of A3 Self-Supervised loss combination for Sequentially Combined Fine-Tuning

The A4 training history identifies epoch 34 as the best-validation checkpoint, with a validation RMSE of 2.0295 m. The periodic test log records a test RMSE of 2.2347 m at that epoch. After training, the selected checkpoint was evaluated over all 652 KITTI Eigen-Split test frames. Aggregating the saved full-frame squared-error totals over the 46,205,813 valid LiDAR pixels gives a pixel-weighted RMSE of 2.2346 m; the frame-averaged AbsRel and δ_1 are 0.0494 and 0.9804, respectively.

At the last recorded epoch (epoch 50), the validation RMSE was 2.0427 m and the periodic test RMSE was 2.2530 m. The headline value of 2.2346 m therefore refers to the full-frame evaluation of the validation-selected epoch 34 checkpoint, not to the final epoch 50 checkpoint. The periodic log and the full-frame CSV aggregation differ slightly (2.2347 m versus 2.2346 m); the latter is reported in the overall test table because it is recomputed from the saved per-frame squared-error totals and valid-pixel counts.

TABLE 5.19: Training and evaluation performance of the sequential self-supervised and supervised fine-tuning experiment using Recipe A4 on KITTI Eigen-Split.

Checkpoint	**Epoch**	**Validation RMSE (m)**	**Test RMSE (m)**
Best validation checkpoint	34	2.030	2.235
Last recorded epoch	50	2.043	2.253

5.8.1 Overall Evaluation Results

The selected checkpoint was evaluated on all 652 frames of the KITTI Eigen-Split test set. The evaluation used the complete valid region within the Eigen crop, containing 46,205,813 valid LiDAR pixels. The final results are presented in Table 5.20.

TABLE 5.20: Full-frame evaluation of the combined fine-tuned DA3-M model on KITTI Eigen-Split.

Evaluation	**PW-RMSE (m)↓**	**AbsRel↓**	**δ_1 ↑**	**Valid Pixels**
Full-frame (Eigen crop)	2.2346	0.0494	0.9804	46,205,813

Compared with the supervised A2 result of 2.2585 m, the combined model reduces the test RMSE by approximately 1.1%. It also improves upon the zero-shot DA3-M result of 2.9379 m by approximately 23.9%. These comparisons indicate an improvement in the recorded depth estimation results after combining self-supervised and supervised fine-tuning. However, since the combined experiment starts from the A3 checkpoint and uses a different supervised loss configuration, the improvement cannot be attributed to self-supervised pretraining alone.

5.8.2 Category-Wise Evaluation

To examine whether the improvement is consistent across different types of objects, the final predictions were also evaluated using the SAM3 semantic masks. Table 5.21 presents the results for the eight super-categories.

TABLE 5.21: Category-wise evaluation of the combined fine-tuned DA3-M model on KITTI Eigen-Split.

Category	**#Masks**	**#Pixels**	**PW-RMSE (m)↓**	**AbsRel↓**	**δ_1 ↑**
animal	2	560	0.3289	0.0289	0.9868
cyclist	204	78,923	1.1940	0.0523	0.9812
pedestrian	482	128,108	1.8192	0.0783	0.9433
road	1,070	20,810,749	0.7287	0.0344	0.9928
stationary_object	6,481	9,572,429	2.2874	0.0969	0.9046
vehicle	1,996	2,430,077	1.1792	0.0555	0.9682
vehicle_lights	642	75,150	0.4017	0.0309	0.9993
vehicle_wheels	1,819	378,583	0.5793	0.0348	0.9956

The category-wise results show that stationary objects have the highest RMSE of 2.2874 m, followed by pedestrians at 1.8192 m. On the other hand, road regions have an RMSE of 0.7287 m, while vehicle lights and wheels achieve 0.4017 m and 0.5793 m, respectively. The relatively higher errors for stationary objects may be related to the wide variation in their appearance, size and distance from the camera. It is also worth noting that the animal category contains only two masks, so its comparatively low error should be interpreted with caution.

5.8.3 Depth-Range Analysis

The effect of distance on the final depth predictions was also examined by dividing the valid depth measurements into four ranges: 0–10 m, 10–20 m, 20–30 m and 30–40 m. Figure 5.16 presents the corresponding PW-RMSE and signed depth bias.

The RMSE increases with distance, with the 30–40 m range showing the largest error. This indicates that depth estimation becomes more difficult for distant regions, even after the combined fine-tuning process. The signed-bias plot also shows a change in error direction with increasing distance. The nearest range has a positive bias,

indicating depth overestimation, whereas the remaining ranges show negative bias, indicating underestimation. The magnitude of this negative bias increases towards the farther ranges.

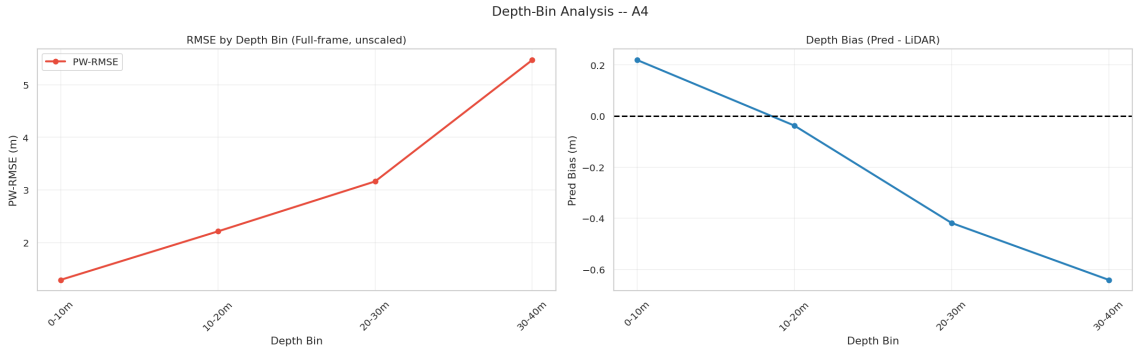


FIGURE 5.16: Depth-range analysis of the combined fine-tuned DA3-M model. The left plot shows pixel-weighted RMSE across depth intervals, while the right plot shows signed bias calculated as predicted depth minus LiDAR depth.

These observations show that the combined model does not completely eliminate depth-dependent errors. Although the overall evaluation metrics improve, errors at larger distances remain relatively high, and the direction of the bias also varies across depth ranges.

Overall, the sequential A3-to-A4 experiment achieved a pixel-weighted RMSE of 2.2346 m. This is lower than the standalone A2 and A4 results, but it is marginally higher than the 2.2335 m result obtained by Recipe A1 in the separate supervised recipe comparison. The difference is small, and these runs do not form a controlled comparison in which checkpoint initialisation is the only changed factor. The sequential result should therefore not be described as the lowest recorded result across all fine-tuning configurations, nor should its performance difference be attributed to self-supervised initialisation alone. The remaining category-wise and depth-dependent errors indicate that further work is still required to obtain more consistent metric depth predictions across different scene regions.

5.9 Summary of Key Results

1. The five-model zero-shot benchmark recorded an unscaled RMSE of 2.938 m for DA3-M and 2.994 m for UniDepth V2 in the whole-frame comparison. Metric rankings depend on the evaluation aggregation, so the thesis distinguishes whole-frame and mask-level results rather than treating them as interchangeable.

2. Vehicle-anchored correction modestly improved the final full-test result: the average per-frame RMSE decreased from 3.7131 to 3.5278 m with scale-only correction and to 3.4897 m with scale-and-shift correction. Both remain far from the oracle affine result, so the method is not a replacement for learned adaptation or LiDAR-based alignment.

3. A separate full-set A2 run used a 50-epoch training configuration. The periodic log records its lowest validation RMSE at epoch 44 (2.0124 m), with a periodic test RMSE of 2.2586 m. Aggregation of the saved full-frame test CSV gives a pixel-weighted RMSE of 2.2585 m, with AbsRel 0.0503 and $\delta_1 = 0.9796$. The periodic test RMSE at epoch 50 is 2.2640 m; therefore, the 2.2585 m headline value should not be attributed to the final epoch.

4. The two standalone full-set A4 runs gave PW-RMSE values of 2.2578 m (50-epoch run) and 2.2565 m (100-epoch run, validation-selected checkpoint). These are distinct runs and are reported separately.

5. Self-supervised fine-tuning with OXTS poses and photometric, smoothness, and temporal-warp losses (Recipe A3) achieved 2.527 m unscaled test RMSE. Other tested recipes were less stable, particularly those relying on predicted-pose rigid-flow consistency.

6. The sequential A3-to-A4 experiment used the best-validation A3 checkpoint (epoch 27) to initialise a new 50-epoch supervised A4 run. The A4 validation-selected checkpoint at epoch 34 achieved a full-frame pixel-weighted test RMSE of 2.2346 m (AbsRel 0.0494; $\delta_1 = 0.9804$). This is lower than the standalone A2 and A4 results, but marginally higher than the 2.2335 m result from Recipe A1 in the separate supervised recipe comparison. These runs do not isolate the contribution of self-supervised initialisation because their training histories and configurations are not fully matched.

7. The augmentation ablation continues to disfavour the tested random crop/resize strategy (RMSE 3.049 m versus 2.406 m for base augmentation) for this metric-depth setup.

Chapter 6

Conclusions and Future Work

6.1 Summary of Work

This thesis investigated monocular metric depth estimation for dashcam imagery through a pipeline consisting of nine stages. The work began with instance segmentation and the evaluation of existing depth models, and then moved towards geometric correction, supervised fine-tuning, self-supervised learning, and a final combination of the two fine-tuning approaches. The main work carried out at each stage is summarised below.

Stage 1 – SAM3 instance segmentation. A text-prompted segmentation pipeline was developed using SAM3. It generated 12,808 valid mask instances across eight semantic super-categories from the 652 KITTI Eigen-Split test frames. Getting consistent masks required several rounds of prompt testing, particularly for vehicle components and objects appearing under different lighting conditions. Once generated, the masks were stored offline and reused in the subsequent evaluation and correction stages.

Stage 2 – Five-model benchmark. Five monocular metric depth estimation models were compared in their zero-shot configurations: DA2, DA3-M, DA3-G, DepthPro, and UniDepth V2 (UD2). The comparison included unscaled predictions as well as median, mean, and affine alignment using LiDAR ground truth as an oracle reference. In the unscaled whole-frame evaluation, DA3-M obtained the lowest RMSE of 2.9379 m, whereas UD2 achieved the lowest AbsRel of 0.0761 and the highest δ_1 of 0.9535. A separate mask-level evaluation was also carried out for DA3-M using the SAM3 instances. In that evaluation, stationary objects and pedestrians had the highest category-wise RMSE values.

Stage 3 – DA3-M error analysis. The signed difference between LiDAR depth and predicted depth was examined to determine whether the errors were mainly due to underestimation or overestimation. For the 80 m clipping setting, 81.54% of the valid LiDAR pixels showed underestimation, and the mean signed difference was +1.0357 m. Here, a positive value means that the predicted depth is smaller than the LiDAR depth. The same tendency remained when the evaluation was restricted to 40 m, although the mean difference decreased to +0.8452 m. This suggests that the underestimation was not limited to distant points.

Stage 4 – Optional reflection removal. An optional reflection-removal workflow based on DC-Gaussian and COLMAP was included for sequences affected by windshield reflections. It was not required for the KITTI images used in this thesis and was not part of the reported experiments. No quantitative improvement is therefore claimed for this stage.

Stage 5 – Vehicle-anchored correction. A training-free scale and shift correction was developed using approximate passenger-car dimensions and vehicle masks. The method used 1,980 vehicle anchors from the 652-frame test set. The scale-and-shift fit reduced the average per-frame RMSE from 3.7131 to 3.4897 m, while the scale-only

correction reached 3.5278 m. These values are average per-frame RMSE, rather than pixel-weighted RMSE. For reference, oracle affine alignment using LiDAR ground truth reached 1.0682 m, but this result is not available during normal inference. The vehicle-based correction therefore gives a measurable but limited improvement.

Stage 6 – Additional dataset preparation. Configurable LiDAR projection templates were added for nuScenes, TIAND, and IDD. The loaders handle point clouds stored in `.bin`, `.pcd`, and `.npy` formats, respectively. These templates provide a starting point for extending the pipeline, but the three datasets were not used to produce the quantitative results in this thesis.

Stage 7 – Supervised fine-tuning. The supervised experiments examined 13 loss functions, three model architecture settings, four augmentation strategies, and five optimiser–scheduler combinations. MAE-Canonical gave the best result among the individual losses, with a pixel-weighted RMSE of 2.4997 m on the smaller 46-frame test subset. The augmentation comparison also showed a noticeable difference: the base setting of horizontal flipping and colour jitter achieved an RMSE of 2.4064 m, while adding random cropping and resizing increased it to 3.0485 m. In the full 652-frame recipe comparison, A1 recorded a pixel-weighted RMSE of 2.2335 m, A2 recorded 2.2585 m, and A4 recorded 2.2578 m. Separate full-set runs provide additional results: the standalone A2 run achieved 2.2585 m (AbsRel 0.0503; $\delta_1 = 0.9796$), while standalone A4 runs achieved 2.2578 m after a 50-epoch configured run (AbsRel 0.0498; $\delta_1 = 0.9795$) and 2.2565 m for the validation-selected checkpoint from a separate 100-epoch run (AbsRel 0.0494; $\delta_1 = 0.9798$). The A2 headline value is aggregated from the saved full-frame test results; its periodic test RMSE at epoch 50 was 2.2640 m. Similarly, the 100-epoch A4 headline value refers to the validation-selected checkpoint, not necessarily the final-epoch checkpoint. These runs are reported separately because they have different training histories and checkpoint-selection

records. The final full-set configurations used a learning rate of 10^{-6} and an effective batch size of 32.

Stage 8 – Self-supervised fine-tuning. A photometric reprojection framework based on the Monodepth2 formulation was implemented. It includes a 6-DoF ResNet-18 PoseNet, differentiable inverse warping, auto-masking, and edge-aware smoothness regularisation. The experiments also used relative camera poses derived from KITTI OXTS as an alternative to PoseNet predictions. The selected A3 configuration combined OXTS-based photometric reprojection, edge-aware smoothness, and temporal depth-warp consistency, with weights of 1.0, 0.001, and 0.1 for the corresponding terms. The run was configured for up to 50 epochs and recorded 42 epochs; the best-validation checkpoint was selected at epoch 27 (validation RMSE 2.5035 m). Its unscaled test result was a pixel-weighted RMSE of 2.5266 m, with AbsRel 0.0575 and $\delta_1 = 0.9742$. LiDAR depth was not used as a training target in this stage, but it was used for validation, checkpoint selection, and final evaluation.

Stage 9 – Sequential combined fine-tuning. The best-validation A3 self-supervised checkpoint from epoch 27 was used to initialise a new supervised A4 training run. The A4 stage began its own epoch schedule at epoch 1; it was not a continuation of an earlier A4 run. It used projected KITTI LiDAR depth and the A4 loss, which combines MAE-Canonical, SILog, gradient, and MaskedDepth terms. The best A4 validation checkpoint occurred at epoch 34, with a validation RMSE of 2.0295 m. Evaluation of this checkpoint on all 652 test frames gave a pixel-weighted RMSE of 2.2346 m, an AbsRel of 0.0494, and a δ_1 value of 0.9804. The periodic test log reports 2.2347 m at epoch 34, while the full-frame CSV aggregation gives 2.2346 m; the latter is used as the headline result. At epoch 50, the periodic test RMSE was 2.2530 m. This sequential result is close to, but slightly higher than, the 2.2335 m result for A1 in the separate full-frame recipe comparison. It should not

be interpreted as a controlled estimate of the effect of SSL initialisation, since the sequential and standalone runs do not constitute a matched experiment in which initialisation is the only changed factor.

6.2 Key Findings

The experiments lead to several observations about the use of monocular foundation models for metric depth estimation on dashcam imagery.

1. **DA3-M was a practical choice for the subsequent fine-tuning experiments.** It achieved the lowest unscaled whole-frame RMSE among the five benchmarked models, at 2.9379 m. UD2 performed better in AbsRel and δ_1, so DA3-M was not the best model across every metric. Its selection was also influenced by the availability of an open implementation and a decoder that could be fine-tuned within the memory limits of the available GPU.

2. **The zero-shot DA3-M predictions show a clear underestimation tendency.** Under the sign convention $D_{\text{LiDAR}} - D_{\text{pred}}$, 81.54% of the valid pixels showed underestimation, with a mean signed difference of +1.0357 m for the 80 m setting. The mean remained positive when the evaluation was limited to 40 m. This indicates a directional error pattern in the zero-shot predictions rather than an error distribution that is evenly split between overestimation and underestimation.

3. **Vehicle-size priors provide only a modest correction.** The vehicle-anchored scale-and-shift method reduced average per-frame RMSE from 3.7131 to 3.4897 m, while scale-only correction reduced it to 3.5278 m. The much lower oracle affine result of 1.0682 m requires LiDAR ground truth and is included

only as a reference. The remaining difference suggests that approximate vehicle dimensions and a single global correction are not sufficient to account for all the depth variations in a scene.

4. **MAE-Canonical was useful for sparse-LiDAR fine-tuning.** It gave the lowest RMSE among the individual loss functions tested in the smaller ablation, at 2.4997 m on the 46-frame held-out set. The implementation also clamps predicted depth to a minimum of 0.5 m before inverse-depth normalisation. This avoids excessively large inverse-depth values from dominating the loss near zero depth.

5. **The tested random crop-and-resize augmentation reduced metric-depth accuracy.** The base augmentation setting (horizontal flip and colour jitter) achieved an RMSE of 2.4064 m, compared with 3.0485 m when random cropping and resizing were added. The likely reason is that cropping changes the effective image geometry and field of view. The result supports avoiding this augmentation in the current setup unless the camera intrinsics and corresponding depth geometry are updated consistently.

6. **Decoder-only fine-tuning was mainly useful because of the available hardware constraints.** Freezing the encoder reduced the number of trainable parameters from 334.2 M to approximately 29.8 M, a reduction of about 91%. In the architecture ablation, full fine-tuning achieved a lower test RMSE (2.5056 m versus 2.5714 m) and AbsRel (0.07131 versus 0.07179), while decoder-only achieved a slightly higher δ_1 (0.96900 versus 0.96616). The decoder-only setting was therefore not the best on every metric; it was retained for the larger experiments because it was more practical within the available GPU memory and left the pretrained encoder unchanged.

7. **AdamW with cosine warmup was retained as the default for later experiments, although it was not the best optimiser on every ablation metric.** AdamW with StepLR produced the lowest RMSE in the optimiser comparison, while SGD with cosine warmup gave the best AbsRel and δ_1. AdamW with cosine warmup was nevertheless selected for the later combined-loss and final runs because of its observed training behaviour across the tested loss configurations.

8. **The supervised and sequential results depend on the recipe and checkpoint used.** The full-frame recipe comparison recorded 2.2335 m for A1 and 2.2585 m for A2. Separate standalone A4 runs recorded 2.2578 m for the 50-epoch run and 2.2565 m for the validation-selected checkpoint from the 100-epoch run. The sequential A3-to-A4 run recorded 2.2346 m. The sequential result is lower than the standalone A2 and A4 results, but is slightly higher than the A1 recipe-comparison result. These values come from distinct runs and checkpoint-selection procedures; they do not form a controlled comparison of initialisation strategies and should not be used to attribute the difference to self-supervised pretraining alone.

6.3 Research Contributions

The main contributions of this work are summarised as follows:

- A modular nine-stage pipeline was implemented, covering instance segmentation, monocular metric depth benchmarking, error analysis, geometric correction, and supervised, self-supervised, and sequential fine-tuning.

- A text-prompted SAM3 segmentation workflow was developed for eight semantic super-categories. The resulting masks were used for category-wise depth evaluation and as geometric inputs to the vehicle-anchor correction stage.

- DA3-M was analysed at both whole-frame and object-category levels on the KITTI Eigen-Split test set. The signed-difference analysis helped identify the extent and direction of its zero-shot depth errors.

- A training-free vehicle-anchor correction method was implemented using approximate passenger-car dimensions, orientation-aware measurements, and a RANSAC–WLS scale/shift fit. Its performance was compared with oracle alignment based on LiDAR ground truth.

- A structured supervised fine-tuning study was carried out across loss functions, trainable model components, data augmentation, and optimiser–scheduler settings. The experiments provide practical observations on the use of MAE-Canonical with a 0.5 m depth floor, decoder-only training, and geometry-consistent augmentation for the available KITTI setup. The small-subset learning-rate search tested 10^{-4}, 5×10^{-5}, and 2×10^{-5}, while the final full-set configuration used 10^{-6} with an effective batch size of 32.

- A Monodepth2-style self-supervised reprojection pipeline was implemented with both PoseNet-estimated motion and KITTI OXTS relative poses. The loss registry described in the methodology contains 18 main variants and five additional stationary-object variants. A sequential training experiment was then conducted by continuing from the selected self-supervised checkpoint with supervised LiDAR-based losses.

6.4 Limitations

There are a few limitations to the present work that should be considered when interpreting the results.

- The quantitative evaluation was carried out on the KITTI Eigen-Split benchmark, using the 652-frame held-out test set. Although this provides a common basis for comparing the experiments, it does not establish that the same performance will be obtained on other datasets or under different driving conditions. The thesis also does not report repeated runs, confidence intervals, or statistical significance tests for the differences between fine-tuning configurations.

- All reported fine-tuning experiments use KITTI data collected in Germany. The model has not been evaluated on Indian road scenes, where traffic composition, road layout, vehicle types, and image conditions can be different. As a result, generalisation to Indian dashcam footage remains an open question.

- The vehicle-anchor method uses approximate passenger-car dimensions. It does not yet distinguish between different vehicle classes, such as cars, buses, trucks, and motorcycles. Errors in segmentation boundaries, orientation estimation, and camera intrinsics can also affect the estimated anchor depths.

- The selected self-supervised A3 configuration uses relative camera poses derived from KITTI OXTS measurements during training. It does not use LiDAR depth as a training target, but it does rely on an external motion signal. The OXTS measurements are not needed when predicting depth from a single RGB frame, but the training setup is therefore not independent of additional sensor data.

- The DA3-M metric-depth conversion used in this thesis assumes the KITTI reference focal length of 721.5377 pixels. Although the depth network takes a single RGB image at inference and does not require LiDAR, the metric conversion in the reported experiments is not independent of camera intrinsics. Applying the same pipeline to a dashcam with a different focal length would require that focal length to be known or estimated. The thesis does not evaluate this calibration or focal-length estimation step across a range of different dashcam cameras, so the reported results should not be interpreted as demonstrating metric depth estimation for arbitrary uncalibrated cameras.

- Some segmentation categories have relatively few instances in the test set. In particular, the animal category contains only two masks, so its category-wise error should not be treated as representative of animal detection or depth estimation more generally.

6.5 Future Work

The results of this thesis provide a starting point for several extensions. The following steps would be useful for continuing the work.

1. **A controlled comparison of fine-tuning initialisation.** The sequential A3-to-A4 experiment provides a useful result, but it is not a controlled comparison against standalone supervised training. A follow-up experiment should keep the A4 supervised objective, training and validation split, augmentation, learning-rate schedule, training budget, and checkpoint-selection rule identical, changing only the initial weights: the original pretrained DA3-M checkpoint in one run and the selected A3 self-supervised checkpoint in the other. Repeating

this comparison with multiple random seeds would help establish whether any difference is consistent.

2. **Evaluation on Indian driving datasets and dashcam footage.** The existing IDD and TIAND projection templates should first be validated using the actual point clouds, camera images, and calibration files. Once the projected depth maps have been checked, these datasets can be considered for supervised fine-tuning or mixed-dataset training. Self-supervised fine-tuning on sequential Indian dashcam footage would provide another way to examine how well a model trained on KITTI transfers to different road environments. This would also help identify whether domain-specific training improves performance under Indian traffic and road conditions.

3. **Inference profiling and model optimisation for embedded hardware.** The inference speed and memory requirements of DA3-M should be measured on an embedded platform such as the NVIDIA Orin before considering real-time use in an ADAS pipeline. If the measured latency is too high, a smaller decoder or a more compact backbone, such as ViT-Small or a MobileNet-based architecture, could be investigated through knowledge distillation. The resulting model would then need to be evaluated for both inference efficiency and metric-depth accuracy.

Appendix A

Fully Supervised Loss Function Equations

This appendix gives the loss formulations used in the supervised fine-tuning experiments. A distinction is made between the small ablation implementation and the later full-set implementation wherever the code changed between the two. In particular, the final full-set run uses a 0.5 m floor for the inverse-depth MAE-Canonical term and a revised Masked Depth implementation. These details should not be interpreted as if every loss had one unchanged implementation throughout the experiments.

Common notation:

- $\hat{D}_i$ denotes the predicted depth at pixel i and D_i denotes the LiDAR ground-truth depth.
- $M_i \in \{0, 1\}$ is the valid-depth mask and $N_M = \sum_i M_i$ is the number of valid pixels.
- $e_i = \hat{D}_i - D_i$ is the metric depth residual.

- $r_i = \log(\hat{D}_i + \varepsilon) - \log(D_i + \varepsilon)$ is the log-depth residual.
- $\varepsilon = 10^{-6}$ is the numerical-stability constant unless a term specifies a different lower bound.
- $\lambda_{\mathrm{SI}} = 0.85$ is the variance weight used by the SILog implementation.

All masked means below are evaluated over valid LiDAR pixels unless stated otherwise.

A.1 SILog – Scale-Invariant Logarithmic Loss [10]

$$\mathcal{L}_{\mathrm{SILog}} = \sqrt{\mathrm{mean}_M(r_i^2) - \lambda_{\mathrm{SI}} \left[\mathrm{mean}_M(r_i)\right]^2 + \varepsilon}. \tag{A.1}$$

The implementation clamps predicted and reference depths to the supported positive depth interval before taking logarithms. The mean is computed over valid pixels.

A.2 BerHu – Reverse Huber Loss [24]

$$\mathcal{L}_{\mathrm{BerHu}}(e_i) = \begin{cases} |e_i|, & |e_i| \le c, \\ \dfrac{e_i^2 + c^2}{2c}, & |e_i| > c, \end{cases} \qquad c = 0.2 \max_{i:M_i=1} |e_i|. \tag{A.2}$$

The threshold is calculated from the maximum valid absolute residual for the batch and is detached from the computational graph. The loss is averaged over valid pixels.

A.3 Gradient Loss

For horizontal and vertical neighbours, define

$$M_i^x = M_i M_{i+1_x}, \qquad M_i^y = M_i M_{i+1_y}, \tag{A.3}$$

where $i + 1_x$ and $i + 1_y$ denote the adjacent pixels in the horizontal and vertical directions. The gradient residuals are

$$g_i^x = (\hat{D}_{i+1_x} - \hat{D}_i) - (D_{i+1_x} - D_i), \qquad g_i^y = (\hat{D}_{i+1_y} - \hat{D}_i) - (D_{i+1_y} - D_i). \tag{A.4}$$

The final full-set implementation normalises each direction by its number of valid adjacent pairs:

$$\mathcal{L}_{\text{grad}}^{\text{final}} = \frac{\sum_i M_i^x |g_i^x|}{\sum_i M_i^x + \varepsilon} + \frac{\sum_i M_i^y |g_i^y|}{\sum_i M_i^y + \varepsilon}. \tag{A.5}$$

The earlier HPT ablation implementation applies the same adjacent-pair masks but takes the tensor mean after multiplying by those masks, rather than dividing by the number of valid pairs:

$$\mathcal{L}_{\text{grad}}^{\text{HPT}} = \text{mean}(|g_i^x| M_i^x) + \text{mean}(|g_i^y| M_i^y). \tag{A.6}$$

The distinction is retained here because the denominator differs between the two notebook versions.

A.4 MAE-Canonical – Inverse-Depth Loss [5]

MAE-Canonical is applied to inverse depth. Let $F_{\min}$ denote the lower depth bound used before inversion:

$$p_i = \frac{1}{\operatorname{clamp}(\hat{D}_i, F_{\min}, 80)}, \qquad q_i = \frac{1}{\operatorname{clamp}(D_i, F_{\min}, 80)}. \tag{A.7}$$

The loss contains a masked inverse-depth L1 term and a multi-scale gradient term:

$$\mathcal{L}_{\text{MAE}} = \frac{\sum_i M_i |p_i - q_i|}{N_M + \varepsilon} + \frac{0.5}{4} \sum_{k=0}^{3} \mathcal{G}_k(p, q, M), \tag{A.8}$$

where $\mathcal{G}_k$ is the horizontal-and-vertical gradient difference at scale k. In the HPT ablation implementation, the inverse-depth maps and validity mask are average-pooled between scales, and the pooled mask is thresholded at 0.5 for the gradient calculation. In the final full-set notebook, pooling is mask-normalised: the mask is average-pooled, and each inverse-depth map is pooled after multiplication by the previous mask and divided by the pooled mask plus ε. This prevents invalid zero-valued pixels from biasing the coarser-scale inverse-depth values.

The lower bound differs between the experiments. In the small HPT ablation implementation, the inverse-depth inputs are clamped using the numerical lower bound (with the configured maximum depth). In the final full-set notebook, the explicit value is $F_{\min} = 0.5\,\text{m}$, so inverse depths are bounded above by $2.0\,\text{m}^{-1}$. The 0.5 m floor therefore applies specifically to the final full-set implementation; it should not be retroactively described as a setting used by every earlier ablation run. The implementation does not perform per-image min–max normalisation of inverse depth.

A.5 Masked Depth Loss

The small HPT ablation notebook uses an inverse-depth-derived confidence weight. For valid pixels, define

$$c_i = \frac{1}{D_i + \varepsilon}, \qquad \bar{c}_i = \frac{c_i}{\mathrm{mean}_M(c_i) + \varepsilon}. \tag{A.9}$$

Its implemented loss is

$$\mathcal{L}_D^{\mathrm{HPT}} = \frac{1}{N_M} \sum_i M_i \left[\bar{c}_i |e_i| - \lambda_c \log(\bar{c}_i + \varepsilon)\right], \qquad \lambda_c = 0.5. \tag{A.10}$$

This is a confidence-weighted metric residual with a logarithmic confidence regulariser; it is not a weighted SILog loss.

The final full-set training notebook uses a revised form. It first clamps the valid ground-truth depths to $D_{i,c} = \mathrm{clamp}(D_i, 0.5, 80)\,\mathrm{m}$, then normalises the inverse-depth weights to sum to one over valid pixels and computes a weighted metric L1 residual:

$$w_i = \frac{M_i / D_{i,c}}{\sum_j M_j / D_{j,c} + \varepsilon}, \qquad \mathcal{L}_D^{\mathrm{final}} = \sum_i w_i |\hat{D}_i - D_{i,c}|. \tag{A.11}$$

The final form does not include the logarithmic regulariser in Eq. A.10. Recipe A4 in the final full-set run uses this final implementation.

A.6 EG-SSI – Edge-Guided Patch Loss

The implemented Edge-Guided SSI loss does not fit one global scale and shift to the complete depth map. Instead, it identifies high-gradient image regions and compares locally standardised depth patches. Let $\mathcal{E}$ be the selected patch centres, obtained

from the strongest 5% of image-gradient responses. Patches have size 16×16; up to 64 patches are sampled, patch boundaries are excluded, and a patch is retained only when at least 25% of its pixels have valid LiDAR depth.

For a patch P, the standardisation used in the code is

$$\mathcal{N}(P)_i = \frac{P_i - \text{median}(P)}{\text{mean}_j \, |P_j - \text{median}(P)| + \varepsilon}. \tag{A.12}$$

The loss is the average L1 difference between the standardised predicted and ground-truth patches:

$$\mathcal{L}_{\text{EG-SSI}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{mean}_{i \in P} \left| \mathcal{N}(\hat{D}_P)_i - \mathcal{N}(D_P)_i \right|. \tag{A.13}$$

Here $\mathcal{P}$ is the set of retained patches. The mask is used to decide whether a patch has at least 25% valid LiDAR coverage; once accepted, the patch loss is averaged over all pixels in that patch, including locations outside the valid-pixel subset. The patch normaliser uses the mean absolute deviation from the median, not the median absolute deviation. When an RGB image is unavailable, the implementation can use ground-truth depth gradients to identify candidate regions.

A.7 Affine-Invariant SILog

For each image, an ordinary least-squares scale and shift are fitted over valid pixels:

$$(s^*, t^*) = \arg\min_{s,t} \sum_i M_i (s\hat{D}_i + t - D_i)^2, \qquad \hat{D}_i^{\text{aff}} = s^* \hat{D}_i + t^*. \tag{A.14}$$

The aligned prediction is clamped to the supported positive depth interval before the logarithmic terms are evaluated. The implemented objective combines an aligned

SILog term, an aligned gradient term and a metric anchor on the unaligned prediction:

$$\mathcal{L}_{\text{AI-SILog}} = \mathcal{L}_{\text{SILog}}(\hat{D}^{\text{aff}}, D) + 0.1\mathcal{L}_{\text{grad}}(\hat{D}^{\text{aff}}, D) + 0.3\mathcal{L}_{\text{SILog},\lambda=0}(\hat{D}, D). \tag{A.15}$$

The final term uses zero scale-invariance weight and therefore retains a metric-scale penalty. The implementation does not use median/MAD standardisation.

A.8 TGM – Temporal Gradient Matching [8]

For consecutive frames t and $t+1$, the joint valid mask is combined with a ground-truth static-change condition:

$$M_{s,i} = M_{t,i} M_{t+1,i}\, \mathbf{1}[|D_{t+1,i} - D_{t,i}| < 0.05 \text{ m}]\,. \tag{A.16}$$

The predicted and reference depth changes are measured in metric depth:

$$\Delta\hat{D}_i = |\hat{D}_{t+1,i} - \hat{D}_{t,i}|, \qquad \Delta D_i = |D_{t+1,i} - D_{t,i}|. \tag{A.17}$$

The temporal matching component is

$$\mathcal{L}_{\text{TGM}}^{\text{match}} = \frac{\sum_i M_{s,i} |\Delta\hat{D}_i - \Delta D_i|}{\sum_i M_{s,i} + \varepsilon}. \tag{A.18}$$

The standalone TGM module adds a current-frame SILog term and applies a factor of 10 to the temporal component:

$$\mathcal{L}_{\text{TGM}}^{\text{standalone}} = \mathcal{L}_{\text{SILog}}(\hat{D}_t, D_t) + 10\mathcal{L}_{\text{TGM}}^{\text{match}}. \tag{A.19}$$

The DA3-Video combined loss below uses the matching component directly with weight 10; it does not add the standalone TGM module's SILog term a second time.

A.9 VNL – Virtual Normal Loss [44]

The implementation uses the scalar KITTI focal length and image centre to back-project valid pixels, with $c_x = W/2$ and $c_y = H/2$. For pixel coordinates (u_i, v_i),

$$\mathbf{P}_i(D) = \begin{bmatrix} (u_i - c_x)D_i/f \\ (v_i - c_y)D_i/f \\ D_i \end{bmatrix}. \tag{A.20}$$

For each sampled valid triplet (i, j, k), a predicted and a ground-truth normal are formed:

$$\mathbf{n}_{ijk}(D) = \frac{(\mathbf{P}_j - \mathbf{P}_i) \times (\mathbf{P}_k - \mathbf{P}_i)}{\|(\mathbf{P}_j - \mathbf{P}_i) \times (\mathbf{P}_k - \mathbf{P}_i)\|_2 + \varepsilon}. \tag{A.21}$$

The loss is

$$\mathcal{L}_{\text{VNL}} = \frac{1}{|\mathcal{T}|} \sum_{(i,j,k)\in\mathcal{T}} \left(1 - \mathbf{n}_{ijk}(\hat{D})^{\mathsf{T}} \mathbf{n}_{ijk}(D)\right), \tag{A.22}$$

where up to 2,000 valid-pixel triplets are sampled per image. The code uses the scalar focal-length formulation rather than a general 3×3 intrinsic-matrix back-projection.

A.10 Log-L1 Loss

$$\mathcal{L}_{\text{LogL1}} = \text{mean}_M \left|\log(\hat{D}_i + \varepsilon) - \log(D_i + \varepsilon)\right| + 0.1\mathcal{L}_{\text{grad}}(\hat{D}, D). \tag{A.23}$$

The gradient term is part of the implemented loss and is not omitted from the definition.

A.11 SSIM+L1 Loss

This ablation compares predicted and reference depth maps, rather than RGB images. Both depth maps are clamped to the supported range and divided by 80 before the local SSIM calculation. A 7×7 uniform averaging window is used, with $C_1 = 10^{-4}$ and $C_2 = 9 \times 10^{-4}$. The implemented combination is

$$\begin{aligned}\mathcal{L}_{\text{SSIM+L1}} = 0.85\,\text{mean}_M\left(\frac{1-\text{SSIM}(\hat{D}/80, D/80)}{2}\right) \\ + 0.15\,\text{mean}_M\left(\left|\frac{\hat{D}-D}{80}\right|\right).\end{aligned} \tag{A.24}$$

A.12 Robust Cauchy Loss [1]

The robust scale is obtained from the detached median absolute valid residual, with a lower bound of 10^{-3}:

$$c = \max\left(10^{-3}, \text{median}_{i:M_i=1}\,|e_i|\right). \tag{A.25}$$

The complete implemented objective is

$$\mathcal{L}_{\text{Cauchy}} = \text{mean}_M\left[\log\left(1+\frac{e_i^2}{2c^2}\right)\right] + 0.5\mathcal{L}_{\text{SILog}}(\hat{D}, D). \tag{A.26}$$

A.13 Combined-Base Loss

$$\mathcal{L}_{\text{combined-base}} = \mathcal{L}_{\text{SILog}} + \mathcal{L}_{\text{BerHu}} + 0.5\mathcal{L}_{L1} + 0.1\mathcal{L}_{\text{grad}}, \tag{A.27}$$

where $\mathcal{L}_{L1} = \text{mean}_M\,|e_i|$. This was used as the baseline loss while the other supervised ablation dimensions were examined.

A.14 DA3-Video Combined Loss

The implemented video-combined objective in the supervised HPT notebook is

$$\mathcal{L}_{\text{DA3-video}} = \mathcal{L}_{D}^{\text{HPT}} + \mathcal{L}_{\text{grad}}^{\text{HPT}} + \mathcal{L}_{\text{EG-SSI}} + 10\mathcal{L}_{\text{TGM}}^{\text{match}}. \tag{A.28}$$

It combines sparse-depth supervision with spatial and temporal constraints. The temporal component requires consecutive frames. The expression is not the SILog-plus-smoothness formulation sometimes used for video depth models.

Appendix B

Fully Supervised Ablation Hyperparameter Summary

This appendix documents the small-subset supervised HPT ablation and the full-set supervised configurations reported in Chapter 5. The standalone A2 and A4 runs are distinguished from the sequential A3-initialised A4 run because their starting checkpoints and training histories differ.

B.1 Small-Subset Supervised HPT Configuration

Table B.1: Verified settings for the small-subset supervised HPT ablation.

Parameter	**Value**
Dataset and split	
Training samples	488
Validation samples	85
Held-out KITTI Eigen test frames	46

Train/validation fraction	85/15 (validation fraction 0.15)
Split seed	42
Model and input	
Base model	`depth-anything/DA3METRIC-LARGE`
Default architecture	Decoder-only
Total parameters	Approximately 334.2 million
Trainable parameters (decoder-only)	Approximately 29.8 million
Input resolution	378×1246 pixels
Depth range for evaluation	$[10^{-3}, 80]$ m
HPT training budget	
Maximum epochs per candidate	5
Early-stopping patience	3 epochs
Learning-rate candidates	$\{10^{-4}, 5 \times 10^{-5}, 2 \times 10^{-5}\}$
Batch size	4
Gradient accumulation	2 steps
Effective batch size	8
Mixed precision	FP16 with gradient scaling
Maximum gradient norm	1.0
Default optimisation configuration	
Default optimiser	AdamW
Default scheduler	Cosine warmup
Weight decay	10^{-4}
Warmup	1 epoch
Minimum learning rate	1% of the selected base learning rate
Evaluation	
HPT selection metric	Validation pixel-weighted RMSE
Checkpoint selection	Lowest validation RMSE

Test-set use	Evaluation after selection of the HPT candidate and checkpoint

The HPT notebook examines 13 individual loss configurations, three architecture modes, four augmentation modes and five optimiser/scheduler combinations. The default settings are decoder-only training, base augmentation (horizontal flipping and colour jitter), and AdamW with cosine warmup. The loss grid and recipe grid are listed below.

B.2 Ablation Dimension Grid

Table B.2: Supervised ablation dimensions and configurations.

Dimension	**Variants**	**Default**
Loss function	`silog`, `berhu`, `combined_base`, `vnl`, `log_l1`, `ssim_l1`, `affine_invariant`, `robust_cauchy`, `masked_depth`, `mae_canonical`, `eg_ssi`, `tgm`, `da3_video_combined`	combined base
Architecture	`decoder_only`, `last2_blocks`, `full`	decoder only
Augmentation	`none`, `flip`, `base` (flip + colour jitter), `full` (flip + colour jitter + crop)	base
Optimiser	`adamw+cosine_warmup`, `adamw+step`, `adamw+onecycle`, `adam+cosine`, `sgd+cosine`	AdamW + cosine warmup

The full-crop augmentation is included as an ablation condition; it is not the selected augmentation for the final full-set run. The final training notebook selects the base augmentation mode.

B.3 Combined-Loss Recipes

TABLE B.3: Supervised combined-loss recipes defined in the training pipeline.

Recipe	**Loss composition**
A1	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + 0.1\mathcal{L}_{\text{grad}}$
A2	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}}$
A3	A1 $+0.5\mathcal{L}_{\text{BerHu}}$
A4	A1 $+0.5\mathcal{L}_{D}$
A5	A1 $+1.0\mathcal{L}_{D}$
B1	$0.5\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + \mathcal{L}_{\text{BerHu}} + 0.5\mathcal{L}_{L1} + 0.1\mathcal{L}_{\text{grad}}$
B2	$\mathcal{L}_{\text{MAE}} + \mathcal{L}_{\text{SILog}} + \mathcal{L}_{\text{BerHu}} + 0.5\mathcal{L}_{L1} + 0.1\mathcal{L}_{\text{grad}}$
C	$\mathcal{L}_{\text{MAE}}$ alone; SGD with Nesterov momentum and learning rate 10^{-4}

B.4 Standalone 100-Epoch A4 Supervised Training Configuration

TABLE B.4: Configuration of the standalone 100-epoch full-set A4 supervised run.

Parameter	**Value**
Dataset and split	
Total available training samples	23,158
Training samples	20,842

Validation samples	2,316
Validation fraction	0.10
Split seed	42
Held-out KITTI Eigen test frames	652
Model and recipe	
Base model	`depth-anything/DA3METRIC-LARGE`
Selected loss recipe	A4
Architecture	Decoder-only
Augmentation	Base (horizontal flip + colour jitter)
Optimiser	AdamW
Scheduler	Cosine warmup
Total parameters	Approximately 334.2 million
Trainable parameters	Approximately 29.8 million
Input resolution	378×1246 pixels
Training budget	
Maximum epochs	100
Early-stopping patience	40 epochs
Learning rate	1×10^{-6}
Minimum learning rate	1×10^{-8}
Warmup	1 epoch
Batch size	8
Gradient accumulation	4 steps
Effective batch size	32
Weight decay	10^{-4}
Maximum gradient norm	1.0
Mixed precision	FP16 with gradient scaling
Depth and loss settings	
Evaluation depth range	$[10^{-3}, 80]$ m

MAE-Canonical depth floor	0.5 m
MAE-Canonical inverse-depth upper bound	$2.0\,\mathrm{m}^{-1}$
SILog variance weight	0.85
Mask in final Masked Depth term	Inverse-ground-truth-depth weights normalised to sum to one
Checkpoint and test evaluation	
Best-checkpoint metric	Validation pixel-weighted RMSE
Checkpoint saving	Every epoch
Periodic test evaluation	Enabled; full test set selected by `PERIODIC_TEST_SAMPLES=0`

The selected final recipe is

$$\mathcal{L}_{A4} = \mathcal{L}_{\mathrm{MAE}} + \mathcal{L}_{\mathrm{SILog}} + 0.1\mathcal{L}_{\mathrm{grad}} + 0.5\mathcal{L}_{D}. \tag{B.1}$$

For this final run, $\mathcal{L}_D$ is the revised weighted metric L1 form in Appendix A, Eq. A.11. The final notebook sets the minimum learning rate to 10^{-8}, which is 1% of the base learning rate.

B.5 Other Full-Set Supervised Runs Reported in Chapter 5

The 100-epoch configuration above is only one of the supervised runs reported in Chapter 5. The additional runs are summarised in Table B.5. In particular, the sequential A3-to-A4 experiment starts from a self-supervised checkpoint, whereas the

standalone A2 and A4 experiments are separate supervised runs. These configurations should not be treated as a single training run or as directly matched experiments.

TABLE B.5: Checkpoint initialisation and principal training settings for the full-set supervised runs discussed in Chapter 5.

Run	**Recipe and starting weights**	**Epochs / patience**	**Batch / accumulation**	**Checkpoint handling**
Standalone A2	A2; pretrained DA3-M	50 / 30	8/4 (effective 32)	Starts at epoch 1; no resumed checkpoint
Supervised A4 (50-epoch configuration)	A4; existing A4 supervised checkpoint (not A3-initialised)	50 / 30	8/4 (effective 32)	Config records `start_epoch=46` and resume from A4 `latest.pth`; periodic log covers epochs 1–50
Standalone A4 (100-epoch configuration)	A4; pretrained DA3-M	100 / 40	8/4 (effective 32)	Separate run; report uses the validation-selected checkpoint
Sequential A3-to-A4	A4; best A3 self-supervised checkpoint from epoch 27	50 / 20	8/4 (effective 32)	Starts A4 at epoch 1; `resumed_from=null`; `init_checkpoint=best_model_SSFT_A3.pt`

For all four supervised configurations, the learning rate was 10^{-6}, weight decay was 10^{-4}, the validation fraction was 0.10 with split seed 42, and the input resolution was 378×1246 pixels. The 50-epoch A4 configuration records a resume point at epoch 46 in its configuration file, while its periodic metric log contains entries for epochs 1–50; Chapter 5 reports both pieces of run metadata rather than treating the configuration

field as a different 50-epoch result. The A2 configuration and both standalone A4 configurations used the 0.5 m MAE-Canonical depth floor. The sequential A3-to-A4 run also used the final A4 supervised loss implementation and projected KITTI LiDAR depth. The A4 sequential initialisation is not a resume of an earlier A4 run: its configuration starts a new supervised schedule and separately specifies the A3 checkpoint as the initial model weights.

Appendix C

Self-Supervised Loss Function Equations

The self-supervised pipeline is described as Stage 8 in Chapter 3. The current registry contains 23 loss entries: 13 photometric, temporal, geometric and regularisation entries; three OXTS ground-truth-pose variants; two hybrid or supervised-control entries; and five stationary-object entries. The 23 entries should not all be interpreted as pure self-supervised objectives. In particular, the OXTS variants use an externally measured motion signal, while the LiDAR anchor is supervised.

Common notation:

- I_t and I_s are the target RGB frame and a source frame, respectively; $s \in \{t-1, t+1\}$ for the three-frame training window.
- $\hat{D}_t$ is the predicted metric depth map at the target frame.
- $\mathbf{T}_{t\to s} = (\mathbf{R}, \mathbf{t}) \in SE(3)$ is the relative camera pose, obtained from PoseNet or KITTI OXTS according to the experiment.

- $\mathbf{K}$ is the camera intrinsic matrix; $V_{t\to s}$ is the valid projection mask.
- $\hat{I}_{s\to t}$ is the source frame warped into the target viewpoint by differentiable bilinear sampling.
- $\varepsilon = 10^{-6}$ and the photometric SSIM coefficient is $\alpha = 0.85$.

C.1 Photometric Losses

C.1.1 Photometric L1: `photo_l1`

For each source frame, the masked channel-mean L1 residual is

$$\mathcal{L}_{\text{photo-L1}} = \frac{1}{|\mathcal{S}|} \sum_{s\in\mathcal{S}} \frac{\sum_i V_{t\to s,i} \left\| \hat{I}_{s\to t,i} - I_{t,i} \right\|_1^{\text{ch}}}{\sum_i V_{t\to s,i} + \varepsilon}, \qquad \mathcal{S} = \{t-1, t+1\}. \tag{C.1}$$

Here $\|\cdot\|_1^{\text{ch}}$ denotes the mean absolute difference across RGB channels.

C.1.2 Photometric SSIM: `photo_ssim`

The standalone SSIM loss uses a 7×7 uniform averaging window with $C_1 = 10^{-4}$ and $C_2 = 9 \times 10^{-4}$:

$$\mathcal{L}_{\text{photo-SSIM}} = \frac{1}{|\mathcal{S}|} \sum_{s\in\mathcal{S}} \frac{\sum_i V_{t\to s,i} \dfrac{1 - \text{SSIM}_{7\times 7}(\hat{I}_{s\to t}, I_t)_i}{2}}{\sum_i V_{t\to s,i} + \varepsilon}. \tag{C.2}$$

C.1.3 Combined SSIM and L1 Residual: `photo_ssim_l1`

The combined photometric residual used by the reprojection losses is

$$\mathrm{pe}(\hat{I}, I) = 0.85 \frac{1 - \mathrm{SSIM}_{3\times3}(\hat{I}, I)}{2} + 0.15 \|\hat{I} - I\|_1^{\mathrm{ch}}, \tag{C.3}$$

where the SSIM calculation uses a 3×3 uniform averaging kernel, $C_1 = 0.01^2$, and $C_2 = 0.03^2$. The channel dimension is averaged, giving a single-channel residual map. The loss is

$$\mathcal{L}_{\text{photo-SSIM+L1}} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \frac{\sum_i V_{t\to s,i}\, \mathrm{pe}(\hat{I}_{s\to t}, I_t)_i}{\sum_i V_{t\to s,i} + \varepsilon}. \tag{C.4}$$

The 3×3 constants above belong to this combined residual; they are not the constants of the standalone 7×7 SSIM loss.

C.1.4 Minimum Reprojection and Auto-Masking

Registered name: `min_reproj_photo`.

For each source frame, the reprojection residual and identity residual are

$$e_{s,i}^{\mathrm{reproj}} = \mathrm{pe}(\hat{I}_{s\to t}, I_t)_i, \qquad e_{s,i}^{\mathrm{id}} = \mathrm{pe}(I_s, I_t)_i. \tag{C.5}$$

The minimum reprojection residual is selected across the source frames. Invalid projected pixels receive a large residual before the minimum is taken. The auto-mask retains pixels where reprojection is strictly better than identity. Its threshold is zero, and a small identity noise term (10^{-5}) is added for tie-breaking:

$$e_i^{\min} = \min_{s \in \mathcal{S}} e_{s,i}^{\mathrm{reproj}}, \qquad A_i = \mathbf{1}[e_i^{\min} < \min_{s \in \mathcal{S}} e_{s,i}^{\mathrm{id}}]. \tag{C.6}$$

The resulting objective is

$$\mathcal{L}_{\text{min-reproj}} = \frac{\sum_i V_i A_i e_i^{\min}}{\sum_i V_i A_i + \varepsilon}. \tag{C.7}$$

C.1.5 Multi-Scale Photometric Loss: `multi_scale_photo`

The implementation uses a four-level image pyramid. At each level, the target and source images and predicted depth are downsampled, and the intrinsic matrix is resized to the corresponding image resolution before reprojection. The minimum-reprojection loss is computed at that scale:

$$\mathcal{L}_{\text{multi-scale}} = \frac{1}{4} \sum_{s=0}^{3} \mathcal{L}_{\text{min-reproj}}^{(s)}. \tag{C.8}$$

The loss is evaluated at each pyramid resolution; depth is not simply upsampled to the original resolution before every warp.

C.2 Temporal and Geometric Consistency

Let $\mathbf{P}_t = \hat{D}_t \mathbf{K}^{-1}[u, v, 1]^\mathsf{T}$ be a back-projected point. After transformation into the source camera, let D_{compute} denote its transformed z coordinate. Let D_{proj} denote the source depth map sampled at the projected location. The relative depth residual is

$$\rho_i = \frac{|D_{\text{proj},i} - D_{\text{compute},i}|}{D_{\text{proj},i} + D_{\text{compute},i} + \varepsilon}. \tag{C.9}$$

The valid mask requires an in-frame projection and positive, valid depth values. Where dynamic-object filtering is enabled, the corresponding dynamic mask is also excluded.

C.2.1 Geometric Consistency

Registered name: `geometric_consistency`.

$$\mathcal{L}_{\text{geom}} = \frac{\sum_i V_i \rho_i}{\sum_i V_i + \varepsilon}. \tag{C.10}$$

C.2.2 Temporal Warp Consistency

Registered name: `temporal_warp_consistency`.

The temporal-warp module uses the same normalised relative depth residual. Its stand-alone wrapper can also include a photometric component, controlled by a separate weight:

$$\mathcal{L}_{\text{temporal-warp}} = \frac{\sum_i V_i \rho_i}{\sum_i V_i + \varepsilon} + \lambda^{\text{TW}}_{\text{photo}} \mathcal{L}_{\text{photo-SSIM+L1}}. \tag{C.11}$$

The default stand-alone wrapper uses $\lambda^{\text{TW}}_{\text{photo}} = 0.5$. In both combined-recommended recipes, this inner photometric weight is set to zero because minimum reprojection is already included. The temporal term in those recipes is therefore only the normalised depth-warp residual.

C.2.3 Naive Temporal Delta

Registered name: `naive_temporal_delta`.

This baseline operates on the predicted depth difference between adjacent frames. It penalises deviation from the mean temporal difference within the sample:

$$\Delta \hat{D}_i = \hat{D}_{t+1,i} - \hat{D}_{t,i}, \qquad \overline{\Delta \hat{D}} = \text{mean}_i(\Delta \hat{D}_i), \tag{C.12}$$

$$\mathcal{L}_{\text{naive-temporal}} = \text{mean}_i \, |\Delta\hat{D}_i - \overline{\Delta\hat{D}}|. \tag{C.13}$$

It does not use a ground-truth static-pixel mask and is not a squared penalty that forces every predicted depth difference to zero.

C.2.4 Rigid-Flow Consistency

Registered name: `rigid_flow_consistency`.

The rigid-flow field is induced by depth and camera-pose reprojection. The loss combines the photometric reconstruction residual with an edge-aware smoothness term on the flow magnitude:

$$\mathcal{L}_{\text{rigid-flow}} = \mathcal{L}_{\text{photo-SSIM+L1}} + 0.1\mathcal{L}_{\text{smooth}}(\|\mathbf{f}\|_2, I_t). \tag{C.14}$$

The current registry name is `rigid_flow_consistency`. The earlier name, `flow_depth_consistency`, is obsolete.

C.3 Feature-Metric Loss

C.3.1 `feature_metric`

A frozen ImageNet-pretrained ResNet-18 feature extractor is used up to `layer1`. The implementation combines an L1 difference between warped and target feature maps with a raw pixel-space L1 residual:

$$\mathcal{L}_{\text{feature-metric}} = 0.5 \left\| \phi(\hat{I}_{s\to t}) - \phi(I_t) \right\|_1 + 0.5 \left\| \hat{I}_{s\to t} - I_t \right\|_1. \tag{C.15}$$

The pixel-space term is raw L1, not the SSIM-plus-L1 photometric residual.

C.4 Edge-Aware Smoothness

C.4.1 `edge_aware_smooth` [16]

The predicted disparity is normalised by its spatial mean:

$$\bar{d} = \frac{1/\hat{D}}{\text{mean}(1/\hat{D}) + \varepsilon}. \tag{C.16}$$

The edge-aware smoothness loss is

$$\mathcal{L}_{\text{smooth}} = \text{mean}\left(|\partial_x \bar{d}| e^{-\gamma|\partial_x I_t|} + |\partial_y \bar{d}| e^{-\gamma|\partial_y I_t|}\right), \qquad \gamma = 1. \tag{C.17}$$

The default weight in the combined recipes is 10^{-3}.

C.5 Sparse LiDAR Anchor

C.5.1 `lidar_anchor_only`

This entry is a supervised control based on sparse LiDAR depth, not a pure self-supervised loss. With r_i denoting the valid-pixel log-depth residual,

$$\mathcal{L}_{\text{anchor}} = \sqrt{\text{mean}_M(r_i^2) - 0.85\left[\text{mean}_M(r_i)\right]^2 + \varepsilon}. \tag{C.18}$$

The hybrid combined recipe uses an anchor weight of 0.5; the pure self-supervised recipe sets that weight to zero.

C.6 OXTS Ground-Truth-Pose Variants

The three OXTS variants use KITTI OXTS relative poses instead of predicted PoseNet poses:

- `gt_photo_reproj`: combined SSIM+L1 photometric reprojection using OXTS pose.
- `gt_rigid_flow_consistency`: rigid-flow consistency using OXTS pose.
- `gt_temporal_warp_consistency`:
  temporal-warp consistency using OXTS pose.

The current OXTS wrappers require the OXTS pose input and do not silently fall back to PoseNet. These variants use ground-truth motion, not ground-truth depth, and are best treated as pose-oracle references.

Current registry key: `gt_rigid_flow_consistency`.

Obsolete earlier key: `gt_flow_depth_consistency`.

C.7 Combined Recipes

C.7.1 Monodepth2-Style Combination

Registered name: `combined_monodepth2`.

$$\mathcal{L}_{\text{mono2}} = \mathcal{L}_{\text{multi-scale}} + 10^{-3}\mathcal{L}_{\text{smooth}}. \tag{C.19}$$

C.7.2 Pure Self-Supervised Combination

Registered name: `combined_recommended_ssl`.

$$\mathcal{L}_{\text{recommended-SSL}} = 1.0\mathcal{L}_{\text{min-reproj}} + 10^{-3}\mathcal{L}_{\text{smooth}} + 0.1\mathcal{L}_{\text{temporal-warp}}^{(\lambda_{\text{photo}}^{\text{TW}}=0)}. \tag{C.20}$$

No LiDAR anchor is included in this pure self-supervised recipe.

C.7.3 Hybrid Combination

Registered name: `combined_recommended`.

$$\mathcal{L}_{\text{recommended}} = 1.0\mathcal{L}_{\text{min-reproj}} + 10^{-3}\mathcal{L}_{\text{smooth}} + 0.1\mathcal{L}_{\text{temporal-warp}}^{(\lambda_{\text{photo}}^{\text{TW}}=0)} + 0.5\mathcal{L}_{\text{anchor}}. \tag{C.21}$$

This is a hybrid objective because it includes sparse LiDAR supervision. The geometric consistency loss is a separate registry entry and is not an additional term in this combined-recommended expression.

C.8 Stationary-Object Consistency Losses

The stationary-object group uses object masks tracked between consecutive frames. Before depth statistics are calculated, masks are eroded by two pixels and resized to the model resolution. A detection is skipped if fewer than five valid depth pixels remain. For each matched object o, the implementation uses median depth and the mask centroid to construct a representative 3D point. Let $\mathbf{P}_{t,o}$ be the current-frame point back-projected using the camera intrinsics, and let

$$\mathbf{P}_{t+1,o}^{\text{exp}} = \mathbf{R}\mathbf{P}_{t,o} + \mathbf{t} \tag{C.22}$$

be its expected location in the next camera frame. Let $d^{\text{obs}}_{t+1,o}$ and $\mathbf{P}^{\text{obs}}_{t+1,o}$ denote the depth and 3D point observed at the matched next-frame centroid. Object weights are proportional to the square root of the average mask area across the pair.

C.8.1 Stationary Depth Motion

Registered name: `stationary_depth_motion`.

$$\mathcal{L}_{\text{stationary-depth}} = \frac{\sum_o w_o \left| d^{\text{obs}}_{t+1,o} - P^{\text{exp}}_{t+1,o,z} \right|}{\sum_o w_o + \varepsilon}. \tag{C.23}$$

C.8.2 Stationary Uniform Motion

Registered name: `stationary_uniform_motion`.

For each sample, define the signed residual $r_o = d^{\text{obs}}_{t+1,o} - P^{\text{exp}}_{t+1,o,z}$. When at least two objects are available, the uniformity term measures the mean absolute deviation from the median residual:

$$\mathcal{L}_{\text{uniformity}} = \text{mean}_{\text{samples}} \left[\text{mean}_o \left| r_o - \text{median}_{o'}(r_{o'}) \right| \right]. \tag{C.24}$$

The current configuration selects the MAD-style option (rather than the optional variance alternative).

C.8.3 Full 3D Ego-Motion

Registered name: `full_3d_ego_motion`.

$$\mathcal{L}_{\text{3D-motion}} = \frac{\sum_o w_o \left\| \mathbf{P}^{\text{obs}}_{t+1,o} - \mathbf{P}^{\text{exp}}_{t+1,o} \right\|_2}{\sum_o w_o + \varepsilon}. \tag{C.25}$$

C.8.4 Mask Chamfer

Registered name: `mask_chamfer`.

For each matched object, up to 200 valid mask pixels are sampled per frame and back-projected using their own predicted depth values. The current-frame point cloud is transformed into the next camera frame. The bidirectional Chamfer distance is

$$\mathcal{L}_{\text{chamfer}}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|_2^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|_2^2. \tag{C.26}$$

The per-object distances are averaged over the available matched objects.

C.8.5 Combined Stationary Consistency

Registered name: `combined_stationary`.

The combined stationary-object module reuses the photometric, smoothness and temporal-warp terms and adds the four object-level constraints:

$$\begin{aligned} \mathcal{L}_{\text{combined-stationary}} = {} & 1.0\mathcal{L}_{\text{min-reproj}} + 10^{-3}\mathcal{L}_{\text{smooth}} + 0.1\mathcal{L}_{\text{temporal-warp}}^{(\lambda_{\text{photo}}^{\text{TW}}=0)} \\ & + 0.5\mathcal{L}_{\text{stationary-depth}} + 0.2\mathcal{L}_{\text{uniformity}} \\ & + 0.5\mathcal{L}_{\text{3D-motion}} + 0.1\mathcal{L}_{\text{chamfer}}. \end{aligned} \tag{C.27}$$

The uniformity and Chamfer components are controlled by separate configuration switches in the notebook.

C.9 Registry Summary

TABLE C.1: Current loss-registry entries and their categories.

Category	**Registered entries**
Photometric, temporal, geometric and regularisation (13)	`photo_l1`, `photo_ssim`, `photo_ssim_l1`, `min_reproj_photo`, `multi_scale_photo`, `geometric_consistency`, `rigid_flow_consistency`, `temporal_warp_consistency`, `naive_temporal_delta`, `feature_metric`, `edge_aware_smooth`, `combined_monodepth2`, `combined_recommended_ssl`
OXTS pose-oracle variants (3)	`gt_photo_reproj`, `gt_rigid_flow_consistency`, `gt_temporal_warp_consistency`
Hybrid/supervised controls (2)	`combined_recommended`, `lidar_anchor_only`
Stationary-object group (5)	`stationary_depth_motion`, `stationary_uniform_motion`, `full_3d_ego_motion`, `mask_chamfer`, `combined_stationary`
Total	**23 registered entries**

Appendix D

Self-Supervised Ablation Hyperparameter Summary

This appendix distinguishes the current self-supervised HPT notebook from the separate final self-supervised training notebook. The HPT notebook is a corrected stationary-object ablation build; its active switches do not execute the complete 23-entry registry. The final notebook uses a larger sequential sample set and a separate set of final recipe switches.

D.1 Self-Supervised HPT Configuration

Table D.1: Configuration in the supplied stationary-fix SSL HPT notebook.

Parameter	**Value**
Dataset and sequence construction	
Dataset subset	KITTI sequential subset (notebook path labelled `tiniest`)

Canonical sequential samples	1,609
Training samples	1,449
Validation samples	160
Validation fraction and seed	0.10; seed 42
Held-out KITTI Eigen test frames	652
Source-frame offsets	$(-1, +1)$ relative to the target frame
Input resolution	378×1246 pixels
Depth model and pose estimation	
Depth model	`depth-anything/DA3METRIC-LARGE`
Depth architecture default	Decoder-only
PoseNet option	ResNet-18 with six-channel paired-frame input for predicted-pose variants
PoseNet output	Axis-angle rotation (3) and translation (3)
Pose output scale	0.1
PoseNet warm-up	1 epoch with depth frozen for predicted-pose variants
PoseNet learning rate	10^{-4}
HPT budget	
Maximum epochs per trial	5
Early-stopping patience	4 epochs
Depth learning-rate candidates	$\{10^{-5}, 10^{-6}\}$
Configured default depth learning rate	10^{-6}
Batch size	4
Gradient accumulation	4 steps
Effective batch size	16
Weight decay	10^{-4}
Warm-up	1 epoch
Maximum gradient norm	1.0

Mixed precision	FP16 for the depth network; geometry and loss calculations in FP32
Camera and numerical settings	
KITTI focal-length reference	721.5377 pixels (used as the configured scalar reference)
Model-predicted focal length	Disabled
Depth range	$[10^{-3}, 80]$ m
Numerical epsilon	10^{-6}
Photometric SSIM mixture coefficient	0.85
Auto-mask threshold	0.0
Smoothness decay	1.0
Stationary-object processing	
Tracker setting	`hungarian` (configured choice; not established as an optimum)
Pose and tracking inputs	OXTS and ground-truth-pose paths enabled
Object tracking	Enabled; Hungarian tracker; IoU threshold 0.15
Track management	Maximum age 5 frames; minimum hits 1
Dynamic-object filter	Class and flow modes enabled; mask dilation 3 pixels
Geometric filter	Disabled
Mask erosion	2 pixels before depth statistics
Minimum valid mask depth pixels	5
Uniformity statistic	MAD-style deviation from median
Maximum Chamfer points	200 per object per frame
Stationary depth weight	0.5
Stationary anchor weight	0.5
Uniformity weight	0.2
3D motion weight	0.5

Chamfer weight	0.1

D.1.1 Saved stationary-object ablation run and results

The supplied experiment configuration identifies code version `v6.2.1-stationary-fix` and the KITTI sequential subset labelled `tiniest`. The stationary-object processing used the configured Hungarian tracker, class- and flow-based dynamic filtering, mask erosion, uniformity statistics, and Chamfer-based terms. Tracking, uniformity, Chamfer processing, dynamic filtering, OXTS, and the ground-truth-pose path were enabled in the saved configuration; the geometric filter was disabled. The HPT budget was five epochs per trial with early-stopping patience of four epochs. Two depth-learning-rate candidates, 10^{-5} and 10^{-6}, were considered, with a per-step batch size of 4 and four gradient-accumulation steps (effective batch size 16).

The saved `ablation_summary.csv` contains five stationary-object configurations. Each record covers 652 KITTI Eigen-Split test frames and records two HPT trials. The selected HPT tag is `lr=1e-6` and the recorded winning epoch is 1 for each configuration. These fields describe the saved sweep summary; the five-epoch value above is the configured trial budget. The resulting test metrics are summarised in Table D.2. The unscaled RMSE and δ_1 values match the corresponding entries in Chapter 5, Table 5.14. One exception is the unscaled AbsRel for Stationary Depth Motion: the supplied summary CSV gives 0.1348, whereas Chapter 5, Table 5.14, lists 0.1344. The CSV value is retained in this appendix because it records the supplied saved summary; the difference should be reconciled if both tables are intended to report the same AbsRel aggregation.

TABLE D.2: Saved stationary-object SSL ablation metrics across unscaled and LiDAR-aligned evaluations. Each cell lists RMSE (m), AbsRel, δ_1, and mean error (LiDAR depth minus predicted depth, m), in that order; positive mean error indicates depth underestimation.

Configuration	**Unscaled**	**Median aligned**	**Mean aligned**	**Affine aligned**
	3.6334 / 0.1348	2.6619 / 0.0634	2.5357 / 0.0623	2.4745 / 0.0614
Stationary Depth	0.8707 / +1.9805	0.9688 / +0.1546	0.9721 / +0.0063	0.9709 / +0.0047
Motion				
	3.5815 / 0.1309	2.6635 / 0.0631	2.5370 / 0.0620	2.4761 / 0.0611
Stationary Uniform	0.8818 / +1.9155	0.9689 / +0.1616	0.9722 / +0.0062	0.9709 / +0.0046
Motion				
	3.7780 / 0.1419	2.6641 / 0.0628	2.5357 / 0.0618	2.4759 / 0.0614
Full 3D Ego Motion	0.8500 / +2.1351	0.9689 / +0.1733	0.9723 / +0.0063	0.9708 / +0.0049
	4.2028 / 0.1558	2.7190 / 0.0627	2.5529 / 0.0621	2.4822 / 0.0637
Mask Chamfer	0.7946 / +2.4904	0.9677 / +0.3380	0.9719 / +0.0051	0.9689 / +0.0050
	4.0129 / 0.1488	2.6813 / 0.0622	2.5332 / 0.0614	2.4694 / 0.0624
Combined Stationary	0.8249 / +2.3344	0.9685 / +0.2765	0.9723 / +0.0053	0.9699 / +0.0049

The unscaled comparison shows that Stationary Uniform Motion has the lowest RMSE among the five configurations (3.5815 m), while Mask Chamfer has the highest (4.2028 m). Median, mean, and affine alignment further reduce the reported errors, but these are LiDAR-ground-truth-aligned diagnostic results rather than inference-time performance. In particular, the aligned columns should not be interpreted as improvements produced by the self-supervised losses alone.

D.2 Final A3 Self-Supervised Training Configuration

TABLE D.3: Configuration of the final OXTS-based A3 self-supervised training run.

Parameter	**Value**
Dataset and split	
Dataset source	KITTI sequential set with OXTS information
Canonical sequential samples	23,092

Training samples	20,783
Validation samples	2,309
Validation fraction and seed	0.10; seed 42
Held-out KITTI Eigen test frames	652
Source-frame offsets	$(-1, +1)$
Model and optimisation	
Depth model	`depth-anything/DA3METRIC-LARGE`
Architecture	Decoder-only
Augmentation	Base (consistent horizontal flip + colour jitter)
Optimiser and scheduler	AdamW + cosine warmup
Depth learning rate	10^{-6}
PoseNet learning rate (predicted-pose recipes)	10^{-4}
Weight decay	10^{-4}
Maximum epochs	50
Early-stopping patience	15
Warm-up	1 epoch
Batch size	4
Gradient accumulation	8 steps
Effective batch size	32
Numerical and geometric settings	
Input resolution	378×1246 pixels
Depth range	$[10^{-3}, 80]$ m
Depth-network mixed precision	FP16
Geometry and loss arithmetic	FP32
PoseNet warm-up	1 epoch for predicted-pose configurations; not used by OXTS-based A3

LiDAR anchor weight in the six final recipes	0
Checkpoint selection	
Validation metric	Pixel-weighted RMSE
Best checkpoint	Lowest validation RMSE

The settings in Table D.3 describe the recorded A3 run that was selected for the sequential Stage 9 experiment. This run used OXTS relative poses, so the PoseNet learning-rate and PoseNet warm-up settings apply only to the predicted-pose configurations listed later; they were not used to estimate the camera motion for A3. The A3 run was configured for 50 epochs and recorded 42 epochs. Its best-validation checkpoint was selected at epoch 27, with validation RMSE of 2.5035 m. Evaluation of that checkpoint gave an unscaled pixel-weighted test RMSE of 2.5266 m, AbsRel of 0.0575, and $\delta_1 = 0.9742$. This is the checkpoint transferred to the sequential supervised A4 stage described in Chapter 5.

The six final recipe definitions are grouped by their source of camera motion. Recipes A1, A3 and B1 use KITTI OXTS relative poses. Recipes C1, C3 and C5 use PoseNet-predicted poses. OXTS supplies motion information only; none of these six recipes uses LiDAR depth or a sparse-depth anchor as a training target. LiDAR depth is retained for validation, checkpoint selection and test evaluation.

TABLE D.4: Final SSL recipe definitions in the supplied notebook.

Recipe	**Pose source**	**Loss composition**
A1	OXTS	Photometric reprojection
A3	OXTS	Photometric reprojection $+10^{-3}$ smoothness +0.1 temporal warp consistency
B1	OXTS	Rigid-flow consistency
C1	PoseNet	Rigid-flow consistency
C3	PoseNet	Photometric L1 +0.1 edge-aware smoothness on rigid-flow magnitude
C5	PoseNet	Photometric L1 +0.1 feature-metric loss

In the saved notebook snapshot, only the B1 execution switch is active (the other recipe switches are false). The saved comparison output contains results for A1, A3, B1, C1 and C3; C5 is defined in the code but is not present in that saved comparison output. The current switch state and the set of previously saved output rows are therefore not identical. To reproduce the entire comparison, the corresponding recipe switches must be enabled deliberately.

D.3 SSL Registry and Execution Coverage

The current registry contains 23 entries, as listed in Appendix C, Table C.1. This total includes the hybrid/supervised reference entries, including `lidar_anchor_only`; it is therefore a registry count, not a count of pure self-supervised objectives. The stationary-object HPT results reported above cover the five stationary-object configurations only. The final recipe comparison covers the selected recipes listed in

Table D.4; these are separate experimental scopes and should not be described as if one run executed the full 23-entry registry.

D.4 Implementation Notes

- **PoseNet architecture:** paired RGB frames are concatenated into a six-channel input. A ResNet-18 encoder feeds a convolutional pose head with $512 \to 256 \to 256 \to 6$ channels. The six outputs represent axis-angle rotation and translation; the output is scaled by 0.1. The final convolutional layer bias is zero-initialised, while its weights retain their default non-zero initialisation.

- **PoseNet warm-up:** for predicted-pose configurations, PoseNet is warmed up for one epoch while the depth model is frozen. OXTS pose-oracle variants do not use this predicted-pose warm-up.

- **Differentiable warping:** depth is back-projected with $\mathbf{K}^{-1}$, transformed using the relative $SE(3)$ pose, projected with $\mathbf{K}$, and sampled with bilinear interpolation. The geometry, projection and loss calculations are explicitly performed in FP32 to avoid numerical problems in pixel-coordinate projection.

- **Augmentation:** geometric transformations are applied consistently across the target and source frames. Horizontal flipping also requires the corresponding camera-intrinsic and pose transformation. Random crop/resize is excluded from the final SSL augmentation because it changes the image coordinate system and principal point.

- **Threshold distinction:** the HPT configuration declares a variable named `TEMPORAL_WARP_THRESHOLD=0.05`, but the supplied temporal-warp loss computes the normalised projected-depth residual and does not use this variable as

a static-pixel gate. The 0.05 m threshold described in Appendix A belongs to the supervised TGM loss and must not be attributed to the SSL temporal-warp term.

- **Tracking setting:** `hungarian` is the tracker selected in the HPT configuration. The notebook comments explicitly identify it as a configuration choice, not a validated optimum; no held-out tracking evaluation is supplied.

References

[1] Jonathan T. Barron. A general and adaptive robust loss function. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[2] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003. URL https://doi.org/10.1109/ICIP.2016.7533003.

[3] Shariq Farooq Bhat, Ibraheem Alhashim, and Peter Wonka. AdaBins: Depth estimation using adaptive bins. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[4] Jia-Wang Bian, Zhichao Li, Naiyan Wang, Huangying Zhan, Chunhua Shen, Ming-Ming Cheng, and Ian Reid. Unsupervised scale-consistent depth and ego-motion learning from monocular video. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32. Curran Associates, Inc., 2019.

[5] Aleksei Bochkovskii, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R. Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.02073.

[6] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuScenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11618–11628, 2020. URL https://arxiv.org/abs/1903.11027.

[7] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2026. URL https://arxiv.org/abs/2511.16719.

[8] Sili Chen, Hengkai Guo, Shengnan Zhu, Feihu Zhang, Zilong Huang, Jiashi Feng, and Bingyi Kang. Video depth anything: Consistent depth estimation for super-long videos. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22831–22840. IEEE, 2025.

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[10] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural*

Information Processing Systems (NeurIPS), volume 27, 2014. URL https://arxiv.org/abs/1406.2283.

[11] Haoqiang Fan, Hao Su, and Leonidas J. Guibas. A point set generation network for 3d object reconstruction from a single image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 605–613, 2017.

[12] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[13] Ravi Garg, Vijay Kumar BG, Gustavo Carneiro, and Ian Reid. Unsupervised cnn for single view depth estimation: Geometry to the rescue. In *European Conference on Computer Vision (ECCV)*, pages 740–756. Springer, 2016. doi: 10.1007/978-3-319-46484-7_45.

[14] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the KITTI vision benchmark suite. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.

[15] Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun. Vision meets robotics: The KITTI dataset. *The International Journal of Robotics Research*, 32(11):1231–1237, 2013. doi: 10.1177/0278364913491297.

[16] Clément Godard, Oisin Mac Aodha, and Gabriel J. Brostow. Unsupervised monocular depth estimation with left-right consistency. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[17] Clément Godard, Oisin Mac Aodha, Michael Firman, and Gabriel J. Brostow. Digging into self-supervised monocular depth estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[18] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aart Bik, Erik Biesterfeld, Jeffrey Zhen, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

[19] Akhil Gurram. *Monocular Depth Estimation for Autonomous Driving*. PhD thesis, Universitat Autònoma de Barcelona, Barcelona, Spain, March 2022.

[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[21] Alex Kendall, Matthew Grimes, and Roberto Cipolla. Posenet: A convolutional network for real-time 6-dof camera relocalization. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015.

[22] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL https://arxiv.org/abs/1412.6980.

[23] Nitish Kumar, Abhilash S, Abhishek Thakur, Omkarthikeya Gopi, Ayush Dasgupta, Arpitha Algole, Bhaskar Anand, Venkata Satyanand Mutnuri, Santhosh Reddy, Naga Praveen Babu Mannam, Srikanth Saripalli, and Rajalakshmi Pachamuthu. Tiand: A multimodal dataset for autonomy on indian roads. In *2024 IEEE Intelligent Vehicles Symposium (IV)*, pages 688–694, 2024. doi: 10.1109/IV55156.2024.10588583.

[24] Iro Laina, Christian Rupprecht, Vasileios Belagiannis, Federico Tombari, and Nassir Navab. Deeper depth prediction with fully convolutional residual networks. In *Fourth International Conference on 3D Vision (3DV)*, 2016.

[25] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Bingyi Kang, Jiashi Feng, and Hengshuang Zhao. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025. URL https://arxiv.org/abs/2511.10647.

[26] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.

[27] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[28] Armin Masoumian, Hatem A Rashwan, Julián Cristiano, M Salman Asif, and Domenec Puig. Monocular depth estimation using deep learning: A review. *Sensors*, 22(14):5353, 2022.

[29] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.

[30] Luigi Piccinelli, Yung-Hsu Yang, Christos Sakaridis, Mattia Segu, Siyuan Li, Luc Van Gool, and Fisher Yu. UniDepth: Universal monocular metric depth estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL https://arxiv.org/abs/2403.18913.

[31] Luigi Piccinelli, Christos Sakaridis, Yung-Hsu Yang, Mattia Segu, Siyuan Li, Wim Abbeloos, and Luc Van Gool. UniDepthV2: Universal monocular metric depth estimation made simpler. *arXiv preprint arXiv:2502.20110*, 2025. URL https://arxiv.org/abs/2502.20110.

[32] Uchitha Rajapaksha, Ferdous Sohel, Hamid Laga, Dean Diepeveen, and Mohammed Bennamoun. Deep learning-based depth estimation methods from monocular image and videos: A comprehensive survey. *ACM computing surveys*, 56(12):1–51, 2024.

[33] René Ranftl, Aleksei Bochkovskii, Katrin Lasinger, and Vladlen Koltun. Vision transformers for dense prediction. *arXiv preprint arXiv:2103.13413*, 2021.

[34] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 44(3):1623–1637, 2022.

[35] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. URL https://arxiv.org/abs/2408.00714.

[36] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586. URL https://doi.org/10.1214/aoms/1177729586.

[37] Chang Shu, Kun Yu, Zhixiang Duan, and Kuiyuan Yang. Feature-metric loss for self-supervised learning of depth and egomotion. In *European Conference*

on Computer Vision (ECCV), pages 572–588, Cham, 2020. Springer. doi: 10.1007/978-3-030-58574-7_34.

[38] Leslie N Smith. A disciplined approach to neural network hyper-parameters: Part 1–learning rate, batch size, momentum, and weight decay. *arXiv preprint arXiv:1803.09820*, 2018.

[39] Girish Varma, Anbumani Subramanian, Anoop M. Namboodiri, Manmohan Chandraker, and C. V. Jawahar. IDD: A dataset for exploring problems of autonomous navigation in unconstrained environments. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1743–1751, 2019. doi: 10.1109/WACV.2019.00190. URL https://doi.org/10.1109/WACV.2019.00190.

[40] Linhan Wang, Kai Cheng, Shuo Lei, Shengkun Wang, Wei Yin, Chenyang Lei, Xiaoxiao Long, and Chang-Tien Lu. DC-Gaussian: Improving 3d gaussian splatting for reflective dash cam videos. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3169. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/b51b50262b492dd89bb9cd3105a46702-Abstract-Conference.html.

[41] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[42] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649, 2017. doi: 10.1109/ICIP.2017.8296962. URL https://doi.org/10.1109/ICIP.2017.8296962.

[43] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *arXiv preprint arXiv:2406.09414*, 2024. URL https://arxiv.org/abs/2406.09414.

[44] Wei Yin, Yifan Liu, Chunhua Shen, and Youliang Yan. Enforcing geometric constraints of virtual normal for depth prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[45] Zhichao Yin and Jianping Shi. GeoNet: Unsupervised learning of dense depth, optical flow and camera pose. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1983–1992, 2018. doi: 10.1109/CVPR.2018.00212.

[46] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022*, pages 1–21, 2022. URL https://arxiv.org/abs/2110.06864.

[47] Tinghui Zhou, Matthew Brown, Noah Snavely, and David G. Lowe. Unsupervised learning of depth and ego-motion from video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[48] Laurent Zwald and Sophie Lambert-Lacroix. The BerHu penalty and the grouped effect. In *arXiv preprint arXiv:1207.6868*, 2012.